

Practice Exam 1

Question

Model Answer

Section 1 — Multiple Choice

MC1. The Nyquist-Shannon sampling theorem states that to accurately reconstruct a signal you must sample at a rate that is:

- A) Equal to the highest frequency component in the signal
- B) At least twice the highest frequency component in the signal
- C) At least three times the highest frequency component in the signal
- D) At most half the highest frequency component in the signal

B

Nyquist: sample at $> 2\times$ the highest frequency. Sampling at exactly $2\times$ is the theoretical minimum; in practice you need a bit more.

MC2. In Elixir's GenServer, which callback *returns a reply* to the calling process?

- A) `handle_cast/2`
- B) `handle_info/2`
- C) `handle_call/3`
- D) `handle_continue/2`

C

`handle_call/3` returns `{:reply, value, new_state}`. The other callbacks return `{:noreply, ...}` and send no reply.

MC3. Which MQTT QoS level guarantees that a message is delivered *exactly once*?

- A) QoS 0 — at most once
- B) QoS 1 — at least once
- C) QoS 2 — exactly once
- D) QoS 3 — ordered delivery

C

QoS 2 uses a four-step handshake (PUBLISH → PUBREC → PUBREL → PUBCOMP). QoS 1 is *at least* once (duplicates possible).

MC4. A C union differs from a C struct in that:

- A) A union can only hold integer types
- B) All members of a union share the same memory location
- C) A union cannot be nested inside a struct
- D) A union always allocates memory for all its members simultaneously

B

All union members are laid out starting at the same memory address. Size = size of the largest member. Only one member is valid at a time.

MC5. In the IoT three-tier model, which tier is most appropriate for hard real-time control — e.g., detecting a threshold breach and triggering an actuator within milliseconds?

- A) Cloud
- B) Fog
- C) Edge
- D) All tiers are equally suitable

C

Edge processing is local, has no network dependency, and meets sub-millisecond latency requirements. Cloud round-trips add 10–100+ ms and fail when connectivity is lost.

MC6. Which LoRa spreading factor gives the <i>longest range</i> but the <i>lowest data rate</i>? • A) SF7 • B) SF9 • C) SF12 • D) SF15	C SF12 gives the longest range (~15 km line-of-sight) and slowest data rate (~250 bps). SF7 is fastest and shortest-range. There is no SF15.
MC7. In Elixir, the = operator is: • A) An assignment operator • B) A match operator • C) A structural equality test • D) A binding operator that always creates a new variable	B In Elixir = is the match operator. <code>x = 5</code> succeeds because <code>x</code> is unbound; <code>5 = x</code> also succeeds (<code>x</code> is already 5); <code>6 = x</code> raises a <code>MatchError</code>.
MC8. What is the primary function of the Low-Power Listening (LPL) layer in the TinyOS network stack? • A) Encrypting packets before transmission • B) Routing packets across multiple hops • C) Duty-cycling the radio to save energy • D) Retransmitting lost packets	C LPL periodically wakes the radio for a brief listen, then sleeps again. Senders prepend a long preamble so the receiver catches it. Dramatically reduces idle listening energy.
MC9. In RDF, a triple is composed of: • A) Class, property, value • B) Subject, predicate, object • C) Entity, attribute, relationship • D) Node, edge, label	B Every RDF statement is a triple: (subject, predicate, object). All three are identified by URIs (or literals for objects).
MC10. The <code>atten</code> field in an ESP32 ADC channel configuration controls: • A) The sampling frequency • B) The bit resolution of the conversion • C) The input voltage attenuation, which determines the measurable voltage range • D) The number of samples to average before returning a reading	C <code>ADC_ATTEN_DB_11</code> sets ~11 dB attenuation, widening the measurable range to ~0–3.9 V. Lower attenuation gives better resolution over a smaller voltage range.
MC11. Which statement about LoRaWAN duty cycle is correct? • A) Duty cycle limits are optional and only enforced by network operators • B) Duty cycle is a regulatory constraint limiting what fraction of time a device may transmit • C) Duty cycle only applies to LoRaWAN Class C devices • D) Higher spreading factors are exempt from duty cycle limits	B In the EU (and most regions) ISM-band devices must not exceed a duty cycle fraction (e.g., 1% on 868 MHz sub-bands). This is a legal requirement, not just a recommendation.
MC12. In a FreeRTOS project, what is the purpose of a binary semaphore?	B

- A) Transferring large blocks of data between tasks

A binary semaphore signals between tasks (e.g., ISR signals a task that data is ready) or protects a critical section. It is *not* for data transfer — use a queue for that.

- B) Signalling that an event has occurred or synchronising access to a shared resource
- C) Replacing the need for task priorities
- D) Allocating heap memory safely across tasks

Section 2 — Multiple Answer

MA1. Which of the following are valid MQTT QoS levels?

MA1 A, B, C**

- A) QoS 0 — fire and forget
- B) QoS 1 — at least once
- C) QoS 2 — exactly once
- D) QoS 3 — guaranteed ordering
- E) QoS 4 — acknowledged delivery

QoS 0, 1, and 2 are the only defined levels in the MQTT specification. QoS 3 and 4 do not exist.

MA2. Which properties make C suitable for use at the Edge tier? Select all that apply.

MA2 A, B, C, E**

- A) No garbage collector — timing is deterministic
- B) Compiles to bare-metal machine code with minimal runtime footprint
- C) Direct hardware access via pointer arithmetic and bit manipulation
- D) Strong memory-safety guarantees prevent buffer overflows
- E) Low-level control without unnecessary runtime overhead

D is wrong — C has *no* memory-safety guarantees; buffer overflows are a well-known C vulnerability. The other four are correct and are the primary reasons C dominates embedded development.

MA3. Under what conditions does GDPR apply to occupancy sensor data?

MA3 A, B, D**

- A) When the data can be used — alone or in combination — to identify a natural person
- B) When desk assignments link occupancy records to named employees
- C) Only when names are explicitly stored alongside the sensor readings
- D) When the building has exactly one person per monitored area, making records uniquely attributable
- E) Always, regardless of whether identification is realistically possible

C is wrong — data does not need to contain a name to be personal data; indirect identification suffices. E is wrong — GDPR applies when identification *is* realistically possible, not unconditionally. A, B, and D all describe situations where the data can identify a specific natural person.

MA4. Which of the following are Elixir/OTP supervision strategies?

MA4 A, B, C, E**

- A) `:one_for_one` — restart only the failed child
- B) `:one_for_all` — restart all children when one fails
- C) `:rest_for_one` — restart the failed child and all children started after it
- D) `:all_for_one` — restart all children regardless of which one failed
- E) `:simple_one_for_one` — for dynamically spawned homogeneous workers

D (`:all_for_one`) does not exist in OTP — `:one_for_all` is the closest real option. All four real strategies (`:one_for_one`, `:one_for_all`, `:rest_for_one`, `:simple_one_for_one`) are correct.

MA5. Which are consequences of sampling a signal *below* the Nyquist rate?

MA5 A, B, D**

• A) Aliasing — high-frequency components appear as artefacts at lower frequencies

• B) The original signal cannot be perfectly reconstructed from the samples

• C) The ADC hardware returns NaN or overflow values

• D) Frequencies above the Nyquist limit are mirrored (folded) into the sampled spectrum

• E) The anti-aliasing filter automatically corrects the problem in software

MA6. Which statements about LoRa and LoRaWAN are correct?

• A) LoRa is a physical-layer chirp spread-spectrum modulation technique

• B) LoRaWAN adds a MAC layer, network server, and application server on top of LoRa

• C) LoRa operates exclusively in licensed frequency bands

• D) Higher spreading factors give longer range at the cost of lower data rate and longer airtime

• E) LoRaWAN Class A devices can receive downlink messages at any time the gateway chooses

MA7. Which of the following are things that go into a C header file?

• A) Function declarations (prototypes)

• B) Type definitions (typedef, struct, enum, union)

• C) The full implementation body of functions

• D) Macro definitions (#define constants)

• E) Include guards (#ifndef / #define / #endif)

C is wrong — ADC hardware has no concept of Nyquist; it always returns a number. E is wrong — anti-aliasing filters are analogue hardware placed *before* the ADC; they don't correct sub-Nyquist sampling in software. A, B, and D all describe aliasing and its direct consequences.

MA6 A, B, D**

C is wrong — LoRa operates in *unlicensed* ISM bands (868 MHz EU, 915 MHz US). E is wrong — Class A devices open two short receive windows only *after* a transmission; the gateway cannot send downlink at will. Class C devices listen continuously.

MA7 A, B, D, E**

C is wrong — function implementations belong in .c files. Putting them in headers causes multiple-definition linker errors. A, B, D, and E are all correct.

Section 3 — Short Answer

SA1. What is a C union, and how does it differ from a struct?

A union is a C data type where all declared members occupy the *same* memory address. Its total size equals the size of its largest member. At any given moment only one member holds a valid value — writing to one member overwrites the others. A struct allocates separate, contiguous memory for every member, so all members can be valid simultaneously. Unions are used when a variable needs to represent one of several mutually exclusive types (e.g., a sensor reading that is either an integer count or a float voltage).

SA2. A FreeRTOS task function has this signature: void task_name(void *pvParameters). Why does it return void, and what is the purpose of the void * parameter?

The function returns void because FreeRTOS tasks are designed to run in an infinite loop and never return. If a task finishes, it must call vTaskDelete(NULL) to remove itself from the scheduler — returning from the function would leave the CPU with no valid return address and crash the system.

The void * parameter is a generic pointer. FreeRTOS's scheduler calls all tasks through the same uniform function pointer type. The void * allows the creator to pass any configuration struct or value at task creation time without changing the scheduler's interface.

SA3. To add a new <code>.c/.h</code> file pair to an ESP-IDF / FreeRTOS project, what must you change and why?	You must add the <code>.c</code> file to the <code>SRCs</code> list in the component's <code>CMakeLists.txt</code>, and ensure the directory containing the <code>.h</code> file is listed in <code>INCLUDE_DIRS</code> (also in <code>CMakeLists.txt</code>). The ESP-IDF build system uses CMake: only files listed in <code>SRCs</code> are compiled into the object archive, and only directories in <code>INCLUDE_DIRS</code> are searched when resolving <code>#include</code> directives. Without this, the linker cannot find the compiled object file and the compiler cannot find the header.
SA4. What is the benefit of persistent sessions in MQTT, and which QoS levels are affected?	With <code>cleanSession=false</code>, the broker retains the client's subscriptions and queues any QoS 1 or QoS 2 messages that arrive while the client is offline. When the client reconnects, the broker delivers the queued messages. This is valuable for power-constrained devices that sleep frequently: they do not miss updates published during their sleep period. QoS 0 messages are fire-and-forget and are <i>never</i> queued, even with a persistent session.
SA5. When should you choose raw LoRa over LoRaWAN, and when should you prefer LoRaWAN?	Choose raw LoRa when you need a custom or proprietary MAC layer, point-to-point links between two specific devices, or full control over duty-cycle scheduling and packet timing. Also appropriate for small closed systems (two to four devices) where the overhead of a full LoRaWAN stack is unjustified. Choose LoRaWAN when you need to connect many devices to managed infrastructure, want standardised security (AES-128 OTAA/ABP session keys), need to use an existing public network (e.g., The Things Network), or need the defined device classes (A/B/C) for controlled downlink.
SA6. Why is it hard to benchmark code running in a public cloud environment? Name at least two distinct reasons.	Noisy neighbours: Other tenants' workloads on the same physical host cause unpredictable CPU, cache, and memory-bandwidth contention. Results vary between identical benchmark runs. Non-deterministic hardware assignment: The VM may run on different physical machines across runs (different CPU generations, cache sizes, NUMA topology), making results incomparable without hardware pinning. Virtualisation jitter: The hypervisor's scheduler introduces latency spikes. CPU frequency scaling and thermal throttling are invisible to the tenant. Shared network: Network round-trip times and bandwidth fluctuate with aggregate load from all tenants.
SA7. What is SHACL, and why is it needed in an RDF-based IoT system?	SHACL (Shapes Constraint Language) is a W3C standard for defining validation rules over RDF graphs. For example: "every sensor node must have exactly one <code>brick:hasUnit</code> property" or "a room must have at least one <code>brick:isPartOf</code> relationship to a floor." It is needed because RDF is schema-free — anyone can add any triple, and without SHACL, downstream queries might silently receive malformed or incomplete data. SHACL acts as the data-quality enforcement layer that rejects or flags non-conforming triples before they enter the system.
SA8. In Elixir's GenServer, what is the difference between <code>call</code> and <code>cast</code>? Describe the scenario in which using <code>call</code> leads to a deadlock.	<code>call</code> is synchronous: the caller sends a message to the GenServer and blocks until <code>handle_call/3</code> returns <code>{:reply, value, new_state}</code>. The caller receives the value. Use it when the result is needed before proceeding. <code>cast</code> is asynchronous: the caller sends a message and immediately gets <code>:ok</code> back, without waiting. <code>handle_cast/2</code> returns <code>{:noreply, new_state}</code>. Use it for fire-and-forget updates.

SA9. What is NTP stratum, and what does the difference between stratum 1 and stratum 3 mean in practice?	Deadlock: If a GenServer calls <code>GenServer.call(self(), :something)</code> inside one of its own callbacks, it deadlocks. The <code>call</code> puts a message in the GenServer's own mailbox and blocks the process waiting for a reply — but the process is blocked and cannot process its own mailbox, so the reply never arrives. The same applies to two GenServers calling each other in a cycle (A calls B, B calls A). NTP stratum describes how many hops a clock is from a primary reference source. Stratum 0 devices are the reference clocks themselves — GPS receivers or atomic clocks — that are not directly on the network. Stratum 1 servers are machines physically connected to a stratum 0 device; they are typically accurate to within microseconds. Stratum 3 servers synchronise from stratum 2 servers, which synchronise from stratum 1 servers. Each hop adds propagation delay and jitter; stratum 3 clocks are typically accurate to a few milliseconds. For IoT applications that correlate sensor readings across devices (e.g., matching a temperature spike to a valve event), knowing your NTP stratum tells you the worst-case timestamp uncertainty.
SA10. What is the role of the Brick Schema in building IoT systems, and what kind of query does it enable that a plain sensor database cannot easily answer?	Brick Schema is a standard RDF ontology for describing building systems: sensors, rooms, floors, HVAC zones, and their relationships (<code>hasPart</code>, <code>isLocatedIn</code>, <code>feeds</code>, <code>hasUnit</code>). It enables queries that a flat sensor database cannot: for example, “find all temperature sensors in rooms on the south side of floor 3 that feed the Zone-4 HVAC system” — traversing structural relationships rather than filtering by column values. The same Brick model supports applications from thermostat control to GDPR auditing (which sensors monitor which spaces occupied by which people) without changing the underlying data.
SA11. What goes into an MQTT topic string, and what do the + and # wildcards do?	An MQTT topic is a /-separated hierarchy of strings, e.g. <code>building/floor3/room-U181/temperature</code>. Hierarchy levels convey structure (<code>building</code> → <code>floor</code> → <code>room</code> → <code>measurement type</code>). + matches exactly one level: <code>building/+/room-U181/temperature</code> matches any floor. # matches zero or more trailing levels: <code>building/floor3/#</code> matches all topics under floor 3 (all rooms, all measurement types). # may only appear at the end of a subscription string.
SA12. What is flash write endurance, and why does it matter for an edge IoT device that logs sensor data locally?	Flash memory cells can only be erased and rewritten a finite number of times — typically 10,000 to 100,000 write/erase cycles per cell before they fail. Additionally, flash must be erased in large blocks (sectors of 4–64 KB) before individual bytes can be rewritten, so even a 1-byte log entry may require erasing and rewriting an entire sector. For an edge device logging sensor data at 1 Hz continuously, a small flash chip could exhaust its endurance within months if the same sector is overwritten repeatedly. Mitigation strategies include wear levelling (distributing writes across all available sectors), circular log buffers, and reducing log frequency by transmitting instead of storing locally.
Section 4 — Detailed Answer	
DA1. Explain how noise in sensor sampling interacts with adaptive sampling and the radio budget. How does it cause the budget to be exceeded, and what can be done to mitigate this?	Adaptive sampling adjusts the sampling rate based on how quickly the measured quantity is changing: high rate during rapid change, low rate during stability. This is efficient when the signal truly is the source of variation.

Noise is a problem because random ADC noise looks identical to real signal change. An adaptive algorithm that interprets noise as genuine change will continuously keep the sampling rate high, even when the physical phenomenon is stable. This generates far more samples than the environment justifies — and correspondingly more radio transmissions.

Radio transmission is roughly 100,000–1,000,000× more energy-expensive per bit than a CPU instruction. Each unnecessary sample that gets transmitted represents a disproportionate energy cost. A radio budget set for, say, 10 transmissions per minute will be blown if noise inflates the effective rate to 60 per minute.

Mitigation: Apply a moving average or median filter to raw ADC values *before* feeding them to the adaptive algorithm. This smooths noise so only genuine environmental changes raise the sampling rate. Add a hysteresis threshold — only increase the rate if the filtered signal changes by more than a noise floor constant (e.g., 2× the measured noise amplitude). This decouples the noise floor from the adaptive trigger and keeps radio transmissions proportional to real physical change, not measurement artefacts.

DA2. Describe the DIKW (Data → Information → Knowledge → Wisdom) hierarchy using a concrete IoT sensor example. Explain what transformation occurs at each step, and why the distinction matters for system design.

Using a humidity sensor in room U181:

Data: 0.73 — a raw dimensionless number from the ADC output. No unit, no location, no time. Completely meaningless without context.

Information: "Sensor ID 42, room U181, relative humidity 73%, measured at 14:32:07 on 2026-02-03." Context added: physical unit (%RH), timestamp, location, sensor identity. The reading is now interpretable.

Knowledge: "Room U181 has been above 70% RH for the past 6 hours. Building standards require <70% RH to prevent mould growth; comfort guidelines recommend 40–60%." Context added: historical trend, domain rules, thresholds, and their implications. The reading is now actionable.

Wisdom: "The elevated humidity is caused by a faulty dehumidifier last serviced 18 months ago. Given that three ongoing meetings are scheduled in adjacent rooms, the optimal response is to increase ventilation now and schedule maintenance within 48 hours — not an emergency shutdown that would disrupt the meetings." Context added: root cause, business constraints, risk/trade-off reasoning, optimal timing.

Why it matters for design: A system that stores only raw data accumulates ~500 TB/year for a 10,000-building network — expensive to store and query. A system designed around DIKW processes close to the source: the Edge filters noise, the Fog annotates with context, and only knowledge-level aggregates (5-minute averages, threshold events) are shipped to the Cloud. This drastically reduces bandwidth, storage cost, and query latency.

DA3. Explain the tradeoffs between Edge, Fog, and Cloud processing tiers. For each tier, give one type of processing it is uniquely suited for and justify why.

Edge is the physical device: constrained RAM (256 KB–4 MB), limited CPU, no reliable network dependency. It is uniquely suited for hard real-time processing — e.g., a fire-alarm threshold check must fire within milliseconds regardless of network state, and a network round-trip of 50–200 ms would violate the timing requirement. Edge processing is also the only option that survives network loss (offline capability). It is unsuitable for cross-device aggregation or anything requiring historical data.

Fog is a regional gateway or local server with moderate compute. It is uniquely suited for aggregation across many edge devices without cloud latency — e.g., computing the mean temperature across 50 rooms in a building and triggering floor-level HVAC adjustments within 1–2 seconds. It reduces bandwidth to the cloud by pre-aggregating, and can maintain local models (e.g., occupancy inference) that require data from multiple sensors simultaneously. It is not suitable for large-scale ML training or long-term storage.

Cloud has unlimited compute and storage but 50–300 ms round-trip latency to the edge and requires internet connectivity. It is uniquely suited for training long-horizon ML models (e.g., predicting HVAC failure months in advance using years of historical sensor data), running complex multi-tenant analytics, and serving global dashboards. It is completely unsuitable for anything with sub-second latency requirements.

The guiding principle: process as close to the source as constraints allow. Only push up the tier hierarchy what the lower tier genuinely cannot handle.

DA4. In Elixir, actors can communicate in two ways. What are they, and why should you choose one over the other?

In Elixir, GenServer processes communicate via message passing. There are two patterns:

`GenServer.call/2` sends a message to the server and *blocks the caller* until the server processes it and returns a reply via `handle_call/3` → `{:reply, result, new_state}`. The call is synchronous — the caller knows the operation completed and receives the result. Use `call` when: the caller needs the return value before continuing, or when you need to ensure the operation completed (e.g., reading a registry entry, performing an atomic check-and-modify).

`GenServer.cast/2` sends a message and immediately returns `:ok` to the caller without waiting for any reply. The server handles it via `handle_cast/2` → `{:noreply, new_state}`. Use `cast` when: the caller does not need a result, the operation is a fire-and-forget notification or state update (e.g., logging an event, incrementing a counter), or when you want to avoid blocking the caller and risking timeout.

Key rule: prefer `cast` for updates and events; prefer `call` for queries where the result is needed. Overusing `call` risks timeout under load.

	Deadlock with call: If a GenServer calls <code>GenServer.call(self(), :msg)</code> inside one of its own callbacks, it deadlocks immediately. The <code>call</code> blocks the process waiting for a reply — but the process is now blocked and cannot service its own mailbox, so the reply will never arrive. The same applies to circular chains: if A calls B and B attempts to call A, A is blocked waiting for B, and B is blocked waiting for A. Use <code>cast</code>, <code>send</code>, or <code>handle_continue</code> for self-messaging.
DA5. Explain beamforming: what problem it solves, how it works physically, and what benefits it provides for IoT deployments.	A standard omnidirectional antenna radiates power equally in all directions. In a deployment with one intended receiver, most of that energy is wasted — and it also increases interference with adjacent devices. Beamforming solves this by concentrating radio energy in the direction of the target. How it works: A beamforming system uses an antenna array (two or more antenna elements separated by roughly half a wavelength). The transmitter introduces controlled phase offsets (time delays) between the signals sent by each element. Because of wave superposition, the signals from all elements constructively interfere in the direction of the target (waves add up) and destructively interfere in other directions (waves cancel). The result is a focused "beam" directed at the receiver. Benefits for IoT: 1. Range: Concentrating the same transmit power in one direction significantly extends effective range — the same EIRP budget reaches farther. 2. Energy efficiency: Achieving the same link budget requires less total transmit power, which is critical for battery-powered IoT nodes. 3. Interference reduction: Devices in non-beam directions receive much weaker signals, reducing co-channel interference in dense deployments. 4. Spatial multiplexing (MU-MIMO): Multiple independent beams can serve multiple clients simultaneously on the same frequency, increasing network capacity. Beamforming is standard in 802.11ac/ax (WiFi 5/6) and 5G NR. For IoT it is most relevant in dense smart-building or factory deployments where interference and range are both limiting factors.
DA6. Explain what the CAP theorem says and what it means for the design of a large-scale IoT time-series storage system.	The CAP theorem states that a distributed data system can guarantee at most two of three properties simultaneously: Consistency (every read sees the most recent write), Availability (every request receives a response), and Partition Tolerance (the system continues operating despite network partitions separating nodes). Since network partitions are a physical reality in any distributed system (including IoT backends), partition tolerance is non-negotiable. The real choice is between Consistency and Availability when a partition occurs. For a large-scale IoT time-series storage system, the practical choice is typically AP (Available + Partition-tolerant), accepting eventual consistency. Here is why: • Sensor readings arrive continuously from thousands of nodes. If we require strict consistency (CP), a network partition could cause write requests to be rejected entirely — meaning we lose readings during the partition period. For time-series data, a lost reading is irrecoverable; a slightly stale read is usually acceptable.

• Queries against time-series databases (e.g., "show me the last 24 hours of temperature") tolerate eventually-consistent results because analysts are reviewing historical trends, not making real-time safety decisions based on a write they just performed.

• Ingest availability is typically more important than read consistency for IoT: the system must always accept new measurements.

The consequence: IoT time-series databases (InfluxDB, TimescaleDB, Cassandra) are typically designed as AP systems. Consistency is sacrificed at partition boundaries and repaired asynchronously. This is acceptable for analytics but means that alerting systems that require strict consistency should use separate, strongly-consistent stores for critical threshold state.

Section 5 — Design Question

Question 15 — Smart Greenhouse

(answer not found)

A research university wants to deploy a smart monitoring system across **20 greenhouses** on a single campus (roughly 1 km²). Each greenhouse contains **40 sensor nodes**, each measuring temperature, humidity, CO₂, and soil moisture. Requirements:

- Staff must be **alerted within 5 seconds** if any sensor leaves its safe operating range.
- All readings must be **logged to a central database** for later analysis (years of retention).
- Researchers need a **web dashboard** to visualise historical trends per greenhouse, per sensor type.

It is up to you to identify and argue for a good balance of the involved tradeoffs.

Section 3 — Short Answer

Subquestion 15.1 (short answer)

How frequently should each sensor type sample, and does any measurement type require a higher rate than the others?

Temperature, humidity, CO₂, and soil moisture all change slowly relative to human timescales:

- **Temperature:** 1-minute intervals are more than sufficient. Thermal time constants in a greenhouse are on the order of minutes.
- **Humidity:** 1 minute.
- **CO₂:** 1 minute; CO₂ levels respond to plant photosynthesis and ventilation, which operate on minute timescales.
- **Soil moisture:** Every 5–10 minutes is adequate; soil moisture changes over hours, not seconds.

None of these measurements require a higher sampling rate. The 5-second alert requirement is a *latency* requirement on the data pipeline, not a sampling frequency requirement — a 1-minute sample can still trigger an alert within 5 seconds of the reading being taken.

Section 4 — Detailed Answer

Subquestion 15.2 (detailed answer)

What should the network topology look like from sensor node to cloud database, and which network technologies (at the relevant layers of the stack) should be used? Justify your choices.

Sensor node → Greenhouse gateway (Edge → Fog):

Each greenhouse has 40 nodes within a small space (~30 m²). A low-power mesh protocol is appropriate: **Zigbee** (IEEE 802.15.4, 2.4 GHz) or **Thread** (IPv6 mesh over 802.15.4). Both support ~40–100 nodes per network, have very low power consumption (suitable for battery-powered nodes), and provide reliable mesh routing so nodes do not need line-of-sight to the gateway. Each greenhouse runs one coordinator/border router acting as the Fog gateway.

Greenhouse gateway → Campus server (Fog → local cloud):

With 20 greenhouses across 1 km², a wired **Ethernet/PoE** backbone is the most reliable option. It eliminates RF interference, provides deterministic latency, and each gateway can draw power from the Ethernet cable. If cabling is not feasible, campus **WiFi** (802.11ac/ax) can serve as an alternative, but requires careful RF planning to avoid interference between greenhouse gateways. LoRaWAN would be too bandwidth-limited for 40 sensors × 4 measurements × 60 readings/minute per greenhouse.

Campus server → Cloud database:

Standard **MQTT over TLS** to a cloud broker (e.g., AWS IoT Core or self-hosted Mosquitto), with **QoS 1** to ensure readings survive transient network blips. The cloud backend writes to a time-series database (InfluxDB or TimescaleDB). The web dashboard reads from the same database via a REST or GraphQL API.

Protocol stack summary:

- Sensor → gateway: Zigbee / Thread (IEEE 802.15.4 physical + mesh MAC)
- Gateway → campus: Ethernet/IP + MQTT
- Campus → cloud: MQTT over TLS/IP

Section 3 — Short Answer

Subquestion 15.3 (short answer)

Where in the topology should the real-time alert logic run, and why?

The real-time alerting logic (5-second requirement) must run at the **greenhouse gateway (Fog tier)**, not in the cloud. A round-trip to the cloud (sensor → gateway → campus server → cloud → alert) introduces at minimum 500 ms to several seconds of latency depending on network conditions. Cloud processing queuing and broker delays easily add another 5–30 seconds. This violates the 5-second requirement.

The gateway receives all 40 nodes' readings from its greenhouse directly via the local mesh, applies threshold rules in-memory, and fires an alert (via MQTT to staff devices or a local alarm) within 1–2 seconds of the reading arriving. The cloud continues to receive all readings for historical storage regardless of whether an alert was triggered.

Threshold checking *could* also happen at the sensor node itself (Edge), which would be even faster. However, some alert conditions may involve comparing readings across multiple nodes in the same greenhouse (e.g., "CO₂ is high and humidity is also high"), which the individual sensor node cannot do. The gateway is the correct tier.

Subquestion 15.4 (short answer)

Design an MQTT topic hierarchy that supports both per-sensor subscriptions and wildcard subscriptions across all greenhouses. Give examples.

Proposed structure:

```
greenhouse/{campus_id}/{greenhouse_id}/sensor/{node_id}/{measurement}
```

Examples:

```
greenhouse/odense/gh-07/sensor/node-23/temperature
greenhouse/odense/gh-07/sensor/node-23/humidity
```

	greenhouse/odense/gh-07/sensor/node-23/co2
	greenhouse/odense/gh-07/sensor/node-23/soil_moisture
	Useful wildcard subscriptions:
	• All data from greenhouse 7: greenhouse/odense/gh-07/# • All temperature readings across all greenhouses: greenhouse/odense/+/sensor/+/temperature • Everything on campus: greenhouse/odense/# • All CO₂ readings campus-wide for the alert service: greenhouse/+/+/sensor/+/co2
	This hierarchy supports both narrow per-sensor subscriptions (the historian subscribes to specific nodes) and broad cross-greenhouse subscriptions (the alerting service subscribes to all CO₂ readings). Adding a new measurement type requires no schema change — just add a new topic level.

Practice Exam 2	
Question	Model Answer
Section 1 — Multiple Choice	
MC1. The Elixir pipe operator <code> ></code> passes the result of the left-hand expression as: • A) The last argument to the right-hand function • B) The first argument to the right-hand function • C) A keyword argument named <code>:input</code> to the right-hand function • D) A return value that the right-hand function may or may not use	B ,
MC2. In BLE (Bluetooth Low Energy), which role broadcasts advertising packets so that other devices can discover it? • A) The GATT server • B) The central • C) The peripheral • D) The L2CAP coordinator	C In BLE, the peripheral advertises; the central scans and initiates connections. A GATT server is a role at a higher layer, not the advertising role.
MC3. What is aliasing in the context of signal sampling? • A) Giving a sensor a human-readable name in the metadata model • B) A phenomenon where high-frequency signal components appear as false lower-frequency artefacts after sampling • C) The process of averaging multiple ADC readings to reduce noise • D) The mapping of a physical GPIO pin to a software interrupt handler	B Aliasing occurs when a signal is sampled below the Nyquist rate; frequencies above the Nyquist limit fold back into the spectrum and appear as false lower-frequency artefacts.
MC4. In Elixir, the pin operator <code>^</code> is used to: • A) Dereference a pointer to a process mailbox • B) Match against the <i>current value</i> of an already-bound variable rather than rebinding it • C) Raise a variable to a power	B <code>^x</code> matches against the current binding of <code>x</code> without rebinding it. Without <code>^</code>, <code>x = new_value</code> in a match would simply rebind <code>x</code>.

• D) Declare a module-level constant that cannot be rebound	
MC5. What is the role of the MQTT broker? • A) It runs on each IoT device and manages local topic subscriptions • B) It is a central server that receives messages from publishers and routes them to matching subscribers • C) It encrypts MQTT payloads end-to-end between publisher and subscriber • D) It stores all historical topic data in a time-series database	B The broker is the central message router: it receives PUBLISH packets and forwards them to all clients with matching subscriptions. It does not run on devices, does not encrypt payloads end-to-end, and is not a time-series database.
MC6. What distinguishes a LoRaWAN Class C device from a Class A device? • A) Class C uses a higher spreading factor • B) Class C devices listen continuously for downlink messages (except when transmitting), whereas Class A only opens two short receive windows after each uplink • C) Class C devices can only communicate with proprietary gateways • D) Class C requires a licensed frequency band	B Class C devices keep their receiver open continuously (except during transmission), enabling the gateway to push downlink at any time. Class A opens two brief windows only after each uplink.
MC7. In a C struct, member access through a pointer uses the -> operator. Why does this operator exist? • A) It is syntactic sugar for (*ptr).member — dereferencing the pointer and then accessing the member • B) It transfers ownership of the struct to the callee • C) It performs a null-pointer check before accessing the member • D) It is equivalent to &ptr.member — taking the address of the member	A ptr->member is syntactic sugar for (*ptr).member — first dereference the pointer to get the struct, then access the member. No null-check is performed.
MC8. In the DIKW hierarchy, what transformation converts <i>information</i> into <i>knowledge</i>? • A) Adding a unit and timestamp to a raw value • B) Applying a domain rule or pattern that gives the information predictive or explanatory power • C) Storing the information in a relational database • D) Transmitting the information to the cloud tier	B Information → Knowledge requires a domain rule or pattern: "relative humidity >70% promotes mould" converts the fact "73% RH" into actionable knowledge. Adding units/timestamps creates <i>information</i> from <i>data</i>.
MC9. What is the purpose of SPARQL in an IoT metadata system? • A) Streaming sensor readings in real time over WebSocket • B) Querying RDF graphs to retrieve resources and relationships that match a pattern • C) Validating RDF triples against a SHACL schema • D) Serialising sensor readings into the Turtle RDF format	B SPARQL (SPARQL Protocol and RDF Query Language) queries RDF graphs using triple-pattern matching. It does not stream sensor data, validate schemas (that is SHACL), or serialise data (that is Turtle/JSON-LD).
MC10. In FreeRTOS, what happens if a task function reaches its closing } and returns normally?	C

• A) The task is automatically deleted and its stack freed • B) The task is suspended until the scheduler restarts it • C) Undefined behaviour — the return address is invalid, typically causing a crash or hang • D) The task restarts from the beginning of the function	FreeRTOS tasks are designed to never return. The function pointer passed to <code>xTaskCreate</code> has no valid return address in the call stack. Returning causes undefined behaviour — typically a jump to address 0 or an unhandled exception. Call <code>vTaskDelete(NULL)</code> to terminate.
MC11. What is a confounding factor in a performance evaluation experiment? • A) A metric that is difficult to measure accurately • B) An uncontrolled variable that influences the result and makes it hard to attribute the outcome to the factor under test • C) A statistical outlier that should be removed from the dataset • D) The target variable whose value you are trying to optimise	B A confounding factor is an uncontrolled variable that correlates with both the independent variable and the outcome, making it impossible to determine whether the measured effect is caused by the factor under test or the confounder.
MC12. In the context of IoT radio communication, what does "duty cycle" refer to? • A) The fraction of time a radio transmitter is actively transmitting relative to total time • B) The number of bits transmitted per second • C) The ratio of data payload bytes to total packet bytes • D) The number of retransmissions allowed per message	A Duty cycle = time transmitting / total time. Regulatory bodies (e.g., ETSI for EU 868 MHz) cap this to limit interference — e.g., 1% duty cycle means at most 36 seconds of transmission per hour.
Section 2 — Multiple Answer	
MA1. Which of the following are characteristics of the Erlang BEAM runtime that make it suitable for concurrent IoT applications? • A) Lightweight processes — millions can run concurrently with a few KB of stack each • B) No shared memory between processes — all communication is via message passing • C) Stop-the-world garbage collection pauses affect the entire VM • D) Per-process garbage collection — one process's GC does not pause others • E) Preemptive scheduling — no single process can starve others	MA1 A, B, D, E** C is wrong — BEAM uses <i>per-process</i> GC, so one process's garbage collection does not pause others. This is a key advantage over JVM-style stop-the-world GC.
MA2. Which are valid consequences of storing <i>more</i> data than you have a stated purpose for, in an IoT system subject to GDPR? • A) Violation of the data minimisation principle • B) Violation of the purpose limitation principle if the data is later used for a different purpose • C) Increased operational cost (storage, ingestion, query) • D) Automatic GDPR compliance because more data enables better anonymisation	MA2 A, B, C, E** D is wrong — storing more data does not automatically improve anonymisation; it typically <i>increases</i> re-identification risk. A, B, C, and E are all real consequences under GDPR and operational reality.

• E) Regulatory penalties if the data includes personal information retained beyond the stated retention period	
MA3. Which wireless technologies operate (at least partially) in the unlicensed 2.4 GHz ISM band? • A) WiFi (IEEE 802.11) • B) Bluetooth / BLE • C) Zigbee (IEEE 802.15.4) • D) NB-IoT • E) LoRa (868 MHz EU band)	MA3 A, B, C** D is wrong — NB-IoT uses licensed LTE bands. E is wrong — LoRa in the EU uses the 868 MHz ISM band (915 MHz in the US). WiFi, Bluetooth/BLE, and Zigbee all operate in 2.4 GHz.
MA4. Which of the following describe valid use cases for fog-tier processing? • A) Aggregating sensor readings from 50 nodes in a building before forwarding to the cloud, to reduce bandwidth • B) Training a neural network on five years of historical building data • C) Running an occupancy model that requires readings from multiple rooms simultaneously • D) Serving a global analytics dashboard to hundreds of researchers worldwide • E) Local MQTT brokering between devices in the same building without requiring internet access	MA4 A, C, E** B is wrong — training on years of historical data requires cloud-scale compute and storage. D is wrong — serving a global dashboard to hundreds of users is a cloud responsibility. A, C, and E are all genuinely fog-tier tasks (regional aggregation, multi-sensor local inference, local brokering).
MA5. Which statements about Elixir's GenServer are correct? • A) <code>GenServer.call/2</code> blocks the calling process until a reply is received • B) <code>GenServer.cast/2</code> blocks the calling process until the server confirms receipt • C) <code>handle_call/3</code> must return a tuple of the form <code>{:reply, response, new_state}</code> • D) A GenServer process holds its state in the return value of each callback — there is no mutable variable • E) A GenServer can be registered under a name so it can be looked up without knowing its PID	MA5 A, C, D, E** B is wrong — <code>cast</code> is <i>asynchronous</i> and returns <code>:ok</code> immediately without waiting for the server to process the message. A, C, D, and E are all correct.
MA6. Which of the following are reasons why the JVM (Java) is unsuitable for Edge-tier IoT devices? • A) The JVM requires 50–200 MB of RAM at startup; most MCUs have 256 KB–4 MB • B) Stop-the-world garbage collector pauses are non-deterministic and can violate real-time constraints • C) Java cannot express bitwise operations • D) JVM cold-start time is measured in seconds, unsuitable for devices that must boot and react in milliseconds • E) Java does not support networking	MA6 A, B, D** C is wrong — Java fully supports bitwise operations (<code>&</code>, <code> </code>, <code>^</code>, <code><<</code>, <code>>></code>). E is wrong — Java has extensive networking libraries. A, B, and D are the three core reasons the JVM is unsuitable for MCU deployment.
MA7. Which of the following are valid sources of device identity in an IoT deployment?	MA7 A, B, C, D**

- A) A hardware serial number burned into the MCU or a dedicated ID chip at manufacture
- B) A UUID assigned by the user during physical installation via a mobile app
- C) A GPS coordinate hardcoded at deployment time (location-derived identity)
- D) The MAC address of the device's radio module
- E) The cloud platform always generates identity — devices cannot self-identify

E is wrong — devices *can* self-identify using hardware serial numbers, manufacturer-burned UUIDs, or GPS-derived location. Cloud-generated identity is one option, not the only one. A, B, C, and D are all valid sources described in the course.

Section 3 — Short Answer

SA1. What is a C pointer, and what does the dereference operator `*` do when applied to it?

A pointer is a variable that stores a memory address rather than a value directly. Declaring `int *p` means "p holds the address of an int." The dereference operator `*p` follows the address stored in p and returns the value stored at that address. Assigning `*p = 42` writes 42 to whatever memory location p points to. Pointers are used to pass large data structures to functions without copying them, to build dynamic data structures (linked lists, trees), and to access hardware registers directly at known memory addresses.

SA2. What is the Elixir Registry module used for, and what problem does it solve compared to using raw PIDs?

Registry is a process name registry: it lets you register a process under a logical name (e.g., `{:room, "U181"}`) and look it up without knowing its PID. The problem it solves: PIDs are ephemeral — if a process crashes and is restarted by a supervisor, its PID changes. Any code holding the old PID can no longer reach it. With Registry, you look up the current PID by name at call time, so restarts are transparent to callers. It supports `unique` mode (one process per key) and `duplicate` mode (many processes per key, for pub-sub fan-out).

SA3. What is the difference between Elixir's Enum (eager) and Stream (lazy) modules? When would you prefer one over the other?

Enum functions process the entire collection immediately, producing an intermediate list at each pipeline step. Stream functions are lazy — they compose a description of the transformation and evaluate it one element at a time only when explicitly consumed (e.g., by `Enum.to_list/1`).

Prefer Enum when collections are small and you need the full result immediately. Prefer Stream when processing large or potentially infinite data sources (files, sensor streams) where holding the entire intermediate collection in memory would be wasteful — Stream maintains constant memory regardless of collection size.

SA4. What is the distinction between a "smart" device and an "intelligent" device in the context of this course?

A **smart** device reacts to predefined rules or remote commands. It has network connectivity and programmable state but does not reason about *why* or *whether* it should act. A smart bulb that turns on when told to, or follows a schedule, is smart.

An **intelligent** device reasons, adapts, and learns autonomously without explicit reprogramming. It would model context, infer preferences from past behaviour, and update its behaviour based on experience. The distinction matters because "smart" is a marketing term that conflates connectivity with capability; the course treats intelligence as requiring inference, adaptation, and learning — properties that generally require Fog or Cloud processing beyond what an individual device can supply.

SA5. What does `OPTIONAL MATCH` do in a Cypher query (Neo4j), and why is it necessary in building metadata queries?

`OPTIONAL MATCH` returns all rows from the preceding match even when the optional pattern is absent — absent relationships produce `null` values rather than dropping the row entirely. It is the Cypher equivalent of a SQL `LEFT JOIN`.

	It is necessary in building metadata queries because not all rooms have the same sensors installed. A query for "all rooms and their thermostat setpoints" would return zero rows for rooms without a thermostat sensor if a regular <code>MATCH</code> were used. <code>OPTIONAL MATCH</code> on the thermostat relationship returns all rooms, with <code>null</code> for rooms that lack one, enabling a complete picture.
SA6. What is a DMA controller, and how does it benefit a microcontroller collecting ADC samples?	A DMA (Direct Memory Access) controller is a hardware unit that transfers data between peripherals (e.g., an ADC, UART, SPI bus) and memory without involving the CPU for each byte. The CPU initiates the transfer, then continues executing other code; the DMA controller handles the byte-by-byte copying in the background and signals the CPU with an interrupt when the transfer is complete. For ADC sampling, DMA allows the hardware to continuously fill a buffer with ADC readings at the configured sample rate — even at kilohertz rates — while the CPU processes earlier samples or enters a low-power idle state. Without DMA, the CPU would have to poll or service an interrupt for every single sample, burning CPU cycles and preventing sleep.
SA7. What is the LMT86 temperature sensor's transfer function, and what does it tell you about how to interpret its output?	The LMT86 is an analogue temperature sensor that outputs a voltage proportional to temperature. Its transfer function is approximately linear: $V_{\text{out}} \text{ (mV)} = -10.9 \times T(^{\circ}\text{C}) + 1777.3$ This means the output <i>decreases</i> as temperature <i>increases</i> (negative temperature coefficient). To convert the raw ADC reading to Celsius, you rearrange the formula: $T = (1777.3 - V_{\text{out}}) / 10.9$. The transfer function is needed because the ADC gives you a dimensionless digital number; without the sensor's transfer function you cannot convert that number to a meaningful temperature in physical units.
SA8. What is "adaptive sampling," and what specific condition should trigger an increase in the sampling rate?	Adaptive sampling dynamically adjusts the sampling rate based on signal behaviour rather than using a fixed rate. The rate should increase when the rate of change of the measured signal exceeds a threshold — i.e., when consecutive samples differ by more than a defined amount, indicating the phenomenon is evolving rapidly and a higher rate is needed to track it accurately. The rate should decrease when the signal is stable (differences below threshold) to save energy and radio budget. A hysteresis band is commonly added to prevent rapid switching between rates at the threshold boundary.
SA9. In C, what is the difference between passing a struct to a function <i>by value</i> and passing it <i>by pointer</i>? When should you prefer each?	Passing a struct by value copies the entire struct onto the callee's stack. Any modifications inside the function affect only the copy — the caller's original is unchanged. This is safe but expensive for large structs (copying dozens or hundreds of bytes per call). Passing a struct by pointer (<code>void f(MyStruct *s)</code>) passes only the address (4 or 8 bytes). The function operates directly on the original struct in memory; modifications are visible to the caller. This is efficient for large structs but requires care — unintended modifications will corrupt the caller's data. Use by-value for small structs where immutability is desired; use by-pointer for large structs or when the function must modify the caller's data.
SA10. What is a LoRaWAN network server, and what two responsibilities does it have that a raw LoRa gateway does not?	The LoRaWAN network server (LNS) sits between the physical gateways and the application server. It has two key responsibilities that a raw LoRa gateway lacks:

	1. De-duplication and routing: Multiple gateways may receive the same uplink packet (they all hear the same transmission). The LNS deduplicates these copies and forwards only one to the application server. A raw LoRa gateway simply forwards everything it hears. 2. MAC layer management: The LNS handles the LoRaWAN MAC protocol — adaptive data rate (ADR), downlink scheduling, session key management (OTAA/ABP join), and device authentication. A raw LoRa gateway is transparent at the physical layer and has no awareness of these MAC functions.
SA11. What is the Elixir <code>spawn/1</code> function, and how does a spawned process communicate its result back to the parent?	<code>spawn(fn -> ... end)</code> creates a new lightweight BEAM process that runs the given function concurrently. It returns the PID of the new process immediately — the spawning process does not wait for the child to finish. To communicate the result back, the child explicitly sends a message to the parent's PID: <code>send(parent_pid, {:result, computed_value})</code>. The parent calls <code>receive do {:result, val} -> val end</code> to collect it. This is the fork-join pattern: spawn <i>N</i> workers, each does computation and sends back a result, the parent collects all results with <code>receive</code>. The parent's PID is typically captured in a variable before spawning and passed into the closure.
SA12. In performance evaluation, what is the difference between <i>accuracy</i> and <i>precision</i> of a measurement? Give an IoT example of a measurement that is precise but inaccurate.	Accuracy is how close a measurement is to the true value. Precision is how repeatable a measurement is — how tightly clustered repeated measurements are around their mean. A measurement can be precise but inaccurate: a miscalibrated temperature sensor that consistently reads 23.4°C when the true temperature is 20.0°C. Every reading is precise (very little variance between samples) but systematically wrong. In IoT, a common source of this error is an ADC with a reference voltage offset — the readings are stable and repeatable but biased by a fixed error from the true physical value.

Section 4 — Detailed Answer

DA1. Describe the BLE protocol stack: what GAP and GATT each define, how they interact, and how a BLE sensor device uses them to expose temperature readings to a mobile app.	BLE's protocol stack separates device discovery/connection (GAP) from data exchange (GATT). GAP (Generic Access Profile) defines roles and procedures for devices to find each other and establish connections. The peripheral broadcasts small advertising packets at regular intervals (typically 20 ms–10 s) on three dedicated advertising channels. The central (e.g., a smartphone) scans for these packets and initiates a connection. GAP also governs security pairing and bonding procedures. GATT (Generic Attribute Profile) defines how data is structured and exchanged once a connection exists. Data is organised into a hierarchy: Services (logical groupings, e.g., "Environmental Sensing Service") contain Characteristics (individual data points, e.g., "Temperature Measurement"), each with a UUID, a value, and optional Descriptors (metadata about the characteristic, e.g., units, update interval). The GATT server (the sensor device) hosts this hierarchy; the GATT client (the phone app) reads or subscribes to characteristics.
--	---

DA2. Describe Elixir's supervision tree: what a supervisor does, how restart strategies work, and why this architecture enables fault-tolerant IoT backend services without global state corruption.

Interaction for a temperature sensor: The sensor (peripheral) advertises via GAP, announcing it has an Environmental Sensing Service. The phone (central) connects via GAP, then uses GATT to discover the Temperature Characteristic and enable notifications. The sensor sends a GATT NOTIFY packet each time a new reading is taken, without the phone needing to poll. The phone decodes the characteristic value using the GATT specification's standard format (16-bit signed integer, 0.01°C resolution) and displays it.

Elixir's OTP supervision tree is the primary mechanism for building fault-tolerant services. It consists of **supervisor processes** arranged in a tree, with **worker processes** (including GenServers) as leaves.

A **Supervisor** monitors its children. If a child process crashes (raises an unhandled exception, exits abnormally), the supervisor restarts it automatically according to its configured **restart strategy**:

- **:one_for_one** — restart only the failed child; other children are unaffected. Used when workers are independent.
- **:one_for_all** — restart all children when one fails. Used when workers share state that becomes invalid if any one of them fails.
- **:rest_for_one** — restart the failed child and all children started after it. Used when later children depend on earlier ones.

Supervisors also enforce a **max restart frequency** (e.g., 3 crashes in 5 seconds): if a child keeps crashing, the supervisor gives up and itself crashes, propagating the failure up to *its* supervisor. This "let it crash" philosophy means workers never need defensive error-handling for unexpected states — they simply crash and let the supervisor restore a known-good state.

Why this matters for IoT backends: A GenServer handling MQTT ingestion for Room U181 might crash if it receives a malformed payload. With a supervision tree, the crash is isolated — other room processes continue unaffected — and the crashed process is restarted within milliseconds, resuming from its initial state. No global lock is corrupted, no other process is affected. This is the foundation of "nine nines" (99.9999999%) availability in Erlang/Elixir systems.

DA3. Explain the energy budget of an IoT edge device. What are the dominant energy consumers, what order-of-magnitude differences exist between them, and how do duty-cycling and adaptive strategies help stay within budget?

An edge device's energy budget defines how much energy it can consume over its operational lifetime (bounded by battery capacity or energy harvesting rate). The dominant consumers are, in order of magnitude:

- **Radio transmission:** Roughly 10–100 mA during TX; on the order of 1–10 mJ per packet. This is **100,000–1,000,000× more expensive per bit** than a CPU instruction.
- **Radio reception / idle listening:** Still significant — 5–20 mA even while just listening.
- **ADC sampling:** Moderate — a few mA for the microseconds of a conversion.
- **CPU execution:** 1–10 mA at full speed, but instructions are cheap in energy/operation terms.
- **Deep sleep:** Microamps. The dominant strategy for battery life is to be asleep as much as possible.

Duty cycling: The device spends most of its time in deep sleep, wakes on a timer or interrupt, takes a reading, and potentially transmits — then returns to sleep. If transmission is needed once per minute and the radio is on for 100 ms, the radio duty cycle is 0.17%, drastically reducing average current draw.

Adaptive strategies: Transmit only when the signal changes meaningfully (adaptive sampling reduces unnecessary transmissions). Aggregate multiple readings into one packet to reduce radio on-time. Use the lowest spreading factor (shortest airtime) that still delivers packets reliably.

Worked example: 3.7 V, 1200 mAh battery. 20 mA during TX for 200 ms every 60 s → average current from radio = $20 \text{ mA} \times (0.2/60) \approx 0.067 \text{ mA}$. Adding 3 μA sleep current gives $\sim 0.07 \text{ mA}$ average. Battery life $\approx 1200/0.07 \approx 700$ days. But if noise causes adaptive sampling to transmit every 5 s instead, average radio current jumps to $\sim 0.8 \text{ mA}$ → battery life drops to ~ 62 days.

DA4. Explain what confounding factors are in the context of IoT performance evaluation. Give two concrete examples from benchmarking IoT systems, and describe how experimental design can control for them.

A confounding factor is an uncontrolled variable that correlates with the independent variable and influences the measured outcome, making it impossible to attribute the result solely to the factor under test.

Example 1 — Cloud benchmarking. You are benchmarking two MQTT broker implementations on a public cloud VM. On Tuesday the results favour implementation A. On Wednesday you re-run the same benchmark and implementation B wins. The confound is *noisy neighbours*: other tenants' workloads on the shared physical host changed between runs, altering available CPU bandwidth and cache state. The benchmark is measuring cloud load, not broker performance. Mitigation: run on dedicated (non-shared) hardware, repeat many times and report confidence intervals, or run both implementations simultaneously on separate VMs using the same physical host.

Example 2 — Radio latency on different days. You measure the end-to-end latency of a LoRa sensor under two duty-cycle configurations. Monday's results look better — but Monday had no other LoRa devices nearby. Wednesday added a neighbour device on the same sub-band, increasing collision probability and retransmissions. The confound is RF environment (channel occupancy). Mitigation: conduct experiments in a controlled RF environment (shielded room or low-activity time window), randomise the order of configurations across repeated trials, and measure channel occupancy as a co-variate.

General control strategies: randomise trial order (eliminates systematic time-of-day effects), repeat many trials (averages out noise), control all variables except the one under test, measure and report confounders as co-variables, use dedicated hardware rather than shared cloud resources.

DA5. Describe MQTT's publish-subscribe model. How does it differ from a direct request-response model, and what are the architectural implications of topic hierarchy design and QoS level choice for a large IoT deployment?

MQTT decouples producers (publishers) from consumers (subscribers) through a central broker. A publisher sends a message to a **topic string** (e.g., `building/floor3/room-U181/temperature`) without knowing who, if anyone, is subscribed. Subscribers register interest in topic patterns with the broker; the broker routes incoming messages to all matching subscribers. Neither side knows the other exists — this is **spatial decoupling**.

This contrasts with a request-response model (e.g., HTTP REST) where the client knows the server's address, sends a request, and waits for a reply. MQTT is also **temporally decoupled** (with persistent sessions): a subscriber that is briefly offline can receive messages published during its absence.

Topic hierarchy implications: A flat topic structure (sensor42) makes it impossible to subscribe to "all temperature sensors on floor 3" without enumerating them. A well-designed hierarchy (building/{floor}/{room}/{type}) enables wildcard subscriptions (building/floor3/#) that automatically include new rooms added in the future. Poor topic design forces changes to all subscribers when the deployment grows.

QoS implications: QoS 0 (fire and forget) minimises broker load and latency but drops messages under network failures — acceptable for high-frequency sensor readings where a lost sample is replaced by the next one. QoS 1 (at least once) adds acknowledgment and retransmission — necessary for alerts and commands where no message must be missed, but may deliver duplicates that the consumer must handle idempotently. QoS 2 (exactly once) is reliable and duplicate-free but requires a four-way handshake, doubling the message cost — appropriate for billing events or irreversible actuator commands. Over-using QoS 2 on a high-frequency sensor stream would overload the broker.

Section 5 — Design Question

Question 16 — Smart Hospital Ward

(answer not found)

A hospital wants to deploy an IoT monitoring system across a single ward of **30 patient beds**. Each bed has:

- A wearable pulse-oximeter (SpO₂ + heart rate, continuous)
- A bedside temperature sensor (skin temperature, every 2 minutes)
- A nurse-call button (event-driven)

Requirements:

- **Nurse-call events** must reach the nursing station display within **1 second**.
- **Vital sign alerts** (SpO₂ < 90% or heart rate outside 40–120 bpm) must reach clinical staff within **3 seconds**.
- All readings must be **stored for 7 years** for regulatory compliance.
- The system must continue operating if the hospital's internet connection drops.

Section 3 — Short Answer

Subquestion 16.1 (short answer)

What sampling rates are appropriate for the pulse-oximeter and the temperature sensor respectively, and why are they different?

The **pulse-oximeter** (SpO₂ + heart rate) should sample at **1–2 Hz** continuously. SpO₂ and heart rate can change meaningfully within seconds during a clinical event (e.g., a desaturation episode); a 5-second alert window requires at least one reading per second to detect a threshold breach before the window closes. Higher rates (e.g., 25 Hz for waveform capture) would provide a plethysmographic waveform but are not needed for threshold alerting and would drain the battery far faster.

The **bedside temperature sensor** samples every **2 minutes** as specified. Skin temperature changes very slowly (over minutes to hours); high-frequency sampling provides no additional clinical value and wastes energy.

They differ because temperature dynamics are slow while SpO₂/HR can change clinically fast enough to require second-level resolution to meet the 3-second alert window.

Section 4 — Detailed Answer

Subquestion 16.2 (detailed answer)

What network topology and radio technologies should be used from wearable to nursing station to long-term storage? Justify every choice and explain what happens during an internet outage.

Wearable → Ward gateway (Edge → Fog):

Each wearable is body-worn and must communicate without wires. **BLE** (Bluetooth Low Energy) is the correct choice: it is designed for body-area networks, is standard in medical-grade wearables, achieves 10–30 m range (sufficient within a single ward room), and its power consumption allows 48-hour battery life with 1 Hz sampling. BLE GATT characteristics map naturally to vital-sign measurements. Each wearable connects to a bedside hub or ward-level gateway acting as a BLE central.

Wearable → Gateway protocol: BLE GATT notifications for continuous vital signs; BLE advertisements for nurse-call events (fast, no connection setup required for sub-1-second delivery).

Ward gateway → Hospital server (Fog → local cloud):

Wired **Ethernet** from each ward gateway to the hospital's local server. Hospital networks are wired infrastructure; wireless would add latency variability and RF interference near medical equipment (also a safety concern). A local hospital server (on-premises, not cloud) handles alerting and temporary storage.

Hospital server → Long-term storage (local → cloud):

MQTT over TLS to a cloud time-series database for 7-year retention. QoS 1 is appropriate — readings must not be lost, but occasional duplicates are acceptable in historical storage. During an internet outage, the local hospital server buffers readings to local disk (the system requirement: must operate during outage) and replays them when connectivity is restored.

Internet outage handling: The critical safety functions (vital-sign alerts, nurse-call routing) all run locally (ward gateway + hospital server) and do not require internet. Long-term cloud storage is delayed until connectivity resumes; no safety-critical function depends on the cloud.

Section 3 — Short Answer

Subquestion 16.3 (short answer)

Where in the topology should the vital-sign threshold alerting logic run, and why can it not run in the cloud?

Vital-sign threshold alerting must run at the **ward gateway (Fog tier)** or on the **hospital's local server** — not in the cloud. The reasons:

A cloud round-trip (wearable → gateway → internet → cloud → alert → nursing station) adds at minimum 500 ms and in practice several seconds of latency (broker queuing, cloud processing, alert routing back). The 3-second requirement would be violated routinely. Hospital internet links also fail or slow under load.

The ward gateway receives BLE notifications directly from wearables with <100 ms latency. Threshold logic running there can trigger an alert on the nursing station display within 1–2 seconds of a reading arriving, meeting the 3-second requirement with margin. The cloud continues receiving all readings for historical storage in parallel, but has no role in real-time alerting.

Subquestion 16.4 (short answer)

The wearable pulse-oximeter is battery-powered and must last at least 48 hours. Identify the dominant energy cost and propose one concrete strategy to extend battery life without missing clinically significant events.

The dominant energy cost is **BLE radio transmission**. At 1 Hz sampling with notifications sent to the gateway, the radio transmits roughly 60 packets per minute. Each BLE notification requires the radio to be active for ~1–5 ms at ~10–15 mA. If the battery is ~150 mAh (typical wearable), radio transmission accounts for the large majority of current draw at rest.

Concrete mitigation: aggregation + burst transmission.

Instead of sending a BLE notification for every individual sample, buffer 5–10 samples in the MCU and transmit them in a single packet every 5–10 seconds. The radio is active for approximately the same time per packet but fires 5–10× less often, reducing average radio current by that factor. SpO₂ and heart rate change slowly enough that a 5–10 second delivery latency still meets the 3-second alert window — the alert triggers on the *next* burst if a threshold is breached, worst-case 5–10 seconds after the event. If this latency is unacceptable, a threshold check can run on-device (Edge): if SpO₂ drops below 90%, the device immediately transmits a single alert packet bypassing the aggregation buffer.

Practice Exam 3

Question

Model Answer

Section 1 — Multiple Choice

MC1. What is quantization error in an ADC conversion?

- A) The delay between a sample being taken and its value being available in memory
- B) The difference between the true analogue input value and the nearest representable digital output value
- C) The noise introduced by electromagnetic interference on the analogue input pin
- D) The rounding error accumulated when averaging multiple samples in floating point

MC2. In C, the `volatile` keyword on a variable declaration tells the compiler to:

- A) Store the variable in read-only flash memory instead of RAM
- B) Always read the variable from memory rather than using a cached register copy
- C) Prevent other tasks from accessing the variable concurrently
- D) Align the variable to a 4-byte boundary for faster access

MC3. CSMA/CA stands for Carrier Sense Multiple Access with Collision Avoidance. How does it differ from CSMA/CD (Collision Detection)?

B

Quantization maps a continuous analogue value to the nearest of 2^N discrete levels. The error is at most $\pm \frac{1}{2}$ LSB (least significant bit). It is an inherent property of digitisation, not EMI or floating-point rounding.

B

`volatile` prevents the compiler from caching the variable in a CPU register across accesses. Without it, the compiler may optimise away repeated reads of a variable it believes hasn't changed — but hardware or an ISR may have changed it in memory.

B

• A) CSMA/CA detects collisions after they happen and retransmits; CSMA/CD prevents them before transmission • B) CSMA/CA tries to avoid collisions by waiting for the channel to be clear and adding a random backoff before transmitting; CSMA/CD detects a collision during transmission and aborts • C) CSMA/CD is used in wireless networks; CSMA/CA is used in wired Ethernet • D) CSMA/CA requires a central coordinator; CSMA/CD is fully distributed	CSMA/CA (wireless): sense the channel, if clear add a random backoff, then transmit — the goal is <i>avoiding</i> collisions before they happen. CSMA/CD (wired Ethernet): transmit, detect a collision <i>during</i> transmission via voltage monitoring, abort and retry. Collision detection during transmission is impossible in wireless because the transmitter's own signal drowns out incoming signals.
MC4. What is a UUID (Universally Unique Identifier) and which version is most commonly used for randomly generated device identities? • A) A 64-bit sequential counter; version 1 • B) A 128-bit identifier formatted as 8-4-4-4-12 hex digits; version 4 (random) • C) A 32-bit hash of the device MAC address; version 3 • D) A 256-bit cryptographic key; version 5	B UUID v4 is 128 bits of (mostly) random data, formatted as xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx. The probability of two random UUIDs colliding is negligible ($\sim 10^{-32}$).
MC5. In Elixir, what does the <code>%{}</code> syntax create? • A) A tuple with named fields • B) An anonymous function • C) A map — a key-value store where keys and values can be any term • D) A module struct with enforced field types	C <code>%{}</code> creates a map. <code>%MyStruct{}</code> creates a struct (which is a map with a <code>__struct__</code> key). Tuples use <code>{}</code> and anonymous functions use <code>fn -> end</code>.
MC6. What is oversampling in the context of ADC measurement? • A) Sampling at exactly twice the Nyquist frequency • B) Taking multiple samples at a rate higher than needed, then averaging them to reduce noise and improve effective resolution • C) Setting the ADC bit depth higher than the MCU's native word size • D) Configuring the ADC to sample continuously in the background using DMA	B Oversampling takes N samples at a rate much higher than necessary and averages them. This reduces random noise (noise averages toward zero) and improves effective resolution. Every factor-of-4 increase in sample count adds approximately 1 bit of effective resolution.
MC7. In LoRaWAN, what does OTAA (Over-The-Air Activation) do? • A) It updates the device firmware wirelessly • B) It allows the device to negotiate session keys dynamically with the network server during a join procedure, rather than having them hardcoded • C) It adjusts the spreading factor dynamically based on signal strength • D) It enables a LoRaWAN gateway to act as a mesh repeater	B OTAA performs a cryptographic join handshake between the device and the network server. The device receives NwkSKey and AppSKey dynamically per session. ABP (Activation By Personalisation) hardcodes keys — less secure and less flexible. OTAA has nothing to do with OTA firmware updates or ADR.
MC8. What is the hidden terminal problem in wireless networks?	B

• A) A device transmits on a hidden (reserved) channel that other devices cannot observe	Classic hidden terminal: device A and device C are both in range of device B (the receiver) but not of each other. A and C cannot sense each other's transmissions, so both decide the channel is clear and transmit simultaneously — causing a collision at B that neither A nor C detects.
• B) Two devices are each within range of a common receiver but out of range of each other, causing them to transmit simultaneously and collide at the receiver without either detecting the collision • C) A device's identity is hidden from neighbouring nodes for security reasons • D) A routing table entry is hidden from certain nodes to prevent routing loops	
MC9. In Elixir, a module attribute defined with <code>@my_attr</code> value is:	B
• A) A mutable global variable shared between all processes in the VM • B) A compile-time constant evaluated once when the module is compiled • C) A per-process dictionary entry stored in the process heap • D) An ETS (Erlang Term Storage) entry accessible across modules	Module attributes are evaluated at compile time and embedded in the BEAM bytecode as constants. They are not mutable at runtime, not per-process, and not ETS. Common uses: <code>@moduledoc</code>, <code>@spec</code>, <code>@default_timeout 5000</code>.
MC10. What is the primary purpose of an anti-aliasing filter placed before an ADC?	B
• A) To amplify weak signals to the full ADC input range • B) To remove frequency components above the Nyquist frequency before sampling, preventing aliasing • C) To convert a differential signal to a single-ended signal • D) To decouple the analogue and digital ground planes	An anti-aliasing filter is a low-pass filter that removes frequencies above the Nyquist limit <i>before</i> the ADC sees the signal. If omitted, those frequencies fold back into the sampled spectrum as aliasing artefacts that cannot be distinguished from or removed from the genuine signal components.
MC11. In Kconfig (used by ESP-IDF / Linux kernel), what does a <code>config</code> entry define?	B
• A) A CMakeLists variable that controls which source files to compile • B) A user-configurable compile-time option exposed in <code>menuconfig</code>, stored in <code>sdkconfig</code> • C) A runtime configuration parameter loaded from flash at boot • D) A FreeRTOS task priority that can be set without recompiling	Kconfig <code>config</code> entries declare named Boolean, integer, or string options. <code>menuconfig</code> renders them interactively; the choices are saved to <code>sdkconfig</code> and become <code>CONFIG_*</code> preprocessor macros in the build. They are compile-time, not runtime, and have nothing to do with task priorities or CMake source lists directly.
MC12. What is dead reckoning for device location estimation?	B
• A) Estimating position by measuring signal strength from multiple known transmitters • B) Estimating current position by integrating known velocity and heading from a last-known position, without external reference signals • C) Using GPS satellites to triangulate position to within 10 metres • D) Reading a hardcoded GPS coordinate from device flash memory	Dead reckoning computes position from a prior known position plus estimated displacement (speed × time, or IMU integration). It accumulates error over time and requires periodic correction from an absolute source. GPS triangulates from satellites. Signal-strength triangulation is a different technique (trilateration).

Section 2 — Multiple Answer

MA1. Which of the following are valid inter-task communication mechanisms in FreeRTOS?

- A) Queues — for passing data items between tasks
- B) Binary semaphores — for signalling that an event has occurred
- C) Counting semaphores — for tracking a pool of available resources
- D) Shared global variables protected by a mutex
- E) Direct task function calls across task boundaries

MA2. Which statements about RDF are correct?

- A) Every RDF resource is identified by a URI (or blank node)
- B) RDF triples can be stored in formats including Turtle, JSON-LD, and RDF/XML
- C) RDF requires a schema to be defined before any data can be inserted
- D) Multiple RDF graphs from different sources can be merged without naming conflicts, because URIs are globally unique
- E) SPARQL is the standard query language for RDF graphs

MA3. Which of the following are valid approaches to reducing quantization error or noise in an ADC measurement?

- A) Oversampling and averaging multiple readings
- B) Increasing the ADC bit depth (e.g., from 12-bit to 16-bit)
- C) Applying a software moving-average filter to successive readings
- D) Increasing the ADC clock frequency beyond the Nyquist rate
- E) Using a lower attenuation setting to use more of the ADC's input range

MA4. Which properties do Zigbee and Thread share?

- A) Both are based on the IEEE 802.15.4 physical and MAC layer
- B) Both operate in the 2.4 GHz ISM band
- C) Both use IPv6 natively for device addressing
- D) Both support mesh networking where devices route for each other
- E) Both use the same application-layer protocol (Zigbee Cluster Library)

MA5. Which are valid ways to reduce a compiled binary's size on a flash-constrained edge device?

MA1 A, B, C, D**

E is wrong — tasks are independent execution contexts; you cannot call another task's function directly the way you would a regular function. Direct calls would execute in the *calling* task's context and violate task isolation. A, B, C (queues, binary semaphores, counting semaphores) are the primary mechanisms. D (shared global variables + mutex) is technically valid but less idiomatic than queues.

MA2 A, B, D, E**

C is wrong — RDF is schema-free; triples can be added without a predefined schema (which is *why* SHACL was introduced later). A, B, D, and E are all correct: URIs identify resources, multiple serialisation formats exist, global uniqueness of URIs enables safe merging, and SPARQL is the standard query language.

MA3 A, B, C, E**

D is wrong — increasing clock frequency beyond the Nyquist rate does not reduce quantization error; it may reduce aliasing if signal bandwidth is the concern, but quantization is a separate issue. A (oversampling + averaging), B (higher bit depth), C (moving average filter), and E (lower attenuation → better use of input range → finer resolution per LSB) are all valid.

MA4 A, B, D**

C is wrong — Zigbee does *not* use IPv6 natively; Thread does (Thread is IPv6-first). E is wrong — Thread uses its own application layer (often combined with Matter/OpenThread); the Zigbee Cluster Library is Zigbee-specific. A (both use 802.15.4 PHY/MAC), B (both use 2.4 GHz), and D (both support mesh) are correct.

MA5 A, B, C**

• A) Compile with size-optimisation flags (-Os)	D is wrong — replacing recursion with iteration reduces <i>stack usage at runtime</i> but does not reduce the compiled binary size (the code itself may be similar in size or larger). E is wrong — CPU clock frequency has nothing to do with compiled binary size. A (-Os optimises for size), B (removing unused Kconfig components), and C (float is 4 bytes vs double's 8 bytes, reducing data segment and library pull-in) are valid.
• B) Remove unused library components via Kconfig / menuconfig • C) Use float instead of double for floating-point calculations • D) Replace recursive algorithms with iterative equivalents to reduce stack usage • E) Increase the CPU clock frequency	

MA6. Which of the following are part of the six IoT domains described in the course?	MA6 A, B, D, E, F, G**
• A) Data Collection	C (Data Encryption) is not one of the six domains. The six are: Data Collection, Data Transportation, Information/Metadata Management, Data Processing, Decision Support, and Actuation.
• B) Data Transportation • C) Data Encryption • D) Information / Metadata Management • E) Data Processing • F) Decision Support • G) Actuation	

Section 3 — Short Answer

SA1. What is the volatile keyword in C, and give a concrete example of why it is needed in an embedded IoT context.	volatile tells the compiler that a variable's value may change at any time outside the normal program flow — e.g., by hardware, by an interrupt service routine, or by another thread. Without volatile, an optimising compiler may read the variable into a register once and use the cached value repeatedly, never re-reading memory. This produces incorrect behaviour when the value changes externally. Concrete embedded example: an ISR sets a global flag uint8_t data_ready = 1; when a UART byte arrives. The main loop polls while (!data_ready) {}. Without volatile, the compiler may read data_ready once, see 0, and optimise the loop into an infinite loop that never re-reads memory — even after the ISR has set it. volatile uint8_t data_ready; forces a memory read on every loop iteration.
SA2. What is an anti-aliasing filter, where in the signal chain must it be placed, and what happens if it is omitted?	An anti-aliasing filter is an analogue low-pass filter that attenuates all frequency components above the Nyquist frequency (half the sample rate). It must be placed in the analogue signal path before the ADC input — i.e., in hardware, not in software. Once a signal has been sampled, frequencies that exceeded the Nyquist limit have already been folded back into the spectrum as aliasing artefacts; software cannot remove them because they are now indistinguishable from genuine lower-frequency components. Omitting the filter causes aliased artefacts that permanently contaminate the digital signal.
SA3. What is quantization error, and how does increasing the ADC bit depth reduce it?	An N-bit ADC maps the full input voltage range to 2^N discrete levels. Quantization error is the difference between the true analogue value and the nearest representable level — at most $\pm\frac{1}{2}$ LSB. For a 12-bit ADC with a 3.3 V range, each LSB = $3.3 / 4096 \approx 0.81$ mV, so the maximum quantization error is ≈ 0.4 mV.

	Increasing to 16-bit: $LSB = 3.3 / 65536 \approx 0.050 \text{ mV}$ — maximum error $\approx 0.025 \text{ mV}$. Every additional bit halves the maximum quantization error and doubles the resolution. Oversampling can achieve a similar effect in software: averaging 4 samples at 12-bit provides approximately 13-bit effective resolution.
SA4. What is the hidden terminal problem, and how does the RTS/CTS handshake in 802.11 (WiFi) attempt to solve it?	The hidden terminal problem: two devices (A and C) can both reach a shared receiver (B) but cannot hear each other. A senses the channel as idle and begins transmitting; C also senses idle (it cannot hear A) and begins transmitting simultaneously — causing a collision at B. RTS/CTS addresses this: before transmitting, A sends a short RTS (Request to Send) frame to B. B replies with a CTS (Clear to Send) frame, which is heard by all devices in B's range — including C. C hears the CTS and knows it must defer. A then transmits its data frame without collision. The cost: RTS/CTS adds two control frames per data frame, increasing overhead and latency. For small packets the overhead may exceed the payload; most 802.11 implementations only enable RTS/CTS above a configurable packet-size threshold.
SA5. What is a UUID, and what two properties of UUIDs make them well-suited as device identifiers in a large IoT deployment?	A UUID (Universally Unique Identifier) is a 128-bit value formatted as 32 hex digits in groups 8-4-4-4-12 (e.g., 550e8400-e29b-41d4-a716-446655440000). Two properties make UUIDs well-suited for IoT device identity: 1. Global uniqueness without coordination: UUID v4 (random) generates identifiers with a collision probability so low ($\sim 10^{-26}$) that no central registry is needed. Any device can generate its own UUID offline without checking with a server. 2. Opacity: UUIDs carry no embedded information about the device's location, manufacturer, or type — preventing reverse-engineering of deployment topology from the identifier alone.
SA6. What is dead reckoning, and when would you use it instead of GPS for location tracking in an IoT device?	Dead reckoning estimates current position by starting from a last known position and integrating motion over time using onboard sensors: an accelerometer or IMU (for acceleration/orientation) or wheel encoder (for speed and direction). No external reference (GPS, cell towers) is needed. Use dead reckoning instead of GPS when: GPS is unavailable (indoors, tunnels, dense urban canyons where satellite signals are blocked); GPS would consume too much power for a battery-constrained device; or fast position updates are needed between infrequent GPS fixes (GPS receiver takes seconds to acquire a fix). The drawback is error accumulation — IMU drift makes positions increasingly inaccurate over time without periodic correction from an absolute source.
SA7. What is a FreeRTOS queue, how does it differ from a semaphore, and when should you use one over the other?	A queue is a FIFO buffer for transferring data items between tasks or between an ISR and a task. Items are copied in and out; the queue holds N items of a fixed declared size. Use a queue when you need to pass actual data (a sensor reading, a command struct) from one task to another. A semaphore (binary or counting) carries no data — it is a signalling mechanism. A binary semaphore has two states (given/taken); a counting semaphore has a count. Use a semaphore when you only need to signal that an event happened (ISR notifying a processing task) or to protect a shared resource via mutual exclusion (mutex). Using a queue to signal events is wasteful; using a semaphore to transfer data is impossible.

SA8. What is Kconfig, and what problem does it solve compared to hardcoding `#define` values in source files?

Kconfig is a configuration system (originally from the Linux kernel, adopted by ESP-IDF and Zephyr) that allows build-time options to be defined in structured `kconfig` files and configured interactively via `idf.py menuconfig`. The chosen values are saved to `sdkconfig` and generated as `CONFIG_*` preprocessor macros.

The problem it solves: hardcoding `#define BAUD_RATE 115200` in source files requires recompiling after editing source. With Kconfig, options are declared with descriptions, types, defaults, and dependencies; `menuconfig` presents a navigable menu; the build system tracks changes and recompiles only what is affected. This separates configuration from code, enables feature gating (enable/disable entire subsystems), and documents options with their valid ranges.

SA9. What is oversampling, and how does taking N samples and averaging them improve the effective bit resolution of an ADC?

Oversampling takes M samples at a rate much higher than the signal requires and averages them. Averaging reduces random noise because noise is uncorrelated between samples — its expected value after averaging N samples is reduced by a factor of $\sqrt{N}$ in amplitude (N in power).

The improvement in effective bit resolution follows from: each factor of $4\times$ in sample count adds 1 bit of effective resolution. So averaging 4 samples improves a 12-bit ADC to ~ 13 -bit; averaging 16 samples improves it to ~ 14 -bit. This works because averaging shifts the quantization-error floor, effectively interpolating between discrete levels. The trade-off is increased CPU load and slower output rate (or the need for a faster ADC clock).

SA10. What is clock drift in the context of IoT time synchronisation, and what are its practical consequences for correlating sensor readings across devices?

Clock drift is the gradual divergence of a device's local clock from true time, caused by imperfections in the crystal oscillator. Typical MCU crystals drift at 20–100 ppm (parts per million), meaning up to 8.6 seconds per day of error.

Practical consequences for IoT: if two sensor nodes each measure a temperature event but have unsynchronised clocks drifted by 5 seconds, fusing their readings in time-series analysis places them at different timestamps even though they are simultaneous. Anomaly detection, cross-device correlation, and causal analysis all become unreliable. Over long deployments, drift accumulates — a device without NTP may be minutes off after weeks. Mitigation: synchronise to NTP on every reconnection; for offline devices, record the NTP offset at last sync and apply it as a correction.

SA11. What is the Elixir case expression, and how does it differ from an `if` expression?

`case` matches an expression against a sequence of patterns (optionally with guard clauses) and evaluates the first matching branch. It is exhaustive pattern matching on a value:

```
case result do
  {:ok, value}    -> process(value)
  {:error, reason} -> log(reason)
end
```

`if` tests a single Boolean condition and has an optional `else`. Both are expressions in Elixir (they return a value). `case` is preferred when the number of alternatives exceeds two, when destructuring is needed, or when the branches depend on the *shape* of a value rather than a simple true/false test. `if` is idiomatic only for simple binary conditions.

SA12. In the context of radio communication, what is the difference between a unicast, a broadcast, and a multicast transmission?

Unicast: a transmission addressed to exactly one recipient device. The frame contains the destination's specific address. All other devices on the medium discard it at the MAC layer.

Broadcast: a transmission addressed to all devices on the network segment simultaneously. Every device receives and processes it. Excessive broadcast traffic can flood a network (broadcast storm).

Multicast: a transmission addressed to a defined group of devices that have explicitly subscribed to a multicast group address. Only group members process the frame. More efficient than sending N unicasts to N recipients but less disruptive than a full broadcast. Used in MQTT over WiFi for group subscriptions, and in LoRaWAN for Class B downlink slots.

Section 4 — Detailed Answer

DA1. Explain the hidden terminal problem in wireless networks. Describe how CSMA/CA detects channel state, why the hidden terminal problem causes CSMA/CA to fail, and how the RTS/CTS mechanism addresses it. What is the cost of using RTS/CTS?

The problem: CSMA requires each node to sense the channel before transmitting. If the channel is busy, the node defers. This works when all nodes can hear each other. The hidden terminal problem arises when two nodes (A and C) are both in range of a common receiver (B) but cannot hear each other's transmissions. A senses the channel idle (it cannot hear C), begins transmitting to B. Simultaneously C also senses idle (it cannot hear A), begins transmitting to B. A collision occurs at B, but neither A nor C detects it — they both believe their transmission was successful. Without an acknowledgment, A and C must time out and retry, repeatedly colliding.

How CSMA/CA alone fails: The random backoff in CSMA/CA helps when nodes *can* hear each other (they both wait, one's backoff expires first, it transmits, the other defers). But if A and C cannot hear each other, they independently draw random backoffs, one happens to be shorter, that node transmits — and the other node, still unable to hear the transmission, fires at any point during it.

RTS/CTS mechanism: Before a data frame, A sends a small RTS (Request to Send) frame to B. B responds with CTS (Clear to Send), which it broadcasts into B's full coverage area. C, being in B's range, hears the CTS even though it cannot hear A. The CTS contains the expected duration of the upcoming data transfer. C sets a NAV (Network Allocation Vector) timer and defers for that duration — effectively silencing itself without having heard A at all. A proceeds to transmit collision-free.

Cost of RTS/CTS: Two additional control frames per data exchange. For small payloads (e.g., a 10-byte sensor reading), the RTS+CTS overhead (typically 20–40 bytes plus SIFS/DIFS gaps) may exceed the payload itself, nearly doubling channel time. For this reason most 802.11 implementations disable RTS/CTS for packets below a threshold (e.g., 2346 bytes by default) and enable it only for large frames where the collision cost justifies the overhead.

DA2. Explain Elixir's pattern matching system in depth. Cover: the match operator, destructuring tuples and maps, the pin operator, guard clauses in function heads, and how multiple function clauses together implement conditional dispatch without `if/case`.

Pattern matching is the foundation of Elixir's control flow. The `=` operator is not assignment — it attempts to *match* the right-hand side against the left-hand pattern, binding variables as needed.

DA3. Explain how time synchronisation works in an IoT system. Cover NTP operation and stratum, clock drift at the edge, the practical consequences of unsynchronised clocks for sensor data correlation, and what to do when NTP is unavailable (e.g., during network outage).

Destructuring: `{a, b, c} = {1, 2, 3}` binds `a=1, b=2, c=3`. `[head | tail] = [1, 2, 3]` binds `head=1, tail=[2,3]`. Nested structures: `%{user: %{name: name}}` = response extracts a name from a nested map. If the pattern doesn't match, a `MatchError` is raised — this is intentional; unexpected shapes are bugs, not edge cases to silently handle.

Pin operator `^`: Once a variable is bound, re-using its name in a match would rebound it. `^x` pins `x` to its current value and matches only if the right-hand side equals that value. Example: `^expected = actual` fails (raises `MatchError`) if `actual ≠ expected`.

Guard clauses: when clauses in function heads add Boolean conditions beyond structural shape. Only a limited set of expressions are allowed in guards (comparison operators, `is_integer/1`, `is_binary/1`, arithmetic, etc.) — side effects are forbidden because guards may be evaluated multiple times.

Multiple function clauses: Instead of a single function with an `if/case` tree, Elixir functions can have multiple clauses matched top-to-bottom:

```
def describe({:ok, val}),      do: "success: #{val}"
def describe({:error, msg}),   do: "error: #{msg}"
def describe(other),          do: "unknown: #{inspect other}"
```

The BEAM matches pattern and guard simultaneously for each clause, evaluating the first that succeeds. This eliminates nested conditionals and makes each case explicit, independently testable, and readable.

NTP operation: NTP (Network Time Protocol) synchronises clocks by exchanging timestamps with a server. The client measures the round-trip delay, estimates one-way latency, and adjusts its clock accordingly. The accuracy of the correction depends on symmetric network delay; asymmetric paths introduce offset errors.

NTP stratum: Stratum 1 servers are directly connected to stratum 0 reference clocks (GPS, atomic). Stratum 2 synchronise from stratum 1, stratum 3 from stratum 2. Each hop adds uncertainty — stratum 3 devices are typically accurate to a few milliseconds; stratum 1 to microseconds.

Clock drift at the edge: MCU crystal oscillators drift 20–100 ppm. At 50 ppm, a device drifts 4.3 seconds per day. Without periodic NTP correction, timestamps become unreliable. In a dense deployment, two drifting devices accumulate independent errors — their clocks may diverge from each other by seconds per day even if both started synchronised.

Consequences for sensor data: Correlating events across devices requires timestamps to be trustworthy. A 5-second clock error means an event recorded at device A at `T=100` and at device B at `T=105` may actually be simultaneous — or may be genuinely 5 seconds apart. Anomaly detection, causal analysis, and data fusion all depend on temporal alignment. A 5-second error is the difference between "simultaneous pressure spike in three pipes" (a water-hammer event) and "three unrelated pressure events."

	NTP unavailable: During network outages, devices must use local time. Strategies: (1) record the NTP offset at the last successful sync and apply it as a known-good correction for subsequent timestamps; (2) note the elapsed time since last sync and flag timestamps with an uncertainty estimate that grows with time since sync; (3) use a hardware RTC (Real-Time Clock) module with a battery-backed crystal, which drifts more slowly than a bare MCU oscillator and retains time across power cycles.
DA4. Describe the COP (Consistency, Overhead, Performance) evaluation model for IoT systems. What does each dimension represent, why can you not optimise all three simultaneously, and how should this guide the design of a performance evaluation experiment?	The COP evaluation model states that for any IoT or distributed system, you can trade off three properties but cannot simultaneously optimise all of them: Consistency (C): The degree to which measurements and system state are accurate and free from noise, quantization error, or logical inconsistency. Improving consistency typically requires more samples (oversampling), better sensors (more expensive), or more processing (filtering, calibration). Overhead (O): The total resource cost of the evaluation or system operation — energy, bandwidth, CPU time, memory. Reducing overhead means faster, simpler, or less frequent operations. Performance (P): The quality of the outcome — throughput, latency, accuracy of prediction, event-detection reliability. High performance usually requires more resources and/or more consistent inputs. The tension: Improving C (more samples, better filtering) increases O (more ADC reads, more computation, more radio transmissions). Improving P (detecting events faster) reduces opportunities to save O (must sample and transmit more frequently). Reducing O (sleep longer, transmit less) degrades both C (fewer samples, more quantization noise) and P (slower event detection, higher alert latency). Experimental design implication: When benchmarking an IoT system, changing one factor inevitably moves at least one other. A fair evaluation must hold two of the three fixed and vary only the factor under test — otherwise the "improvement" in one dimension may simply be the result of relaxing another. For example, claiming "our new protocol improves throughput" without noting that it uses 3× more energy is misleading unless O is held constant.
DA5. Explain the six IoT domains and give a concrete example of each for a smart building energy management system. Explain why the domains must be traversed in order and what happens if one domain is poorly designed.	The six domains form a strict pipeline; each depends on the output of the previous. 1. Data Collection: Physical sensing of the environment. Example: DS18B20 digital temperature sensors on each floor reading every 30 seconds; CO sensors and PIR occupancy detectors in each room. Poor design here (wrong sensor, bad placement, too-low sampling rate) corrupts everything downstream — garbage in, garbage out.

2. Data Transportation: Moving raw readings from sensor to the next tier. Example: Sensors publish over MQTT to a local Zigbee-to-Ethernet gateway; the gateway forwards to a cloud broker over TLS/TCP. A poorly designed transport layer loses messages (QoS 0 under a congested WiFi network) or introduces latency that makes real-time alerting impossible.

3. Information / Metadata Management: Giving raw values context. Example: An RDF Brick Schema model maps sensor IDs to rooms, floors, and buildings; associates units (Kelvin, ppm) and calibration offsets; links occupancy PIR sensors to HVAC zones. Without this, `sensor_17 = 0.73` is meaningless to any downstream consumer.

4. Data Processing: Transforming contextualised readings into derived quantities. Example: A moving average smooths noisy temperature readings; an occupancy model infers whether a room is occupied from PIR readings over the past 10 minutes; absolute humidity is computed from temperature + relative humidity. A poorly designed processing step (e.g., no noise filtering) causes the Decision Support layer to fire false alarms.

5. Decision Support: Presenting processed information for human or automated decisions. Example: A dashboard shows per-zone temperature vs. setpoint with colour coding; an alert fires when CO₂ exceeds 1000 ppm. A well-designed Decision Support layer presents only actionable information — too many alerts causes alert fatigue.

6. Actuation: Acting on the physical world based on decisions. Example: A HVAC controller adjusts valve positions via Modbus; a relay toggles the heating element on/off. Actuation closes the feedback loop. A poorly designed actuation layer (wrong protocol, no acknowledgment, actuator stuck) means decisions are never translated into physical change.

Why order matters: You cannot actuate without a decision; you cannot make a decision without processed data; you cannot process without transported data; you cannot transport without collected data. Skipping or poorly implementing any domain cascades failures through all subsequent ones.

Section 5 — Design Question

Question 17 — Smart City Waste Management

(answer not found)

A city council wants to deploy IoT sensors across **500 public waste bins** spread over a city of 10 km². Each bin has an ultrasonic fill-level sensor. The system should:

- **Report bin fill level** to a central platform every 15 minutes (routine).
- **Send an immediate alert** when a bin exceeds 90% full, so a collection truck can be dispatched within 2 hours.
- **Optimise truck routes** daily using historical fill data (which bins fill fastest, at what times of day).

The bins are powered by a **small solar panel with a 4000 mAh battery backup**. There is no wired infrastructure — all communication must be wireless.

Section 3 — Short Answer

Subquestion 17.1 (short answer)

Which radio technology is most appropriate for the sensor-to-network link, and why? Discuss at least two alternatives and explain why you reject them.

Best choice: LoRaWAN. The bins are spread across 10 km² with no wired infrastructure. LoRaWAN's key advantages here: long range (a single gateway covers several km in an urban environment), very low power consumption (a LoRa transmission takes ~100–500 ms and the radio sleeps otherwise — enabling years of operation on a small battery), and suitability for low-data-rate, infrequent reporting (a fill-level reading is tens of bytes, once per 15 minutes). Public LoRaWAN infrastructure (TTN or municipal gateways) may already exist.

Why not WiFi (802.11): WiFi requires the bin to connect to an access point. Street infrastructure has no WiFi coverage. Even if hotspots existed, WiFi consumes ~150–300 mA during connection setup — incompatible with a solar/battery budget at this scale.

Why not BLE: BLE has a range of 10–30 m outdoors. Covering 500 bins across 10 km² would require thousands of relay nodes or gateways. It is designed for personal-area networks, not city-scale deployment.

Why not cellular (NB-IoT): NB-IoT is technically viable (low power, long range, uses licensed spectrum) and is a reasonable alternative if a municipal SIM contract is available. The tradeoff: NB-IoT has recurring SIM costs and carrier dependency; LoRaWAN can use free public infrastructure or a one-time private gateway deployment.

Subquestion 17.2 (short answer)

The fill-level sensor runs on the same battery as the radio. Describe the duty-cycle strategy you would use to meet the 15-minute reporting interval while maximising battery life.

The bin should spend almost all its time in **deep sleep**. The duty-cycle strategy:

1. Wake from deep sleep on a hardware timer every 15 minutes.
2. Power up the ultrasonic sensor, take a fill-level reading (takes ~50–100 ms).
3. Compare to 90% threshold locally on the MCU.
4. If below 90%: transmit one LoRa packet with the fill level. Radio on-time ≈ 100–500 ms depending on SF.
5. If above 90%: transmit an immediate alert packet (higher priority).
6. Return to deep sleep.

At SF9 (moderate range/data-rate balance) a packet takes ~185 ms. 15-minute interval: radio duty cycle = 185 ms / 900,000 ms = 0.02%. Average current from radio ≈ 20 mA × 0.0002 = 0.004 mA. Deep sleep current for an ESP32 ≈ 10–20 μA. Total average ≈ 0.02–0.03 mA — a 4000 mAh battery lasts ~6–8 years even without solar. In practice solar tops up the battery daily, making lifetime effectively indefinite.

Section 4 — Detailed Answer

Subquestion 17.3 (detailed answer)

Step 1 — Edge (on the bin MCU): Read the ultrasonic sensor, convert distance to fill percentage, apply threshold check. Transmit one LoRa packet: {bin_id, fill_pct, timestamp, battery_voltage}. The edge does no further processing — it is too constrained, and aggregation requires data from all bins.

Design the data pipeline from the sensor to the route-optimisation engine. What are the processing steps, where does each step run (Edge / Fog / Cloud), and how is the immediate 90%-full alert delivered reliably despite the constrained radio?

Step 2 — Fog (LoRaWAN gateway + local network server): The gateway receives the radio packet and forwards it over IP to the LoRaWAN network server. The network server de-duplicates packets (multiple gateways may receive the same transmission), decrypts the payload, and forwards the decoded reading to the cloud application server via MQTT or HTTP. The network server also manages downlink scheduling for any control messages.

Step 3 — Cloud (application server + time-series database): The application server receives readings, writes them to a time-series database (e.g., InfluxDB or TimescaleDB) with `bin_id`, `timestamp`, and `fill_pct`. This is the authoritative record for all historical data.

Step 4 — Alert path: The application server applies the 90% threshold on every ingested reading. If exceeded, it publishes an alert to an MQTT topic `waste/alert/{bin_id}`, which triggers a push notification to dispatch software and the truck driver's mobile app. LoRaWAN is inherently reliable enough for this: QoS 1 in the LoRaWAN MAC ensures the packet is acknowledged by the gateway. If a packet is lost, the bin retransmits on the next wake cycle (15 minutes later) — within the 2-hour dispatch window.

Step 5 — Route optimisation (Cloud batch job): A nightly batch job reads historical fill data from the time-series database, builds a model of fill rate per bin per time-of-day, and generates the next day's optimal collection route (minimising truck distance while ensuring no bin exceeds capacity). This is pure cloud analytics — no latency requirement, requires months of historical data.

Section 3 — Short Answer

Subquestion 17.4 (short answer)

Over 10 years, each bin transmits a fill-level reading every 15 minutes. Estimate the total data volume for the entire 500-bin fleet and argue whether you should store raw readings or aggregated statistics for the route-optimisation use case.

Each reading: approximately 20 bytes (`bin_id`: 4 bytes, `fill_pct`: 1 byte, `timestamp`: 4 bytes, `battery`: 2 bytes, overhead: ~9 bytes).

Per bin per year: $(60/15) \times 24 \times 365 = 35,040$ readings $\times$ 20 bytes = **700 KB/bin/year**.

500 bins $\times$ 700 KB = **350 MB/year** for the whole fleet. Over 10 years: **3.5 GB total**.

3.5 GB is trivially small by modern standards — a single hard disk. Raw readings should be stored in full. The argument for raw storage: route optimisation requires detailed temporal patterns (which hours fill fastest, weekday vs. weekend differences, seasonal trends). Aggregating to, say, daily averages would destroy the hourly signal needed to optimise truck departure times. Storage cost at 3.5 GB/decade is effectively zero compared to the operational cost of the sensor network.

The GDPR data minimisation argument does not apply here: fill-level readings from a public waste bin are not personal data (they describe the bin, not a person), so there is no regulatory pressure to aggregate or delete.

Practice Exam 4

Question

Model Answer

Section 1 — Multiple Choice

MC1. In C, a static variable declared inside a function:

- A) Is placed on the stack and deallocated when the function returns
- B) Is initialised every time the function is called
- C) Retains its value between calls and is initialised only once
- D) Is visible to all functions in the translation unit

C

A static local variable is allocated in the BSS/data segment, not the stack. It is zero-initialised (or explicitly initialised) only once at program start and retains its value across calls. A is the behaviour of non-static locals; B is wrong; D is wrong (it has function scope).

MC2. What does the MQTT retain flag on a published message do?

- A) It prevents the broker from forwarding the message to any subscriber
- B) It causes the broker to store the last retained message for a topic and deliver it immediately to new subscribers
- C) It tells the broker to keep the message in a queue until the publisher confirms delivery
- D) It sets the QoS level to 2 (exactly-once delivery)

B

The retain flag tells the broker to keep the last message published to that topic and push it immediately to any client that subscribes later. This means a newly connected subscriber sees the current state without waiting for the next publish. A, C, D are all wrong.

MC3. What is 6LoWPAN?

- A) A LoRaWAN variant that supports IPv6 addressing
- B) An adaptation layer that compresses IPv6 headers to fit inside IEEE 802.15.4 frames (≤ 127 bytes)
- C) A 6 GHz band extension of the LoRa PHY
- D) A 6-byte addressing scheme for BLE peripherals

B

6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks) is an IETF adaptation layer (RFC 4944 / RFC 6282) that compresses IPv6 headers from 40 bytes down to a few bytes so they fit in 802.15.4's 127-byte max frame. A, C, D misidentify it.

MC4. In Elixir, the with expression is used to:

- A) Define a module-level constant visible to all functions
- B) Chain multiple pattern matches and short-circuit to an else clause on the first mismatch
- C) Spawn a linked child process that shares the parent's state
- D) Import all public functions from another module into the current scope

B

with evaluates a series of match clauses left-to-right; if any clause fails to match, execution jumps to the else block. It is used to flatten nested case/ok/error chains. A, C, D are unrelated.

MC5. In LoRa, increasing the spreading factor (SF) from SF7 to SF12:

- A) Increases the data rate and reduces range
- B) Reduces the data rate but increases link budget, time-on-air, and energy per message

B

Each step up in SF doubles the number of chips per symbol, halving the bit rate (e.g., SF7 $\approx$ 5.5 kbps vs SF12 $\approx$ 250 bps at BW 125 kHz). The longer symbol time improves sensitivity by ~ 2.5 dB per SF step, extending range. Time-on-air grows exponentially with SF. A is backwards; C and D are wrong.

• C) Has no effect on time-on-air; only frequency matters • D) Enables frequency hopping across multiple sub-bands	
MC6. What is a watchdog timer in an embedded system?	B A watchdog is a countdown timer in hardware. Software must periodically write a "kick" value to prevent it reaching zero; if it does, the MCU resets. This recovers from infinite loops, deadlocks, or stack overflows. A, C, D are wrong.
• A) A timer that generates an interrupt every millisecond for the RTOS scheduler • B) A hardware timer that resets the MCU if software fails to periodically "kick" it, recovering from hangs • C) A DMA channel that monitors ADC overflow conditions • D) A software counter that tracks elapsed time since last GPS fix	
MC7. What is Adaptive Data Rate (ADR) in LoRaWAN?	B LoRaWAN's network server monitors link quality and instructs devices to use the fastest SF and lowest TX power that still maintains the link, reducing ToA and saving battery. A, C, D are wrong.
• A) A mechanism that dynamically adjusts the MQTT QoS level based on link quality • B) A network-server mechanism that adjusts a device's spreading factor and transmit power to optimise data rate while respecting duty-cycle limits • C) A protocol that switches between LoRa and NB-IoT depending on signal strength • D) A retransmission scheme for Class B devices	
MC8. In Elixir, how does <code>Task.async/1</code> + <code>Task.await/2</code> differ from bare <code>spawn/1</code> ?	B <code>Task.async</code> sets up a monitor link so a crash in the task propagates to the caller; <code>Task.await</code> blocks until the result is returned or timeout occurs. <code>spawn</code> is truly fire-and-forget: no result, no crash propagation. A, C, D are wrong.
• A) <code>Task.async</code> creates a named process; <code>spawn</code> creates an anonymous one • B) <code>Task.async</code> links the task to the caller, propagates crashes, and returns a value via <code>Task.await</code>; <code>spawn</code> is fire-and-forget with no built-in result return • C) <code>Task.async</code> runs the function synchronously in the caller's process • D) <code>spawn</code> is supervised; <code>Task.async</code> is not	
Section 2 — Multiple Answer	
MA1. Which of the following are characteristics of NB-IoT that distinguish it from LoRaWAN?	MA1 A is correct: NB-IoT operates in licensed LTE bands (e.g., 700–900 MHz); LoRaWAN uses unlicensed ISM. B is correct: NB-IoT inherits 3GPP QoS mechanisms, ACKs, and retransmissions; LoRaWAN is best-effort with no guaranteed delivery (Class A). C is correct: NB-IoT supports PSM and eDRX but can receive downlink at any time when awake; LoRaWAN Class A only opens two short RX windows after each uplink. D is correct: NB-IoT requires a SIM and carrier agreement; LoRaWAN requires a gateway but no subscription. E is wrong: NB-IoT is generally better for latency-sensitive or higher-throughput use cases; LoRaWAN is better for very low duty-cycle, battery-constrained nodes.
• A) NB-IoT uses licensed cellular spectrum; LoRaWAN uses unlicensed ISM bands • B) NB-IoT provides guaranteed QoS via 3GPP standards; LoRaWAN is best-effort • C) NB-IoT supports bidirectional communication with low latency downlink; LoRaWAN Class A has constrained downlink windows • D) NB-IoT requires a SIM card and mobile network subscription; LoRaWAN does not • E) Both NB-IoT and LoRaWAN are equally power-efficient for all duty cycles	

MA2. Which of the following are valid Elixir error-handling patterns?

- A) `{:ok, value} / {:error, reason}` tagged tuples from function returns
- B) `try / rescue` for catching exceptions raised by `raise/1`
- C) `catch` for catching thrown values from `throw/1`
- D) Supervision trees that restart crashed processes rather than handling errors inline
- E) Using `nil` as a universal error sentinel (same as in Erlang)

MA3. Which parameters increase LoRa time-on-air (and therefore energy per transmission)?

- A) Larger spreading factor (e.g., SF12 vs SF7)
- B) Wider bandwidth (e.g., 500 kHz vs 125 kHz)
- C) Lower coding rate (e.g., 4/8 vs 4/5)
- D) Longer payload (more bytes)
- E) Higher transmit power (dBm)

MA4. Which of the following are valid RDF serialisation formats?

- A) Turtle (`.ttl`)
- B) JSON-LD (`.jsonld`)
- C) RDF/XML (`.rdf`)
- D) N-Triples (`.nt`)
- E) CSV with subject/predicate/object columns

MA2

A is correct: the idiomatic Elixir pattern is to return `{:ok, value}` or `{:error, reason}` and pattern-match on the result. B is correct: `try/rescue` catches exceptions raised with `raise`. C is correct: `catch` catches values thrown with `throw` (used for non-local returns). D is correct: supervision trees are the primary fault-tolerance mechanism — let a process crash and the supervisor restarts it. E is wrong: using `nil` as a universal error sentinel is not idiomatic; it discards the reason and leads to silent failures.

MA3

A is correct: higher SF increases the number of chips per symbol, directly increasing ToA (approximately doubles per SF step). B is wrong: wider bandwidth shortens ToA (more chips transmitted per second). C is correct: lower coding rate means more redundancy bits per payload bit ($4/8 = 50\%$ overhead vs $4/5 = 20\%$), increasing ToA. D is correct: more payload bytes directly add symbol time. E is wrong: TX power affects link budget and energy per unit time but does not change ToA (it changes total energy only because the radio draws more current at higher power).

MA4

A is correct: Turtle is the most human-readable RDF format. B is correct: JSON-LD is an RDF serialisation that is also valid JSON. C is correct: RDF/XML is the original W3C serialisation format. D is correct: N-Triples is a line-based, easily parsed format. E is wrong: plain CSV with s/p/o columns is not a standard RDF format; it has no URI syntax, prefix mechanism, or datatype support.

Section 3 — Short Answer

SA1. What is a watchdog timer, and give a concrete example of when it would save an IoT device from requiring a manual reset?

A watchdog timer is a hardware countdown register that resets the MCU if software does not "kick" it (write a reset value) within a configured interval. If the firmware enters an infinite loop, deadlocks on a mutex, or hangs waiting for a peripheral, the kick never comes and the MCU reboots automatically. Concrete example: an IoT weather station's I²C sensor driver hangs waiting for an ACK that never arrives (sensor unplugged or bus stuck). Without a watchdog, the device is frozen permanently. With a 10-second watchdog, it reboots, re-initialises the bus, and resumes reporting.

SA2. What does the MQTT retain flag do, and why is it useful for an IoT device that connects intermittently?	When a message is published with <code>retain=true</code>, the broker stores it as the "last known good" value for that topic. Any client that subscribes to the topic later receives this retained message immediately, without waiting for the next publish. This is valuable for intermittently-connected IoT devices: a dashboard that reconnects after a network outage immediately sees the current sensor state rather than showing "unknown" until the device next transmits.
SA3. What is 6LoWPAN, and why is an adaptation layer needed between IPv6 and IEEE 802.15.4?	6LoWPAN is an IETF adaptation layer (RFC 4944, RFC 6282) that sits between IPv6 and IEEE 802.15.4. It is needed because 802.15.4 has a maximum frame size of 127 bytes, whereas an uncompressed IPv6 header alone is 40 bytes, leaving only ~87 bytes for payload after MAC overhead. 6LoWPAN compresses the IPv6 header (often to 2–3 bytes using context-based compression), fragments large IPv6 datagrams across multiple 802.15.4 frames, and reassembles them, making it possible to use standard IPv6 protocols over constrained mesh networks.
SA4. What is the Elixir with expression, and how does it improve readability over deeply nested case expressions?	with chains a series of pattern <code><- expression</code> clauses. If every clause matches, the final <code>do</code> block executes with all bound variables in scope. If any clause fails to match, the <code>else</code> block receives the non-matching value. This avoids the "pyramid of doom" — deeply nested <code>{:ok, x}</code> <code>-></code> blocks — that grows one indent level per fallible step. Instead, the happy path reads as a flat sequence of steps, and error handling is collected in one <code>else</code> block.
SA5. What is LoRa time-on-air, and why does it matter for a device subject to the 1% duty-cycle limit in the EU 868 MHz ISM band?	Time-on-air (ToA) is the duration a LoRa packet occupies the radio channel. It depends on SF, bandwidth, coding rate, and payload length — at SF12, BW 125 kHz, a 20-byte packet takes roughly 1.5–2 seconds. The EU 868 MHz ISM band imposes a 1% duty-cycle limit: per 100 seconds, a device may transmit at most 1 second. If a single transmission takes 2 s, the mandatory off-time is at least 200 s (~3.3 minutes), regardless of application requirements. This forces designers to either reduce SF, shorten payloads, or space messages further apart.
SA6. In FreeRTOS, what is the difference between <code>xTaskCreate</code> and <code>xTaskCreateStatic</code>? When would you use the static variant?	<code>xTaskCreate</code> allocates the task stack and TCB (Task Control Block) from the FreeRTOS heap at runtime; if the heap is exhausted it returns <code>pdFAIL</code>. <code>xTaskCreateStatic</code> takes caller-provided buffers for both the stack and TCB, so no heap allocation occurs — stack memory is typically a statically declared array. Use the static variant in safety-critical or memory-constrained systems where heap fragmentation is unacceptable, or when you need guaranteed allocation at compile time. The trade-off is that you must size the stack array manually and waste memory if over-estimated.
SA7. What is a moving-average filter, and what is the trade-off between a short window (e.g., $N=4$) and a long window (e.g., $N=32$) for smoothing sensor readings?	A moving average computes the arithmetic mean of the last N samples. A short window ($N=4$) responds quickly to genuine signal changes but provides little noise suppression — high-frequency noise still appears in the output. A long window ($N=32$) smooths out noise effectively but introduces latency: a step change in the physical signal takes up to 32 samples to fully propagate to the output, potentially delaying threshold detection. Choose N based on the expected rate of physical change relative to the measurement noise power — faster dynamics require smaller N.

SA8. What is the difference between `#pragma once` and a traditional `#ifndef` include guard in C? Are there any portability concerns?

A traditional include guard uses `#ifndef MYFILE_H / #define MYFILE_H / ... / #endif` to prevent a header from being included twice. `#pragma once` achieves the same effect with a single directive and is supported by all major compilers (GCC, Clang, MSVC). The portability concern is that `#pragma once` is not mandated by the C/C++ standard — it is a widely-supported extension. In practice it is safe for almost all modern embedded toolchains, but legacy or exotic compilers may not support it. `#pragma once` also has a subtle difference: it is file-identity-based (inode or path), while include guards are name-based; this matters in edge cases involving hard links or unusual build systems.

Section 4 — Detailed Answer

DA1. Calculate the approximate time-on-air for a LoRa packet with the following parameters: SF12, BW 125 kHz, CR 4/5, 20-byte payload, explicit header. Show your reasoning. Then explain how the EU 868 MHz 1% duty-cycle limit constrains the minimum inter-message interval, and what this means for the design of a sensor that must report every 5 minutes.

The symbol duration for LoRa is $T_s = 2^{\text{SF}} / \text{BW}$. At SF12, BW 125 kHz: $T_s = 4096 / 125000 \approx 32.77 \text{ ms}$. A preamble of 8 symbols takes $8 \times 32.77 \approx 262 \text{ ms}$. The number of payload symbols is computed by the LoRa formula: with CR=4/5, 20-byte payload, explicit header enabled, the payload symbol count is approximately 40 symbols (using the standard ceiling formula from the Semtech datasheet). That gives $40 \times 32.77 \approx 1311 \text{ ms}$ of payload ToA. Total ToA $\approx 1575 \text{ ms} \approx 1.6 \text{ seconds}$.

The EU 868 MHz 1% duty-cycle rule means: after a 1.6 s transmission, the device must remain silent for at least $1.6 / 0.01 - 1.6 = 158.4 \text{ s} \approx 2.6 \text{ minutes}$. Since 2.6 minutes < 5 minutes, the sensor can legally report every 5 minutes — the duty cycle constraint is satisfied with margin. However, energy per message is high at SF12: at a typical TX current of 120 mA and 3.3 V, $1.6 \text{ s} \approx 0.176 \text{ mJ}$. A device targeting years of battery life on AA cells should consider whether SF7 or SF8 would provide sufficient link budget; at SF7 ToA drops to $\sim 50 \text{ ms}$, reducing energy per message by $\sim 30\times$ and freeing duty-cycle headroom for the same 5-minute interval.

DA2. Explain Elixir's "let it crash" philosophy. When is it the right approach, and when is it wrong to rely on it? Cover: what supervisors do, what kinds of errors are recoverable via restart, what kinds require explicit error handling, and how this differs from defensive programming in C.

"Let it crash" means that instead of writing defensive code to handle every possible error inline, you allow processes to crash on unexpected conditions and rely on a supervisor to restart them in a known-good state. This works because Elixir processes are isolated: a crash affects only that process and its dependents, not the whole system. The supervisor's restart strategy (`:one_for_one`, `:one_for_all`, `:rest_for_one`) determines which sibling processes are also restarted.

When it is right: transient errors that are self-healing on retry — a network timeout, a temporary sensor glitch, a database connection drop. Restarting the process resets all state cleanly without leaving inconsistent data structures. When it is wrong: errors that will recur immediately on restart (a bad configuration, an unreachable dependency at startup) will cause rapid crash-restart loops (hitting the supervisor's max-restart threshold), eventually taking down the supervisor tree. These must be handled explicitly — validate configuration before starting a process, use `:ignore` or `{:error, reason}` returns from `init/1` instead of crashing.

This differs fundamentally from C: in C there is no isolation — a null pointer dereference corrupts the process address space. Defensive `if (ptr == NULL) return -1;` checks are the only option. In Elixir the VM enforces isolation, making "crash and restart" a viable strategy that in practice produces simpler, shorter code with cleaner separation between the happy path and error recovery.

DA3. Explain how stream processing can be structured in an Elixir IoT backend. Compare Enum, Stream, Flow, and GenStage — what each one is, when you would use it, and how they map to the stages of an IoT data pipeline (ingestion → filtering → aggregation → alerting).

Enum is eager: functions like Enum.map/2 and Enum.filter/2 process the entire collection in memory and return a new list. Use it for small, finite datasets where simplicity matters — transforming a batch of sensor readings from a database query.

Stream is lazy: operations are composed into a pipeline that pulls one element at a time, only executing when a terminal function (Enum.to_list, Stream.run) forces evaluation. Use it for large or potentially infinite sequences — reading a CSV file line by line, or processing a GenStage producer's output — to avoid loading the entire dataset into memory.

Flow (from the flow library) adds parallelism: it partitions a Stream across multiple worker processes, processes elements concurrently, and merges results. Use it for CPU-bound batch processing — feature extraction, data normalisation — where the pipeline can be parallelised safely.

GenStage is the foundation for back-pressure-aware producer/consumer pipelines. A producer emits events; consumers demand them using explicit demand signals. If a consumer is slow, demand backs up and the producer slows down, preventing buffer overflow. Use it for the full IoT ingestion pipeline: a producer reads from an MQTT broker or TCP socket, intermediate stages filter and decode, a consumer stage writes to a time-series database. Flow is built on top of GenStage and hides the demand protocol for simpler use cases.

In an IoT pipeline: ingestion (MQTT subscriber → GenStage producer) → decoding/filtering (Flow parallel stage) → aggregation (Stream windowing with Stream.chunk_every) → alerting (GenStage consumer that calls a notification API when a threshold is crossed).

Section 5 — Design Question

Question 18 — Smart Parking Lot

(answer not found)

A city council wants to deploy a smart parking system across a **surface lot of 200 spaces** (400 m × 100 m). Each space has a sensor that detects whether it is occupied. The system should:

- **Show real-time occupancy** on a public display at the lot entrance (updated every 30 seconds).
- **Allow drivers** to query availability via a mobile app.
- **Charge parked vehicles** automatically: the system must know when each vehicle arrived and left in order to bill correctly.
- **Alert maintenance staff** when a sensor appears to have failed (no state change for 7 days).

Each sensor node is battery-powered (2 × AA). There is no wired infrastructure.

Section 3 — Short Answer

Subquestion 18.1 (short answer)

Compare two sensor technologies for detecting vehicle presence (e.g., magnetic vs. ultrasonic). Choose one and justify the choice considering power budget, installation, and false-positive rate.

Inductive loop / magnetometer sensors detect the distortion of Earth's magnetic field caused by a vehicle's steel body. They are passive once deployed, consume very little power (μA range when polling slowly), are immune to rain, snow, and ambient light, and have a very low false-positive rate because only ferromagnetic masses trigger them. The main drawback is that installation requires cutting the loop into the asphalt (inductive loop) or burying the sensor, which is disruptive. A magnetometer surface-mounted in a housing is easier but sensitive to calibration drift.

Ultrasonic sensors measure round-trip time of a 40 kHz pulse to detect distance to the ground (absent vehicle) vs. a vehicle roof. They work regardless of vehicle material but consume more power (mA during active pulse), are affected by temperature (speed of sound varies), and can give false readings from debris, birds, or heavy rain. They are better suited for indoor garages with stable temperature.

Choice: magnetometer. For an outdoor surface lot, the magnetometer wins on power budget (years of AA life vs months for ultrasonic), robustness to weather, and lower false-positive rate. Installation in a small surface-mounted puck is acceptable and avoids road cutting.

Section 4 — Detailed Answer

Subquestion 18.2 (detailed answer)

Design the data pipeline from sensor to mobile app and billing backend. What radio technology do you use from sensor to gateway? Where do you run the occupancy state machine (edge/fog/cloud)? How do you ensure billing accuracy if a gateway reboots mid-session?

Sensor to gateway: Use LoRaWAN. The lot spans $400\text{ m} \times 100\text{ m}$; a single LoRaWAN gateway covers the entire area with margin. Magnetometer sensors change state infrequently (when a car arrives or departs), so duty-cycle constraints are not a problem — occupancy events are short packets (~ 10 bytes) well within the 1% limit. Each node runs on $2 \times \text{AA}$ for 2–5 years at SF9/SF10.

Occupancy state machine: Run on a fog-tier server co-located with the gateway (e.g., a Raspberry Pi or small x86 box on-site). The state machine holds a $\{\text{space_id} \rightarrow \{\text{occupied} \mid \text{free}, \text{arrived_at}\}\}$ map. When a "vehicle detected" event arrives, it records the arrival timestamp; when a "vehicle gone" event arrives, it closes the session. Running this at the fog tier means the state survives internet outages — occupancy counts continue to update, the entrance display (local LAN) continues to refresh, and session records accumulate locally.

Billing accuracy through gateway reboots: Session records (space_id , arrival_time , departure_time) are written to a local SQLite database with a synced flag. On reboot, the fog node re-reads open sessions from the DB and resumes. Once the internet reconnects, it bulk-syncs unsynced records to the cloud billing backend. LoRaWAN Class A devices retransmit unacknowledged uplinks, so a brief gateway reboot does not lose events — they are queued in the device's retransmit buffer. For billing disputes, a tamper-evident log of raw LoRaWAN payloads (timestamped by the gateway hardware clock, synchronised via NTP) serves as the audit trail.

Mobile app and billing: The fog node exposes a REST/WebSocket API consumed by the entrance display and the mobile app (via a cloud reverse-proxy that tunnels to the fog API). The billing backend receives completed session records via HTTPS from the fog node.

Section 3 — Short Answer

Subquestion 18.3 (short answer)

A sensor has reported no state change for 7 days. Describe how you distinguish a genuinely empty space from a failed sensor, and what automated action the system should take.

A sensor that genuinely covers an empty space will report no state change. To distinguish it from a faulty sensor, the system can: (1) check whether the space has been occupied at all in the past 7 days, compared to adjacent spaces that show normal occupancy cycling — a healthy empty space is plausible at 3 AM but implausible if every adjacent space shows daily turnover; (2) send a LoRaWAN downlink "ping" to the node requesting a status response including battery voltage and sensor self-test result — if the node responds with healthy status, the space is genuinely unoccupied; if there is no response after 3 retries across 24 hours, the node is presumed dead.

Automated action: raise a maintenance alert to staff with the space ID and last-seen timestamp, suppress the space from the occupancy count so billing logic does not incorrectly assume it is free, and schedule a physical inspection. The system should not automatically mark the space as "occupied" (which would block billing), but should mark it as "unknown" with reduced confidence.

Practice Exam 5

Question

Model Answer

Section 1 — Multiple Choice

MC1. How does CoAP (Constrained Application Protocol) differ from HTTP?

- A) CoAP runs over TCP and provides ordered reliable delivery; HTTP runs over UDP
- B) CoAP runs over UDP with optional reliability, uses binary headers and 4-byte base header, and is designed for constrained devices; HTTP runs over TCP with verbose text headers
- C) CoAP is a publish-subscribe protocol; HTTP is request-response only
- D) CoAP requires TLS; HTTP can run without encryption

MC2. What is the MQTT Last Will and Testament (LWT)?

- A) A message the broker sends to all subscribers when it shuts down gracefully
- B) A message the client pre-registers with the broker that is published automatically to a specified topic if the client disconnects unexpectedly
- C) A retained message that expires after a configured TTL
- D) A QoS 2 message that guarantees delivery even after a broker restart

B

CoAP is an IETF standard (RFC 7252) designed for constrained nodes. It runs over UDP, has a 4-byte fixed header (vs HTTP's multi-hundred-byte text headers), and provides optional reliability via Confirmable messages with ACK. It maps closely to HTTP's REST model (GET/POST/PUT/DELETE) but is much lighter. A is backwards; C misidentifies it (CoAP is request-response, though it has observe); D is wrong.

B

At connect time the client gives the broker a "will" — a topic, payload, QoS, and retain flag. If the broker detects an unexpected disconnect (no PINGREQ, TCP reset), it publishes that message automatically. Subscribers can use this to detect device failures. A, C, D are wrong.

MC3. In Elixir, how does a process receive messages from its mailbox? • A) By calling <code>Process.read/1</code> with its own PID • B) By using a <code>receive</code> block that pattern-matches against messages in arrival order • C) By subscribing to a named channel via <code>GenServer.subscribe/2</code> • D) Messages are delivered directly to a callback function registered at spawn time	B Elixir processes have a mailbox. <code>receive</code> pattern-matches the first message that matches a clause; unmatched messages stay in the mailbox. A is not a real function; C and D do not exist.
MC4. In FreeRTOS, when two tasks are both ready to run at the same priority level, the scheduler uses: • A) First-come-first-served — the task that became ready first runs until it blocks • B) Round-robin time-slicing — each task gets one tick period before the other runs • C) Priority inheritance — the lower task temporarily raises its priority to match • D) Earliest-deadline-first scheduling	B FreeRTOS uses preemptive scheduling with round-robin for equal-priority tasks. Each task gets one tick period (typically 1 ms) before the scheduler switches to the next ready task at the same priority. A describes cooperative scheduling; C describes priority inheritance (a different mechanism); D is not the FreeRTOS default.
MC5. On an ESP32 in deep sleep, which component remains powered? • A) The main CPU cores and all peripherals • B) Only the RTC (Real-Time Clock) controller and RTC memory; main CPU and most peripherals are powered off • C) The WiFi radio in beacon-monitoring mode • D) The ADC in continuous-sampling mode	B Deep sleep powers off both CPU cores, most RAM, WiFi, BT, and peripherals. Only the RTC controller, RTC slow memory (8 KB), and RTC fast memory remain on, along with optional ULP co-processor. Typical deep sleep current is 5–10 μA. This is why you must store wake-up state in RTC memory. A, C, D are wrong.
MC6. What is the primary difference between SPI and I²C as on-board sensor buses? • A) SPI is wireless; I²C requires a physical connection • B) SPI uses separate clock, MOSI, MISO, and chip-select lines per device (full-duplex, fast); I²C uses two shared lines (SCL, SDA) with device addressing (half-duplex, slower but fewer wires) • C) I²C supports up to 1 Gbps; SPI is limited to 100 kbps • D) SPI devices are addressed by 7-bit I²C addresses; I²C uses chip-select lines	B SPI uses 4 wires (SCK, MOSI, MISO, CS per device) and supports full-duplex operation at speeds up to tens of MHz. I²C uses only 2 wires (SCL, SDA) and addresses up to 127 devices on the same bus, but is half-duplex and typically 100 kbps or 400 kbps. A, C, D are wrong.
MC7. Why is CBOR (Concise Binary Object Representation) preferred over JSON for payloads on constrained IoT devices? • A) CBOR is human-readable and therefore easier to debug on the device	B CBOR (RFC 7049) is the binary equivalent of JSON: it supports the same data model (maps, arrays, strings, numbers, booleans, null) but encodes them in 20–50% fewer bytes with faster parse time. This matters for LoRaWAN where payload is limited to ~51–242 bytes depending on SF. A is wrong (CBOR is binary, not human-readable); C and D are wrong.

• B) CBOR encodes the same data structures as JSON but in a compact binary form, reducing payload size and parse time on MCUs with limited RAM and CPU • C) CBOR supports encryption natively; JSON does not • D) CBOR is the only format supported by LoRaWAN	
MC8. In Elixir, what is the Agent module used for? • A) Running external OS processes and capturing their output • B) Maintaining a piece of shared mutable state in a separate process with a simple get/update interface • C) Sending HTTP requests to external APIs on behalf of IoT devices • D) Supervising a pool of worker processes with back-pressure	B Agent is a thin wrapper around a GenServer that holds a single piece of state and exposes <code>Agent.get/2</code> and <code>Agent.update/2</code>. It is the right abstraction when you need shared state without the full boilerplate of a GenServer. A, C, D are wrong.
Section 2 — Multiple Answer	
MA1. Which of the following are characteristics of LoRaWAN ABP (Activation By Personalisation), compared to OTAA? • A) Session keys (NwkSKey, AppSKey) are hardcoded into the device at manufacture rather than negotiated • B) ABP devices skip the join procedure and can transmit immediately after power-on • C) ABP is more secure than OTAA because keys are never transmitted over the air • D) ABP devices have a fixed frame counter that can cause issues if the device resets without the network server being informed • E) ABP requires a working internet connection at first boot to receive the keys	MA1 A is correct: ABP devices have NwkSKey and AppSKey burned in at manufacture — no join exchange. B is correct: because there is no join procedure, an ABP device can transmit its first uplink immediately after boot. C is wrong: ABP is generally considered less secure than OTAA because the keys are static and never refreshed; a compromised device leaks keys permanently. D is correct: ABP uses a fixed DevAddr and the frame counter is stored in non-volatile memory; if the device resets and the counter resets to 0, the network server may reject frames (replay protection) until the counter catches up — this is a well-known ABP pitfall. E is wrong: ABP needs no internet connection; keys are pre-provisioned.
MA2. Which properties does a BLE GATT characteristic support? • A) Read — client can fetch the current value • B) Write — client can set a new value • C) Notify — server pushes updates to subscribed clients without acknowledgement • D) Indicate — server pushes updates with client acknowledgement • E) Broadcast — value is included in advertising packets without a connection	MA2 All five are valid GATT characteristic properties defined in the Bluetooth specification. Read (A) and Write (B) are the basic client-initiated operations. Notify (C) allows the server to push values without confirmation — useful for high-frequency sensor data where a lost notification is acceptable. Indicate (D) is like Notify but requires an ATT-layer acknowledgement, ensuring delivery at the cost of higher overhead. Broadcast (E) allows a characteristic value to be included in advertising packets so it can be received without establishing a connection — used for beacons.
MA3. Which memory sections exist in a typical C program on an MCU, and what do they hold?	MA3

SA3. How does Elixir's receive block work, and what happens to messages that do not match any clause?	receive blocks the calling process and inspects its mailbox for a message matching one of the listed patterns. Pattern matching proceeds in clause order; the first matching message is consumed and its variables are bound. Messages that match no clause are left in the mailbox — they accumulate and are checked again on the next receive call. If no matching message is present and no after timeout is specified, the process blocks indefinitely. This is important: an unhandled message type in a long-running process that never matches will grow the mailbox unboundedly, eventually causing a memory leak.
SA4. What are FreeRTOS task priorities, and what problem arises when a low-priority task holds a resource needed by a high-priority task?	FreeRTOS tasks have an integer priority (0 = lowest). The scheduler always runs the highest-priority ready task. Priority inversion occurs when a high-priority task (H) blocks waiting for a mutex held by a low-priority task (L), and a medium-priority task (M) preempts L and runs indefinitely. H is starved even though it has the highest priority. FreeRTOS mutexes implement priority inheritance to mitigate this: when L holds a mutex that H is waiting for, L temporarily inherits H's priority so M cannot preempt it, allowing L to release the mutex promptly.
SA5. What is deep sleep on an ESP32, and how does it differ from light sleep in terms of power draw and wake latency?	In deep sleep, both CPU cores, most RAM, WiFi, BT, and almost all peripherals are powered off. Only the RTC controller, RTC memory (8 KB slow + 192 KB fast), and optionally the ULP co-processor remain on. Typical current: 5–10 μA. Wake-up causes a full reset; code restarts from <code>app_main</code> (state must be stored in RTC memory or flash). In light sleep, the CPU is clock-gated (paused) but RAM and peripheral state are preserved. Typical current: 0.8–1 mA. Wake-up is near-instant (microseconds) and execution resumes from the instruction after <code>esp_light_sleep_start()</code>. Choose deep sleep for multi-minute intervals; light sleep for sub-second intervals where you need to preserve state.
SA6. What is I²C, and when would you choose it over SPI for connecting a sensor to an MCU?	I²C (Inter-Integrated Circuit) is a two-wire synchronous serial bus: SCL (clock) and SDA (data). A master generates the clock; up to 127 devices share the bus, each with a unique 7-bit address. Standard mode is 100 kbps, fast mode 400 kbps, fast-plus 1 Mbps. Choose I²C over SPI when: you need to connect many sensors with few GPIO pins (I²C needs only 2 wires regardless of device count, while SPI needs 3 shared wires plus one CS per device), or when bandwidth requirements are modest (most environmental sensors — temperature, humidity, pressure — generate data at well under 1 kbps). Use SPI when you need higher throughput (e.g., a display or fast ADC) or full-duplex transfers.
SA7. What is CBOR, and what specific property of LoRaWAN makes compact payload encoding important?	CBOR (Concise Binary Object Representation, RFC 7049) is a binary data format that encodes JSON-equivalent data structures (maps, arrays, strings, numbers) in significantly fewer bytes — typically 30–50% smaller than equivalent JSON. LoRaWAN's maximum payload size is severely constrained by spreading factor: SF12 at 125 kHz allows only 51 bytes per packet (after MAC overhead). A JSON object like <code>{"t":22.5,"h":61,"id":42}</code> is 24 bytes as JSON but can be ~10 bytes as CBOR. On SF12 with a 1% duty cycle, every saved byte directly reduces time-on-air and extends the number of sensors that can share a gateway.

SA8. What is the Elixir Agent module, and how does it differ from a full GenServer?	Agent is a module in the standard library that wraps a GenServer holding a single term as state. It provides a minimal API: <code>Agent.start_link(fn -> initial_state end)</code>, <code>Agent.get(agent, fn state -> ... end)</code>, and <code>Agent.update(agent, fn state -> new_state end)</code>. It is the right choice when you need shared mutable state (e.g., a running counter, a cache map) without the overhead of writing a full GenServer with named callbacks. If you need complex message handling, timeouts, <code>handle_info</code>, or multiple operations, use a GenServer instead. Agent is essentially GenServer with the boilerplate hidden.
SA9. In C, what is the difference between the stack and the heap? Give one situation where each is the right choice for allocating data in an embedded system.	The stack is a fixed-size LIFO region managed automatically: local variables are allocated on function entry and freed on return. It is fast (single pointer decrement), but size is fixed at compile/link time. Use the stack for short-lived, small, bounded data: local buffers, loop variables, sensor readings. The heap is managed explicitly via <code>malloc/free</code>; allocation can fail and fragmentation accumulates over time. In embedded systems, heap allocation is often avoided entirely: fragmentation can cause unpredictable <code>malloc</code> failures after hours of operation. Use static or stack allocation where possible; use the heap only when data size is truly unknown at compile time (e.g., a variable-length message), and always check the return value of <code>malloc</code>.
SA10. What is an interrupt service routine (ISR) in FreeRTOS, and why must ISRs use the <code>FromISR</code> variants of FreeRTOS API calls?	An ISR (Interrupt Service Routine) is a function called directly by hardware when an interrupt fires — it runs outside any task context, preempting whatever was running. Standard FreeRTOS API calls like <code>xQueueSend</code> may internally call <code>taskYIELD</code> or access scheduler structures that are not interrupt-safe. The <code>FromISR</code> variants (e.g., <code>xQueueSendFromISR</code>, <code>xSemaphoreGiveFromISR</code>) use interrupt-safe access mechanisms and accept a <code>pxHigherPriorityTaskWoken</code> output parameter. If a higher-priority task was unblocked, the ISR sets this to <code>pdTRUE</code> and calls <code>portYIELD_FROM_ISR()</code> at the end, causing an immediate context switch to that task when the ISR returns. Calling normal FreeRTOS APIs from an ISR causes crashes or scheduler corruption.

Section 4 — Detailed Answer

DA1. Describe BLE's GAP and GATT layers in detail. What does GAP define, what does GATT define, and how do they interact? Walk through the complete sequence from device discovery to a mobile app reading a temperature value from a BLE sensor.	GAP (Generic Access Profile) defines how BLE devices discover each other and establish connections. A peripheral device advertises by broadcasting small packets (31 bytes) on the three advertising channels (37, 38, 39) at a configurable interval (20 ms–10 s). The packets contain the device name, supported services (UUIDs), and optionally manufacturer data. A central device (e.g., a smartphone) scans these channels. When it finds an interesting peripheral, it sends a connection request; GAP governs the timing parameters of the connection interval and slave latency. GAP also handles security: pairing modes (Just Works, passkey, OOB) and bonding (storing long-term keys so re-connection does not require re-pairing). GATT (Generic Attribute Profile) defines the data model once a connection is established. It organises data into a hierarchy: a server (the peripheral) holds Services, each containing Characteristics. Each Characteristic has a UUID, a value, and a set of Properties (read, write, notify, indicate). A Descriptor attached to a Characteristic provides metadata — most importantly the Client Characteristic Configuration Descriptor (CCCD) that a central writes to enable notifications.
---	--

DA2. Explain how interrupt-driven sensor reading works in FreeRTOS. Cover: what an ISR is, why you cannot use blocking FreeRTOS calls inside an ISR, how deferred interrupt processing works using a task and a semaphore or queue, and why this pattern is preferred over polling.

Full sequence for a mobile app reading temperature: (1) The sensor peripheral advertises, including the Environmental Sensing Service UUID (0x181A). (2) The phone scans, recognises the UUID, and sends a Connect Request. GAP parameters are negotiated. (3) The phone performs GATT service discovery — it reads the server's attribute table to enumerate Services and their Characteristics. (4) It writes 0x0001 to the Temperature Characteristic's CCCD to enable Notify. (5) The sensor samples temperature every second and calls `ble_gatts_notify()`, pushing the value to the phone without the phone needing to poll. (6) If the phone needs the current value immediately, it issues a GATT Read Request; the peripheral responds with the current value. The phone displays the result.

An ISR is a function registered with the hardware interrupt controller that runs immediately when a peripheral raises an interrupt — for example, a GPIO pin going high when a sensor's data-ready line fires. ISRs run at a higher CPU priority than any FreeRTOS task; the scheduler is not running during the ISR. Because of this, blocking FreeRTOS calls (which yield the CPU to another task) are illegal inside an ISR — they would corrupt scheduler state. The `FromISR` variants are safe because they manipulate the ready list without yielding and defer the actual context switch to the ISR's return.

The deferred interrupt processing pattern: (1) A sensor raises its data-ready line, triggering the ISR. (2) The ISR calls `xSemaphoreGiveFromISR(data_ready_sem, &woken)`. This unblocks a waiting task if one exists, storing the wake signal in the semaphore. (3) The ISR calls `portYIELD_FROM_ISR(woken)` — if a higher-priority task was unblocked, the CPU switches to it immediately when the ISR returns. (4) The sensor-reading task, which was blocked on `xSemaphoreTake(data_ready_sem, portMAX_DELAY)`, wakes up and reads the sensor value over I²C or SPI in a normal task context where all blocking calls are legal.

This is preferred over polling for two reasons: energy efficiency (the reading task sleeps when there is no data, not burning CPU cycles), and latency (the task wakes within microseconds of the interrupt rather than waiting up to one polling interval).

DA3. An IoT edge device runs on 2 × AA batteries (total ~5 Wh). It wakes every 60 seconds, reads an I²C sensor (10 ms, 2 mA), transmits a LoRa packet (1.5 s, 40 mA), and sleeps (deep sleep, 10 μA). Estimate the average current draw and the expected battery life. What is the single biggest lever for extending battery life further?

Average current = sum of (current × fraction of time in that state).

- Sleep: 60 s period; active time ≈ 10 ms (I²C) + 1500 ms (LoRa TX) = 1510 ms ≈ 1.51 s active per cycle.
- Sleep fraction: $(60 - 1.51) / 60 \approx 0.975$; active fraction: $1.51/60 \approx 0.0252$.
- I²C read fraction: $0.010/60 \approx 0.000167$; at 2 mA: 0.33 μA average contribution.
- LoRa TX fraction: $1.5/60 = 0.025$; at 40 mA: 1000 μA = 1.0 mA average contribution.
- Sleep fraction: 0.975; at 10 μA: 9.75 μA average contribution.
- Total average current = $0.33 + 1000 + 9.75 \mu\text{A} \approx \mathbf{1.01 \text{ mA}}$.

2 × AA ≈ 3000 mAh total. Battery life ≈ 3000 mAh / 1.01 mA ≈ **2970 hours ≈ 124 days**.

The single biggest lever: **LoRa transmission time dominates** (99% of active-state current budget). Reducing SF from SF12 to SF7 reduces ToA from ~1.5 s to ~50 ms, cutting the TX current contribution from 1.0 mA to ~33 μ A — dropping average current to ~43 μ A and extending battery life to roughly 8 years (if link budget permits). Alternatively, reducing report frequency from 60 s to 5 minutes would also help significantly.

Section 5 — Design Question

Question 19 — Cold Chain Monitoring

(answer not found)

A logistics company ships refrigerated pharmaceuticals in **50 trucks**, each carrying up to **20 temperature-sensitive pallets**. Each pallet has a small wireless sensor logging temperature and humidity. Requirements:

- Temperature must stay within **2°C–8°C** at all times; any excursion outside this range must trigger an **alert within 60 seconds**.
- Readings must be **logged every 2 minutes** for regulatory audit (data retained 5 years).
- Sensors must last the **full 5-day trip** on a coin cell (CR2032, ~225 mAh).
- The truck has a 4G/LTE gateway but cellular coverage is **not guaranteed** in rural areas.

Section 3 — Short Answer

Subquestion 19.1 (short answer)

BLE (Bluetooth Low Energy) is the best choice. The pallets are inside a truck (max 15 m $\times$ 3 m space), so range is not a constraint — BLE covers this easily at 0 dBm with concrete walls. BLE's advertising mode allows a sensor to broadcast temperature/humidity packets without maintaining a persistent connection, keeping current draw to ~5 μ A average on a CR2032 (sufficient for 5 days). The truck gateway (a small Linux board or smartphone) can run a BLE scanner and aggregate all 20 pallet readings.

Which radio technology connects pallet sensors to the truck gateway, and why? Discuss at least one alternative and why you reject it.

Alternative rejected — LoRaWAN: Massive overkill for in-vehicle range; SF7 ToA is still ~50 ms per packet, and coordinating 20 sensors under the 1% duty cycle limit within the metal truck enclosure is complex. Also, LoRa modules draw more power at TX than BLE, threatening the CR2032 coin cell budget.

Alternative rejected — Zigbee/Thread: Also viable on range and power, but requires a mesh coordinator inside the truck. BLE is simpler to implement and better supported on commodity gateways (phones, Raspberry Pi).

Subquestion 19.2 (short answer)

During the 4-hour cellular outage, the truck gateway continues to: (1) receive BLE readings from all 20 pallet sensors every 2 minutes as normal — the gateway remains the BLE coordinator and this path is unaffected by cellular; (2) evaluate the 2°C–8°C threshold locally and trigger a local alarm (buzzer, dashboard LED, driver alert over CAN) within the 60-second SLA using on-board logic — alerting does not require cloud connectivity; (3) buffer all timestamped readings to local non-volatile storage (SD card or eMMC) with a write-ahead log ensuring durability.

The 4G link drops for 4 hours in a rural area. Describe exactly what happens to alerting, logging, and data delivery during and after the outage.

After connectivity resumes, the gateway bulk-uploads the buffered records to the cloud in chronological order, using MQTT with QoS 1 and a persistent session so the broker acknowledges each batch. The cloud backend deduplicates based on (device_id, timestamp) and stores the complete audit trail. No data is lost; the only consequence of the outage is a 4-hour delay in cloud visibility, not a gap in the audit log.

Section 4 — Detailed Answer

Subquestion 19.3 (detailed answer)

Design the full data pipeline from pallet sensor to pharmaceutical company dashboard and regulatory audit store. Specify where each processing step runs (sensor / truck gateway / cloud), how the 60-second alert SLA is met despite intermittent cellular, and how you guarantee no audit records are lost even if the truck gateway crashes.

Sensor layer (CR2032, BLE): Each sensor samples temperature and humidity every 2 minutes and broadcasts a BLE advertisement packet (16 bytes: sensor ID 2B, temperature 2B, humidity 2B, sequence number 2B, battery voltage 2B, flags 2B + MAC). No connection required — the gateway passively scans. For the 60-second alert SLA, sensors also broadcast immediately on any reading outside 2–8°C, using a second advertising slot with higher advertising frequency (100 ms interval for 5 s) to ensure fast reception.

Truck gateway (fog tier — Raspberry Pi or industrial PC): Runs a BLE scanner process that collects all pallet advertisements and writes decoded readings to a local time-series store (SQLite or InfluxDB Lite). A threshold-monitoring process evaluates each reading against 2–8°C; on excursion it immediately: (a) triggers a local driver alert (CAN bus message to dashboard, audible alarm), and (b) queues an alert message to the MQTT broker with QoS 1. The local alert fires within seconds regardless of cellular state, satisfying the 60-second SLA. A sync process maintains a "high-water mark" of records uploaded to the cloud; on cellular reconnect it resumes from the mark. The local store retains all data for the 5-day trip plus 7 days of margin.

Cloud tier: An MQTT broker (e.g., AWS IoT Core or HiveMQ) receives telemetry and alert topics. A stream processor (e.g., AWS Kinesis or Apache Kafka) fans data to: (a) a time-series database (InfluxDB or TimescaleDB) for the regulatory audit store with a 5-year retention policy and immutable write semantics; (b) a real-time dashboard (Grafana) showing current temperature per pallet per truck; (c) an alerting service that sends push notifications and SMS to fleet managers on excursions. The audit store uses append-only writes with cryptographic hashing of each batch so data integrity can be proven to regulators.

Crash recovery: The gateway writes each reading to local storage before acknowledging the BLE packet. The MQTT client uses a persistent session (CleanSession=false) so the broker queues QoS 1 messages during gateway restarts; on reconnect they are delivered. The local store's high-water mark ensures no record is uploaded twice.

Practice Exam 6

Question

Model Answer

Section 1 — Multiple Choice

MC1. Sigfox uses Ultra-Narrow Band (UNB) radio. What does this mean for its data rate and interference resilience?

B

• A) Very wide channel (500 kHz), high data rate (~250 kbps), vulnerable to narrowband interference • B) Very narrow channel (~100 Hz), extremely low data rate (100–600 bps), highly resilient to wideband noise because energy is concentrated in a tiny slice of spectrum • C) Frequency-hopping spread spectrum across 80 channels, moderate data rate (~1 Mbps) • D) Direct-sequence spread spectrum, 2 Mbps, used in the 5 GHz band	Sigfox transmits on a channel of roughly 100 Hz width, concentrating all transmit power into an extremely narrow slice of the 868/915 MHz band. This makes the signal very hard to drown out with broadband noise, giving excellent range (~40 km rural). The trade-off is a maximum data rate of 100 bps (uplink) / 600 bps (downlink), and strict limits on message count. A, C, D describe other spread-spectrum technologies.
MC2. In Elixir, what is the difference between <code>Process.link/1</code> and <code>Process.monitor/1</code>? • A) <code>link</code> creates a bidirectional dependency — if either process dies the other receives an exit signal; <code>monitor</code> is unidirectional — the monitoring process receives a <code>DOWN</code> message but does not die • B) <code>link</code> is only for <code>GenServers</code>; <code>monitor</code> works with all process types • C) <code>monitor</code> kills the monitored process when the monitor is removed; <code>link</code> does not • D) They are identical; <code>monitor</code> is the newer alias for <code>link</code>	A <code>Process.link(pid)</code> creates a bidirectional link: if either process exits abnormally, the other receives an exit signal and also dies (unless it traps exits). <code>Process.monitor(pid)</code> is one-way: the monitoring process receives a <code>{:DOWN, ref, :process, pid, reason}</code> message but continues running. Use <code>link</code> for tightly coupled processes that must fail together; use <code>monitor</code> when you want to observe another process's lifecycle without coupling your own survival to it.
MC3. FreeRTOS software timer callbacks execute in the context of: • A) The task that called <code>xTimerStart</code> • B) The dedicated RTOS timer service task (daemon task) • C) The idle task, when no other task is ready • D) An interrupt handler triggered by the hardware timer peripheral	B FreeRTOS creates a dedicated timer service task (also called the RTOS daemon task) at startup. All software timer callbacks execute within this task's context. This means callbacks must be short and non-blocking — a blocking call in a callback starves all other timer callbacks queued behind it. A is wrong (the starting task only calls the API); C and D are wrong.
MC4. In C, <code>__attribute__((packed))</code> applied to a struct tells the compiler to: • A) Place the struct in read-only flash memory • B) Remove all padding bytes between members, making the struct layout match the byte order of fields exactly as declared • C) Align the struct to a 64-byte cache line for SIMD instructions • D) Mark the struct as thread-safe for concurrent access	B By default the C compiler inserts padding bytes between struct members to satisfy alignment requirements (e.g., a <code>uint32_t</code> must be 4-byte aligned). <code>__attribute__((packed))</code> removes all padding, making the struct's in-memory layout identical to the declared field order with no gaps. Useful for mapping hardware register layouts or network protocol headers where exact byte positions are specified. A, C, D are wrong.
MC5. Why is DTLS (Datagram TLS) used with CoAP instead of standard TLS?	B

• A) DTLS uses less CPU than TLS because it skips encryption • B) Standard TLS requires TCP's ordered, reliable stream; CoAP runs over UDP, so DTLS provides the same security guarantees (authentication, encryption, integrity) adapted for unreliable datagrams • C) DTLS compresses CoAP payloads before encryption • D) TLS is patented; DTLS is royalty-free	TLS was designed for TCP's byte-stream abstraction; it assumes ordered, reliable delivery. CoAP uses UDP, which can drop, reorder, or duplicate datagrams. DTLS (RFC 6347) is TLS adapted for datagrams: it adds explicit sequence numbers, retransmission of handshake messages, and handles reordering, providing the same mutual authentication, encryption, and integrity as TLS. A is wrong (DTLS uses the same cipher suites); C and D are wrong.
MC6. What is the purpose of the NVS (Non-Volatile Storage) partition on an ESP32? • A) Storing compiled bytecode for MicroPython scripts • B) Storing small key-value pairs (strings, integers, blobs) that persist across reboots, such as WiFi credentials and calibration data • C) A RAM-backed cache that is flushed to flash on deep sleep entry • D) The OTA update staging area where a new firmware image is written before being activated	B NVS (Non-Volatile Storage) is an ESP-IDF library that manages a flash partition as a key-value store. It handles wear levelling across pages, power-failure safety (it writes to a new page before invalidating the old one), and provides a simple API: <code>nvs_set_str</code>, <code>nvs_get_i32</code>, etc. Typical uses: storing WiFi SSID/password after provisioning, calibration offsets, device configuration. A, C, D describe other flash uses.
MC7. In Elixir, how does a <code>DynamicSupervisor</code> differ from a regular <code>Supervisor</code>? • A) A <code>DynamicSupervisor</code> can only supervise <code>GenServer</code> processes; a regular <code>Supervisor</code> can supervise any OTP behaviour • B) A <code>DynamicSupervisor</code> starts with no children; children are added at runtime via <code>DynamicSupervisor.start_child/2</code>, making it suitable for spawning workers on demand • C) A <code>DynamicSupervisor</code> restarts children immediately without any backoff; a regular <code>Supervisor</code> uses exponential backoff • D) A <code>DynamicSupervisor</code> shares state between all its children; a regular <code>Supervisor</code> isolates them	B A regular <code>Supervisor</code> has a fixed child spec list defined at compile time in <code>init/1</code>. A <code>DynamicSupervisor</code> starts empty and children are added at runtime with <code>DynamicSupervisor.start_child/2</code>. This is the right choice when the number of workers is not known at compile time — for example, spawning one <code>GenServer</code> per connected IoT device. A, C, D are wrong.
MC8. The Shannon-Hartley theorem states that channel capacity $C = B \log_2(1 + S/N)$. If you double the bandwidth B while keeping S/N constant, the capacity: • A) Quadruples • B) Doubles • C) Increases, but less than doubles, because the noise power also doubles with bandwidth • D) Stays the same, because S/N degrades proportionally	B Shannon-Hartley: $C = B \log_2(1 + S/N)$. Doubling B doubles the first factor and keeps $\log_2(1 + S/N)$ unchanged (assuming S/N is held constant by adjusting signal power proportionally), so C doubles. Note: in practice, thermal noise power scales with bandwidth ($N = kTB$), so S/N falls as B grows unless transmit power also grows — but the question stipulates S/N is constant. A, C, D are wrong under that assumption.

Section 2 — Multiple Answer

MA1. Which of the following are characteristics of Sigfox (compared to LoRaWAN)?

- A) Uplink only — devices cannot receive downlink messages at arbitrary times
- B) Maximum payload of 12 bytes per uplink message
- C) Maximum 140 uplink messages per device per day
- D) Uses licensed spectrum, requiring a carrier agreement like NB-IoT
- E) Infrastructure (base stations) is operated exclusively by the Sigfox network operator — you cannot deploy your own

MA2. Which of the following are valid use cases for Elixir ETS (Erlang Term Storage) tables?

- A) A fast in-memory lookup table shared between multiple processes without message passing
- B) Caching computed results that are read frequently but written rarely
- C) Durable storage that persists data across application restarts
- D) A process registry mapping names to PIDs
- E) Rate-limiting counters accessed concurrently by many request-handler processes

MA3. Which of the following are required properties of a safe OTA (Over-The-Air) firmware update mechanism for an embedded device?

- A) The device must be able to verify the integrity and authenticity of the new image (e.g., cryptographic hash and signature check) before applying it
- B) The update must be applied atomically — the device must never boot a partially-written image

MA1

A is correct: Sigfox devices are limited to 4 downlink messages per day, and downlink only occurs in a narrow window after an uplink — there is no arbitrary server-initiated downlink. B is correct: maximum uplink payload is 12 bytes; downlink is 8 bytes. C is correct: the network limits each device to 140 uplink messages per day (roughly one every 10 minutes). D is wrong: Sigfox uses unlicensed ISM spectrum (868 MHz EU, 902 MHz US), not licensed spectrum — no SIM or carrier contract is needed. E is correct: Sigfox operates as a network operator model; you pay a subscription to use their infrastructure and cannot deploy your own base stations.

MA2

A is correct: ETS tables can be read and written by any process without message passing, making them ideal for shared lookup tables. B is correct: ETS is frequently used as an in-process cache for data that is expensive to compute and read often but updated infrequently. C is wrong: ETS is purely in-memory; it does not persist across application restarts (use DETS or a database for persistence). D is correct: Registry (the Elixir standard library module) is itself built on ETS and is a standard use case for name-to-PID mappings. E is correct: a shared ETS table with atomic `update_counter` operations is a common pattern for per-key rate-limit counters in high-concurrency servers.

MA3

A is correct: the device must verify a cryptographic hash (integrity) and signature (authenticity) before applying any image — otherwise a corrupted download or a malicious image could brick the fleet. B is correct: if power is lost mid-write, the device must not attempt to boot the partial image; the bootloader must only switch to the new partition after the write is confirmed complete. C is correct: rollback is essential — if the new firmware crashes on first boot or fails a self-test, the device must revert to the known-good previous version to avoid a permanent outage. D is wrong: many OTA implementations correctly apply updates while in a low-power maintenance mode; there is no requirement to remain fully operational. E is wrong: re-flashing the bootloader is dangerous (a failed bootloader flash bricks the device permanently) and is almost never required as part of a normal application OTA.

- C) The previous firmware version must be retained so the device can roll back if the new version fails to boot or fails a self-test
- D) The update must be applied while the device is fully operational, without entering any low-power or maintenance mode
- E) The device must re-flash the bootloader as part of every OTA update

MA4. Which properties distinguish a time-series database (e.g., InfluxDB, TimescaleDB) from a general-purpose relational database for IoT sensor data?

- A) Data is indexed and queried primarily by time range, making range scans much faster than in a row-oriented RDBMS

- B) Write throughput is optimised for high-frequency sequential inserts; time-series DBs typically achieve 10–100× higher ingest rates than PostgreSQL for append-only workloads

- C) Time-series DBs support automatic downsampling and retention policies, expiring or aggregating old data without manual intervention

- D) Time-series DBs enforce foreign key constraints between sensor readings and device metadata

- E) Compression is specialised for time-ordered numeric data (e.g., delta-of-delta encoding), achieving much better ratios than general-purpose compression

MA4

A is correct: the primary query pattern is `WHERE time BETWEEN t1 AND t2`; time-series DBs maintain a time-ordered index that makes this $O(\log n)$ or even $O(1)$ with chunk-based storage. B is correct: InfluxDB and TimescaleDB are benchmarked at 10–100× the ingest rate of vanilla PostgreSQL for high-frequency sensor data because they batch and compress writes. C is correct: retention policies (e.g., "keep raw data for 30 days, then keep 5-minute averages for 2 years") are a built-in feature — crucial for managing storage over multi-year IoT deployments. D is wrong: time-series DBs typically do not support relational foreign key constraints — metadata joins are usually done at the application layer or in a companion relational DB. E is correct: delta encoding (store differences rather than absolute values), delta-of-delta, and XOR-float compression (Gorilla algorithm) yield 10–30× compression on slowly-changing sensor streams.

Section 3 — Short Answer

SA1. What is Sigfox, and what are two concrete limitations that make it unsuitable for applications requiring frequent or large data transmissions?

Sigfox is an LPWAN operated by a single network provider using ultra-narrow band (UNB) radio on unlicensed ISM spectrum. Two concrete limitations: (1) Maximum 140 uplink messages per device per day — a device reporting every 10 minutes exhausts its daily budget in under 24 hours; a sensor that needs per-minute reporting cannot use Sigfox at all. (2) Maximum 12 bytes per uplink payload — a JSON or CBOR object with multiple sensor readings may not fit; a temperature + humidity + battery voltage reading in 12 bytes leaves no room for metadata or error codes. Combined, these make Sigfox unsuitable for anything requiring frequent, multi-value, or bidirectional communication.

SA2. What is the difference between `Process.link` and `Process.monitor` in Elixir, and when would you choose each?

`Process.link(pid)` creates a bidirectional crash dependency. If either linked process exits abnormally, the other receives an `:EXIT` signal and also terminates (unless it traps exits with `Process.flag(:trap_exit, true)`). Use it when two processes form a unit and neither should outlive the other — e.g., a connection handler and its reader task. `Process.monitor(pid)` is one-way: the monitoring process receives a `{:DOWN, ref, :process, pid, reason}` message when the monitored process exits, but continues running. Use it when you want to observe another process's lifecycle without coupling your own — e.g., a supervisor watching a pool of worker PIDs.

SA3. What are FreeRTOS software timers, and why must their callbacks be kept short and non-blocking?	Software timers are FreeRTOS objects that call a user-supplied callback after a specified period (one-shot or auto-reload). They are implemented entirely in software — no dedicated hardware timer peripheral is used. All callbacks run in the context of the timer service task (daemon task), a single dedicated FreeRTOS task created at scheduler start. Because all timer callbacks share this one task, a callback that blocks (e.g., calls <code>vTaskDelay</code>, takes a mutex, or waits on a queue) prevents all other timer callbacks from firing on time, and can starve the timer service entirely. Callbacks must complete quickly: update state, give a semaphore, send to a queue — then return.
SA4. What does <code>__attribute__((packed))</code> do in C, and give one situation where it is essential and one where it introduces a performance problem?	<code>packed</code> removes the compiler-inserted padding that normally aligns struct members to their natural alignment boundary. It is essential when the struct must exactly mirror a hardware register map or an on-wire protocol frame, where the specification mandates specific byte offsets. For example, parsing a LoRaWAN MAC header from a received byte array is safe only if the C struct has no padding. The performance problem: on ARMv7-M and similar architectures, accessing a <code>uint32_t</code> at an odd address causes a hardware fault or a slow unaligned-access trap. If a packed struct contains a <code>uint32_t</code> field that ends up at an offset not divisible by 4, every access incurs a multi-cycle penalty or a fault, making packed structs dangerous in performance-critical or safety-critical code.
SA5. What is DTLS, and at what layer of the CoAP stack does it sit?	DTLS (Datagram Transport Layer Security, RFC 6347) is the UDP equivalent of TLS. It provides mutual authentication (via certificates or pre-shared keys), encryption, and message integrity for datagram protocols. It adds a record sequence number and retransmission logic for handshake messages to cope with packet loss. In the CoAP stack, DTLS sits between UDP and CoAP — the same layer as TLS sits between TCP and HTTP. CoAP messages are encrypted and authenticated before being handed to UDP; the recipient DTLS layer decrypts and verifies them before passing plaintext up to CoAP.
SA6. What is the ESP32 NVS partition, and what happens if you exceed its write endurance?	NVS is an ESP-IDF library that implements a wear-levelled key-value store in a dedicated flash partition. It writes new values to fresh pages and marks old pages as stale, spreading writes across the partition to extend flash lifetime. Standard NOR flash has an endurance of ~100,000 erase cycles per sector. If a value is written frequently (e.g., a counter incremented every second), NVS eventually exhausts the partition's endurance — at that point <code>nvs_set_*</code> calls start returning <code>ESP_ERR_NVS_NO_FREE_PAGES</code> or similar errors. The device can no longer store configuration or calibration data, and if the partition is corrupted, factory reset or reflash is required. For high-frequency writes, use a RAM buffer and commit to NVS only on significant changes or on shutdown.
SA7. What is a <code>DynamicSupervisor</code> in Elixir, and give a concrete IoT backend scenario where it is the right choice over a static supervisor?	<code>DynamicSupervisor</code> is a supervisor variant that starts with no children and allows new children to be added at runtime via <code>DynamicSupervisor.start_child/2</code>. It applies the same restart strategy (<code>:one_for_one</code> only) and maximum-restart limits as a regular <code>Supervisor</code>, but the child list grows and shrinks dynamically. Concrete IoT backend scenario: a MQTT subscriber process that spawns one <code>GenServer</code> per connected device to hold that device's state. At boot, no devices are connected; as devices join, <code>DynamicSupervisor.start_child(MyDeviceSupervisor, {DeviceServer, device_id})</code> creates a supervised worker for each. If a device's <code>GenServer</code> crashes, the supervisor restarts it. When the device disconnects, <code>DynamicSupervisor.terminate_child/2</code> removes it cleanly.

SA8. What does the Shannon-Hartley theorem tell us about improving a radio link's data rate, and why can't you always just increase bandwidth?

Shannon-Hartley states $C = B \log_2(1 + S/N)$: maximum error-free capacity grows linearly with bandwidth B and logarithmically with signal-to-noise ratio S/N . To double capacity you can double bandwidth, but doubling S/N only adds one bit per symbol. The implication for IoT: you cannot arbitrarily increase bandwidth because (a) spectrum is a licensed or regulated resource — ISM bands have fixed widths, and (b) thermal noise power scales with bandwidth ($N = kTB$), so doubling B halves S/N if transmit power is fixed, partially cancelling the gain. In practice LoRa chooses narrow bandwidths (125–500 kHz) and accepts low data rates to maximise link budget for long range, accepting the Shannon limit as a fundamental constraint.

SA9. What is a CRC (Cyclic Redundancy Check), how does it differ from a simple byte checksum, and where in the LoRa/LoRaWAN stack is a CRC applied?

A simple checksum sums all bytes modulo 256 (or 16-bit, etc.). It detects single-byte errors but misses many multi-byte error patterns — two bytes can be swapped without changing the sum. A CRC (Cyclic Redundancy Check) treats the data as a polynomial and divides it by a generator polynomial, storing the remainder. CRC-16 and CRC-32 detect all single-bit errors, all burst errors up to the CRC length, and a much larger fraction of random multi-bit errors than a checksum. In the LoRa physical layer, an optional CRC-16 is appended to the payload; if the receiver's CRC check fails, the packet is discarded at the PHY layer before being passed to the LoRaWAN MAC stack. LoRaWAN itself uses a 4-byte MIC (Message Integrity Code) based on AES-CMAC at the MAC layer for authentication.

SA10. What is UART, and how does it differ from SPI and I²C in terms of connection topology and typical use cases in embedded IoT systems?

UART (Universal Asynchronous Receiver/Transmitter) is point-to-point, full-duplex, asynchronous (no shared clock — baud rate is agreed in advance). Each connection is one device to one device over two wires (TX, RX). Used for GPS modules, GSM modems, debug serial output — anything that is a single external module communicating in a human-readable or binary stream. I²C is a shared two-wire bus (SCL, SDA) supporting up to 127 devices; half-duplex, synchronous, 100–1000 kbps. Best for short-range, low-speed sensor clusters where pin count matters. SPI is a four-wire bus (SCK, MOSI, MISO, CS) with one CS line per device; full-duplex, synchronous, up to tens of MHz. Best for high-speed peripherals — displays, ADCs, SD cards — where bandwidth matters more than pin count.

Section 4 — Detailed Answer

DA1. Explain how OTA firmware updates work on a constrained embedded device. Cover: the dual-partition (A/B) flash layout, how the bootloader selects which partition to run, the role of a cryptographic signature check, what a rollback watchdog does, and what can go wrong if any of these mechanisms is absent.

A safe OTA system uses a dual-partition (A/B) flash layout. The flash contains: bootloader (small, rarely updated), partition table, two equal-sized application partitions (slot A and slot B), and auxiliary partitions (NVS, OTA data). The OTA data partition records which slot is "active" and which is "pending". At runtime, the device downloads the new firmware image over the network and writes it to the inactive slot — this is safe because the running slot is not touched. Once the write is complete, the device computes a SHA-256 hash of the written image and verifies it against a hash embedded in the manifest. It also verifies a digital signature (e.g., ECDSA over the hash) against a public key stored in the bootloader or a trusted partition — this prevents a malicious or corrupted image from being applied.

	The OTA data partition is updated to mark the new slot as "pending validation." On next reboot, the bootloader reads this flag and boots from the new slot. The application then runs a self-test (checks peripherals, network connectivity, expected firmware version). If the self-test passes within a configurable timeout, the application calls <code>esp_ota_mark_app_valid_cancel_rollback()</code> to confirm the new slot as active. If the self-test fails or the application crashes before confirming, the rollback watchdog (a hardware watchdog or a boot-count counter in NVS) triggers: on next reboot the bootloader detects that the new slot was never confirmed and reverts to the previous slot. What goes wrong without each mechanism: without the signature check, a MITM attack or storage corruption can brick a fleet with a bad image. Without atomicity (write to inactive slot), a power loss mid-write produces a half-written slot — booting it causes a crash. Without rollback, a new firmware that works electrically but fails to connect to the network leaves devices permanently unreachable. Without the watchdog, an application that crashes in a loop never self-heals.
DA2. Explain Elixir ETS tables in depth. What is ETS, how does it differ from a GenServer's state, what access patterns does it support, what are its consistency and isolation guarantees, and when should you prefer ETS over a GenServer?	ETS (Erlang Term Storage) is an in-memory table store built into the BEAM VM. Unlike a GenServer's state (which is private to one process and accessed only via message passing), an ETS table is a shared data structure accessible by any process in the same node without sending a message. ETS tables support four table types: <code>:set</code> (unique key, $O(1)$ lookup), <code>:ordered_set</code> (sorted keys, $O(\log n)$), <code>:bag</code> (multiple values per key), and <code>:duplicate_bag</code>. A table is owned by a process; when the owner dies, the table is garbage collected (unless <code>heir</code> is configured). Access patterns: <code>ets:lookup/2</code> returns all rows for a key; <code>ets:insert/2</code> inserts or replaces; <code>ets:select/2</code> matches multiple rows against a pattern. Reads are concurrent — many processes can read simultaneously with no locking. Writes lock only the affected row (for <code>:set/:ordered_set</code>), so concurrent inserts to different keys proceed in parallel. Consistency: ETS is not transactional. There is no multi-row atomicity — two concurrent inserts may interleave. There is no journaling — a BEAM crash loses all ETS data. Use ETS when: data is read far more often than written (cache), the data is ephemeral (per-session state), or you need sub-microsecond lookup without message-passing overhead. Use a GenServer instead when: you need atomic multi-step updates (read-modify-write), you need the server to perform logic on the data, or you want backpressure (the GenServer's mailbox naturally queues callers). Prefer ETS for a per-sensor "last seen" lookup table in an MQTT ingestion pipeline: thousands of requests per second need to check whether a sensor ID is registered, and a GenServer would become a bottleneck — all reads serialise through its mailbox. ETS allows all reader processes to check concurrently.
DA3. Compare Sigfox, LoRaWAN, and NB-IoT across the following dimensions: spectrum (licensed vs unlicensed), maximum payload size, downlink capability, approximate maximum messages per day, infrastructure ownership, and a concrete IoT use case where each is the best fit.	Sigfox uses unlicensed ISM spectrum with UNB modulation. Max uplink payload: 12 bytes. Downlink: severely limited (4 messages/day, only after an uplink). Max messages: 140 uplinks/day. Infrastructure: closed, Sigfox-operated only. Best use case: simple asset tracking or tamper detection where a device sends a 1-byte alert at most a few times per day and never needs a downlink command — e.g., a container seal that transmits "opened" once.

LoRaWAN uses unlicensed ISM spectrum with CSS modulation. Max payload: 51–242 bytes depending on SF. Downlink: two receive windows after each uplink (Class A), or listen continuously (Class C). Messages: limited by duty-cycle regulation (~140–1000/day depending on SF and region). Infrastructure: deploy your own gateway or use a public network (TTN, Helium). Best use case: a distributed environmental sensor network where an operator wants full control of the infrastructure, needs to send occasional downlink commands (e.g., reconfigure sampling rate), and has a mix of device densities — e.g., smart agriculture over several square kilometres.

NB-IoT uses licensed LTE spectrum (700–900 MHz), operated by mobile carriers. Max payload: ~1600 bytes per message (much larger than LoRa/Sigfox). Downlink: full bidirectional at any time (subject to PSM/eDRX sleep cycles). Messages: essentially unlimited (billed by data volume or subscription). Infrastructure: mobile network operator — no self-deployment. Best use case: applications needing guaranteed QoS, larger payloads, or server-initiated downlink — e.g., a smart water meter that must accept firmware updates or configuration changes pushed by the utility at any time, or a medical device requiring SLA-backed connectivity.

Section 5 — Design Question

Question 20 — Smart Water Leak Detection

(answer not found)

A property management company wants to deploy leak detection across a **200-unit apartment building**. Each apartment has **3 water sensors** (under the kitchen sink, bathroom, and washing machine).

Requirements:

- **Shut an electric valve** in the apartment's water supply within **30 seconds** of any sensor detecting water.
- **Alert the building manager** with the apartment number and sensor location.
- **Log all events** (detections, valve operations, sensor battery levels) for insurance purposes (retain 10 years).
- Sensors must last at least **2 years** on a single coin cell (CR2032, ~225 mAh).

The building has concrete floors and walls. There is a wired Ethernet backbone in the utility rooms on each floor, but no WiFi or wired infrastructure inside apartments.

Section 3 — Short Answer

Subquestion 20.1 (short answer)

Zigbee (IEEE 802.15.4 with the Zigbee stack) is the best choice. It operates at 2.4 GHz, supports mesh networking (sensors route through each other through concrete walls), and has a typical latency of 15–30 ms per hop — well within the 30-second requirement even through several hops. Zigbee end devices sleep between events and draw ~5 µA in sleep mode; on a CR2032 (225 mAh) with one wake per minute for a heartbeat, battery life exceeds 2 years comfortably. A Zigbee coordinator on each floor connects to the wired Ethernet backbone.

Which radio technology connects sensors to the building network, and why? Consider the 30-second latency requirement, building construction, and the coin-cell power budget.

	Alternative rejected — BLE: BLE mesh (Bluetooth Mesh) is viable on range and power, but its flooding-based mesh has higher latency variability and the specification is more complex to implement reliably on bare MCUs. BLE advertising intervals of 100 ms and mesh relay delays can add up to seconds in a 200-unit building — marginal for a 30-second SLA. Alternative rejected — LoRaWAN: Excellent range through concrete, but duty-cycle limits and 1–2 second ToA at SF9/SF10 make it unsuitable for the 30-second actuation requirement with 600 sensors needing simultaneous event reporting during a burst (e.g., a pipe burst triggers many sensors at once, all exceeding the duty cycle budget).
Subquestion 20.2 (short answer) The valve must shut within 30 seconds. Where does the threshold logic run, and why does it not run in the cloud?	The threshold logic (water detected → close valve) must run on the per-floor Zigbee coordinator / fog gateway, not in the cloud. Cloud latency is 50–300 ms in the best case and can be seconds or unavailable during a network outage — both are incompatible with a 30-second hard deadline when a burst pipe can flood an apartment in minutes. The gateway receives the Zigbee alert from the sensor, immediately sends a Zigbee command to the valve actuator in the same apartment, and only then forwards the alert to the cloud. The complete local round-trip (sensor → gateway → valve) takes well under 1 second over Zigbee mesh. This architecture ensures the valve closes even when the building's internet connection is down.
Section 4 — Detailed Answer	
Subquestion 20.3 (detailed answer) Design the full system: sensor to valve actuation, sensor to management alert, and long-term audit log. How does the system behave if the per-floor gateway loses power? How do you test that valves and sensors are still functional without causing a real water shutoff?	Sensor layer: Each sensor node contains a capacitive or resistive water-contact probe, an MCU, and a Zigbee radio. Normally the sensor sleeps for ~55 seconds, wakes, checks the probe (5 ms), sends a heartbeat if no water is present, and returns to sleep. On water detection it immediately sends a high-priority Zigbee alarm frame and re-sends every 5 seconds until acknowledged. The valve actuator is a motorised ball valve with its own Zigbee radio; it listens as a Zigbee router (always-on, mains-powered from the building's electrical supply). Per-floor gateway (fog tier): A small Linux SBC (e.g., Raspberry Pi or industrial PC) with a Zigbee USB coordinator connects to the floor's Ethernet switch. It runs a local MQTT broker and a state machine for each apartment: on receiving a water alarm from any of the 3 sensors in apartment N, it immediately publishes a Zigbee close-valve command to valve N and sends a local MQTT message to the building management system over Ethernet. It logs all events to a local SQLite database with write-ahead logging. If the floor gateway loses power, the Zigbee mesh remains intact (sensors and valves continue routing through each other), but no new valve commands can be issued — a UPS on the gateway (30-minute backup) covers most power transients.

Building management server (fog/cloud boundary): An on-premises server or cloud service receives MQTT messages from all floor gateways. It runs the alert dashboard for the building manager (push notification with apartment number and sensor location) and writes all events to a long-term audit database (PostgreSQL with TimescaleDB extension, 10-year retention policy, daily backups to object storage). For insurance purposes, each record includes: timestamp, sensor ID, apartment number, sensor location, event type (water_detected / valve_closed / heartbeat / battery_low), and a sequence number for gap detection.

Health monitoring and testing: Each sensor sends a heartbeat every 60 seconds. If no heartbeat is received for 5 minutes, the gateway flags the sensor as potentially failed and sends an alert to the building manager. To test valves without shutting off water: the gateway sends a "partial close" command (if the valve supports it) or sends a test pulse command that actuates the valve solenoid briefly without fully closing — confirming mechanical function. To test sensors without real water, maintenance staff use a small spray bottle during annual inspection; the test mode flag prevents the valve from actually closing during a maintenance session.

Practice Exam 7

Question

Model Answer

Section 1 — Multiple Choice

MC1. In the context of IoT data annotation, what are the three orthogonal axes every measurement should carry?

B

- A) Value, Unit, Precision
- B) Identity, Time, Space
- C) Device, Firmware, Network
- D) Sensor, Gateway, Cloud

Identity (which device/entity produced the reading), Time (when the event occurred), and Space (where the device was) are the three orthogonal annotation axes introduced in the course. Without any one of them, the data is ambiguous or uninterpretable in a multi-device, multi-location system. A, C, D are not the defined framework.

MC2. What is trilateration, and how does it differ from triangulation?

B

- A) Trilateration uses three angle measurements from known points; triangulation uses three distance measurements
- B) Trilateration computes position from distances to three or more known reference points; triangulation uses angle measurements from known baselines
- C) They are synonyms — both describe GPS positioning using four satellites
- D) Trilateration is for 2D positioning; triangulation is for 3D only

Trilateration computes position by measuring distances (e.g. signal travel time or RSSI) to three or more reference points with known positions, then finding the intersection of circles/spheres. Triangulation uses angles measured from a baseline to locate a target — used in surveying and some radar systems. GPS uses trilateration (range from satellites). A swaps the definitions; C and D are wrong.

MC3. What does HDOP (Horizontal Dilution of Precision) quantify in GPS?

B

MC7. In the Big Data context, what are the "3 Vs"?

- A) Volume, Velocity, Variety
- B) Validity, Veracity, Value
- C) Virtualisation, Visualisation, Verification
- D) Volume, Visibility, Volatility

MC8. In an ESP32 application using ESP-IDF, what event signals that the device has received an IP address after connecting to a WiFi access point?

- A) `WIFI_EVENT_STA_CONNECTED`
- B) `IP_EVENT_STA_GOT_IP`
- C) `WIFI_EVENT_AP_STACONNECTED`
- D) `WIFI_EVENT_STA_DISCONNECTED`

A

The original Big Data "3 Vs" coined by Doug Laney: Volume (amount of data), Velocity (speed at which data is generated and must be processed), Variety (range of data types and sources). B lists additional Vs (Veracity, Value) that were added later. C and D are wrong.

B

In ESP-IDF, `WIFI_EVENT_STA_CONNECTED` fires when the 802.11 association succeeds but before DHCP completes. `IP_EVENT_STA_GOT_IP` fires once DHCP has assigned an IP address and the device is fully ready to open sockets. Applications should wait for `IP_EVENT_STA_GOT_IP` before attempting any TCP/UDP connections. A fires too early; C is for AP mode; D is the disconnect event.

Section 2 — Multiple Answer

MA1. Which of the following are valid direct position sources for an IoT device?

- A) GPS / GNSS receiver on the device
- B) WiFi RSSI triangulation from known access points
- C) A hardcoded GPS coordinate written into flash at deployment
- D) UWB (Ultra-Wideband) ranging to fixed anchors
- E) Cell-tower ID (Cell-ID) lookup

MA2. Which of the following correctly describe how OWL (Web Ontology Language) extends plain RDF/RDFS?

MA1

A is correct: an on-device GPS/GNSS receiver directly measures its own position from satellite signals — a direct source. B is wrong: WiFi RSSI triangulation is an indirect source — it derives position from signal measurements to external infrastructure, not from a first-principles ranging signal on the device. C is correct: a hardcoded GPS coordinate is a form of direct (manual) position annotation — valid for fixed deployed devices, though it breaks if the device is moved. D is correct: UWB ranging to fixed anchors is a direct ranging technology — the tag actively measures round-trip time to known anchors and computes position. E is wrong: cell-ID is indirect — it looks up the cell tower's known location in a database, not the device's own measurement.

MA2

• A) OWL adds formal description-logic semantics that allow a reasoner to infer new triples not explicitly stated

• B) OWL supports `owl:equivalentClass` to declare that two classes from different ontologies represent the same concept

• C) OWL replaces Turtle as the preferred RDF serialisation format

• D) OWL supports cardinality restrictions, e.g. declaring that a room has exactly one temperature sensor

• E) OWL queries are written in SPARQL; without OWL a SPARQL endpoint cannot function

MA3. Which of the following are consequences of a sensor fleet generating more data than the system was designed to handle (the "Big Data consequences" problem)?

• A) Network congestion — uplinks saturate the available bandwidth, causing queuing delays and packet loss

• B) Storage exhaustion — raw data accumulates faster than retention policies can expire it

• C) Query performance degradation — time-range queries over billions of rows slow to unusable speeds without partitioning or downsampling

• D) Automatic improvement in data quality, because more data averages out noise

• E) Increased operational cost for ingestion pipelines, storage, and cloud compute

MA4. Which of the following are valid timestamping points along the path from a sensor event to a stored database record, ordered from most to least accurate?

• A) The moment the physical event occurs at the sensor (true event time)

• B) The sensor's local clock at the time of measurement

• C) The LoRaWAN gateway's timestamp when the packet is received

• D) The application server's clock at the time the record is written to the database

• E) The database's `NOW()` function called during the INSERT

A is correct: OWL adds description-logic semantics. An OWL reasoner can infer, for example, that if every instance of class A is also an instance of class B, and x is an instance of A, then x is an instance of B — even if that triple is not explicitly stated. B is correct: `owl:equivalentClass` and `owl:sameAs` allow merging data from different ontologies that use different URIs for the same concept. C is wrong: OWL is expressed in Turtle, RDF/XML, or JSON-LD like any RDF — it is not a serialisation format. D is correct: OWL supports `owl:cardinality`, `owl:minCardinality`, `owl:maxCardinality` restrictions on properties. E is wrong: SPARQL works on any RDF graph regardless of whether OWL is used.

MA3

A is correct: 10,000 sensors at 1 Hz each produce 10,000 packets/s; a single MQTT broker or database ingestion pipeline may saturate, causing queuing and loss. B is correct: at 1 reading/s per sensor with a 4-byte float + 8-byte timestamp = ~12 bytes, 10,000 sensors generate ~1 TB/year — without a retention policy the storage grows unboundedly. C is correct: a naive SELECT over billions of unpartitioned rows can take minutes; without time-partitioning or continuous aggregates, dashboards become unusable. D is wrong: more data does not automatically improve quality — noisy data at higher volume is still noisy, and storing it increases cost without benefit. E is correct: cloud ingestion pipelines, storage, and query compute all scale linearly with data volume and incur direct cost.

MA4

All five are valid timestamping points, but accuracy differs. A (true event time) is the ground truth but is only approximated by B (sensor clock), which is itself only as good as the sensor's last time synchronisation. C (gateway receipt) adds network propagation delay (~ms for LoRa). D (application server write time) adds further processing delay (ms to seconds). E (`NOW()` in the database INSERT) is effectively the same as D but may add additional queuing delay. For exam purposes A–D are the four clearly ordered timestamping points; E is nearly identical to D.

Section 3 — Short Answer

SA1. What are the three IoT data annotation axes (Identity, Time, Space)? For each, give one example of what goes wrong when it is missing.

Identity answers "which entity produced this reading." Without it, readings from 200 sensors are indistinguishable — you cannot associate a value with a room or device. Time answers "when did this event occur." Without it, readings cannot be ordered, correlated across devices, or compared to historical trends — a fire alarm without a timestamp cannot be used for post-incident analysis. Space answers "where was the sensor." Without it, you cannot map readings to physical locations — a temperature value without a room identifier is useless for HVAC control or for tracing a cold-chain excursion to a specific pallet.

SA2. What is SHACL, and how does it complement SPARQL in an RDF-based IoT metadata system?

SHACL (Shapes Constraint Language, W3C standard) lets you define validation rules — called shapes — that RDF nodes must satisfy: required properties, cardinality limits, datatype constraints, value ranges. A SHACL validator compares a data graph against a shapes graph and reports violations. In an IoT metadata system it complements SPARQL: SPARQL retrieves data; SHACL ensures that data conforms to the expected schema before it is used. For example, a SHACL shape might require every `sosa:Observation` to have exactly one `sosa:resultTime` with an `xsd:dateTime` value — catching sensors that publish readings without timestamps before they corrupt downstream queries.

SA3. What is the difference between OWL and plain RDF/RDFS? Give one concrete example of something you can express in OWL that you cannot in plain RDF.

Plain RDF lets you state triples. RDFS adds `rdfs:subClassOf` and `rdfs:subPropertyOf` for basic taxonomies. OWL adds description-logic semantics that a reasoner can exploit to infer new facts. Example: in OWL you can declare `owl:inverseOf` between two properties — e.g., `hasLocation` and `isLocationOf`. If you assert `sensor:42 hasLocation room:U181`, the OWL reasoner automatically infers `room:U181 isLocationOf sensor:42` without you explicitly stating it. This is impossible in plain RDF/RDFS because there is no mechanism for inverting properties — you would have to add the inverse triple manually.

SA4. What is absolute humidity, how is it calculated from relative humidity and temperature, and why does an IoT pipeline that only stores relative humidity lose information?

Relative humidity (RH) is the ratio of current water vapour pressure to saturation vapour pressure at a given temperature, expressed as a percentage. Absolute humidity is the actual mass of water per unit volume of air (g/m^3), which depends on both RH and temperature. Approximation: using the Magnus formula, saturation vapour pressure $e_s(T) \approx 6.1078 \times 10^{(7.5T/(237.3+T))}$ hPa; actual vapour pressure $e = \text{RH}/100 \times e_s$; absolute humidity $\text{AH} \approx 216.7 \times e / (T + 273.15)$ g/m^3 . If you store only RH, two readings of RH=70% taken at 10 °C and 30 °C represent completely different actual moisture levels ($\approx 6 \text{ g}/\text{m}^3$ vs $\approx 21 \text{ g}/\text{m}^3$). A pipeline correlating humidity from sensors in different thermal environments will draw wrong conclusions unless absolute humidity is computed and stored.

SA5. What is the Big Data "3 Vs" model, and give a concrete example of each V for a 10,000-sensor building management deployment?

Volume: the sheer amount of data. Example — 10,000 sensors at 1 Hz each, 12 bytes per reading, generates ~ 3.8 TB/year. Without a retention/aggregation strategy, storage and query costs grow unboundedly. Velocity: the speed at which data arrives and must be processed. Example — the same fleet produces 10,000 inserts/second into a time-series database; a naive relational schema cannot sustain this; a specialised ingestion pipeline (Kafka + InfluxDB) is needed. Variety: the diversity of data types and sources. Example — the same building system produces numeric sensor readings, binary alarm events, image snapshots from cameras, and unstructured maintenance logs — each requiring different storage formats, schemas, and processing pipelines.

SA6. What is an Elixir struct, and what are two advantages it has over a plain map for modelling a domain entity like an IoT sensor reading?

A struct is defined with `defstruct` inside a module and behaves like a map with a fixed set of keys and default values. Two advantages over plain maps: (1) Pattern matching by module name — `%SensorReading{value: v} = x` fails at runtime if `x` is a plain map or a different struct, making code self-documenting and catching type errors early. (2) Compile-time key validation — `%SensorReading{valeu: 21.5}` (typo) raises a compile error, whereas a plain map `%{valeu: 21.5}` silently creates a map with the wrong key. Structs also serve as documentation of the expected shape of data flowing through the system.

SA7. What is threshold hysteresis, and why is a single fixed threshold insufficient for triggering alerts on noisy sensor data?

A single fixed threshold (e.g., alert when temperature $> 25^{\circ}\text{C}$, clear when $\leq 25^{\circ}\text{C}$) causes rapid alert/clear oscillation when the signal hovers near the threshold due to measurement noise — a phenomenon called "chattering." Hysteresis solves this with two thresholds: a higher one to trigger the alert and a lower one to clear it (e.g., alert at $> 25^{\circ}\text{C}$, clear at $< 23^{\circ}\text{C}$). Once the alert fires, the signal must drop 2°C below the trigger before the alert clears, creating a deadband that absorbs noise. Without hysteresis, a dashboard shows dozens of spurious alert/clear transitions per minute, operators stop trusting alerts, and notification systems are flooded.

SA8. What is WiFi WPA2 (802.11i), and what two cryptographic mechanisms does it use to provide authentication and data confidentiality?

WPA2 (IEEE 802.11i) is the WiFi security standard that replaced the broken WEP. It uses two main cryptographic mechanisms: (1) **4-way handshake with CCMP/AES** — after the initial 802.1X or PSK authentication, a 4-way handshake derives a per-session Pairwise Transient Key (PTK) for data encryption using AES-CCMP (Counter Mode CBC-MAC Protocol), providing confidentiality, integrity, and replay protection for all data frames. (2) **Authentication:** in Personal mode (WPA2-PSK) the pre-shared passphrase is used to derive a Pairwise Master Key (PMK); in Enterprise mode (WPA2-Enterprise) 802.1X with EAP authenticates users against a RADIUS server, providing per-user credentials and forward secrecy. Every unicast data frame is encrypted with a unique session key derived from the PTK.

SA9. In the context of position annotation, what is cell-ID localisation, and for which IoT use cases is its 100–2000 m accuracy acceptable?

Cell-ID localisation identifies which cellular base station (cell tower) a device is associated with and looks up that tower's known position (from a public or proprietary database) as the device's inferred location. Accuracy is typically 100–2000 m depending on cell density. Acceptable use cases: fleet tracking for logistics (knowing which city or district a truck is in), supply chain monitoring (which country a container is in), coarse presence detection (is this mobile asset on-site or off-site), and as a fallback when GPS is unavailable. Unacceptable: room-level indoor localisation, navigation, any application where 100 m error would cause a wrong decision.

SA10. What are the three main identity assignment points in an IoT system (at the node, at the gateway/fog, at the cloud), and what is the advantage of assigning identity as early as possible?

The three main points are: (1) At the node — the device stamps its own identity (MAC, UUID, serial) onto every message before transmission. (2) At the gateway/fog — the gateway infers identity from which radio channel, network address, or join credentials a message arrived on. (3) At the cloud — the backend assigns identity by correlating arrival metadata (gateway ID, time, signal strength) with a device registry. Assigning identity as early as possible (at the node) is advantageous because: it is unambiguous even if the message is relayed through multiple gateways; it survives network reconfigurations without re-mapping; and buffered messages retain their identity during outages, so no data is anonymised by a gap in the gateway-to-cloud mapping.

DA1. Explain the full identity/time/space annotation pipeline for the following system: 200 soil moisture sensors deployed across a 10 km² agricultural field, reporting raw ADC values over LoRaWAN every 15 minutes to a cloud dashboard. For each annotation axis: where is the annotation assigned, what is the source, what failure modes produce incorrect annotations, and how would you detect and recover from them?

Identity axis: Each sensor node should have a UUID burned in at manufacture or assigned during physical installation via a mobile app scan of a QR code on the housing. The UUID is included in every LoRaWAN uplink payload. The cloud backend maintains a registry mapping UUID → (field ID, deployment location, sensor type). Assignment is at the node; source is the UUID chip or provisioning app. Failure mode: two sensors swapped during maintenance now report the wrong field position — the UUID still uniquely identifies the hardware, but the registry record links it to the wrong location. Detection: if a sensor's reported moisture pattern suddenly diverges from its neighbours in an implausibly large spatial gradient, flag it for inspection. Recovery: update the registry record after confirming the physical swap.

Time axis: Each LoRaWAN gateway timestamps packet arrival using its own clock, synchronised to NTP. The sensor itself has no reliable real-time clock — it wakes from sleep and transmits without knowing absolute time. Therefore the gateway receipt timestamp is used as the measurement time, with the understanding it is systematically late by the LoRa ToA (~1–2 s) plus any gateway processing delay (~ms). Failure mode: the gateway's NTP synchronisation lapses during a network outage; its local RTC drifts 1–10 ppm, accumulating up to ~0.86 s/day error. Detection: monitor the NTP sync status; alert if clock drift exceeds 1 minute. Recovery: once connectivity returns, NTP re-synchronises within seconds; historical records can be corrected by applying the known drift rate to the affected window if the drift start time is logged.

Space axis: Sensor location (GPS coordinates of the deployment point) is assigned at installation time by the technician using a GPS survey app or by reading coordinates from a map. This is stored in the device registry as a static annotation — soil sensors do not move. Source: GPS survey at deployment. Failure mode: a sensor is physically relocated (e.g., dug up and replanted in a different field section) without updating the registry — all subsequent readings are mapped to the wrong grid cell on the dashboard. Detection: spatial anomaly detection — if sensor readings are physically inconsistent with adjacent sensors (e.g., one sensor in a dry grid cell suddenly matches a distant irrigated cell), flag it. Recovery: on-site confirmation and registry update.

DA2. A temperature and humidity sensor (e.g. DHT22) produces relative humidity (RH%) and temperature (°C). Explain why you should compute and store absolute humidity at the point of ingestion rather than storing only RH%. Cover: the physical meaning of RH vs absolute humidity, the Magnus formula or approximation, a concrete scenario where storing only RH leads to a wrong conclusion, and what this implies about data transformation placement in the IoT pipeline.

Relative humidity (RH%) expresses how close the air is to saturation at the current temperature, as a percentage. It does not directly express the mass of water in the air. The saturation vapour pressure $e_s(T)$ follows an approximately exponential relationship with temperature (Magnus formula: $e_s \approx 6.1078 \times 10^{(7.5T/(237.3+T))}$ hPa). At $T=20^\circ\text{C}$ and $\text{RH}=60\%$, $e_s \approx 23.4$ hPa, so $e = 0.6 \times 23.4 = 14$ hPa, giving absolute humidity $\approx 216.7 \times 14 / 293 \approx 10.4$ g/m³. At $T=30^\circ\text{C}$ and $\text{RH}=60\%$, $e_s \approx 42.4$ hPa, giving $\text{AH} \approx 19.0$ g/m³ — 83% more water despite the same RH.

Concrete wrong conclusion: a cold-chain system stores only RH. A sensor in an uninsulated truck reads $\text{RH}=55\%$ at 5°C ($\text{AH} \approx 3.5$ g/m³) and another in a warm warehouse reads $\text{RH}=55\%$ at 25°C ($\text{AH} \approx 12.8$ g/m³). A dashboard comparing the two by RH shows identical values — no apparent difference. But the pharmaceutical product in the warehouse is exposed to 3.7× more moisture, potentially causing degradation. A system that stored absolute humidity would immediately flag the discrepancy.

	Pipeline placement: the conversion should happen as close to the sensor as possible — ideally on the fog gateway or at ingestion into the time-series database, not in the dashboard query. This ensures that all downstream consumers (alerting, ML models, reports) work on the physically meaningful quantity. Converting in the dashboard is fragile: different dashboards may use different formulas, and historical queries that join RH data from different temperature environments will silently produce wrong results. The rule is: derive the canonical physical quantity at ingestion and store it; never store a context-dependent proxy (RH) as if it were the final value.
DA3. Explain how you would design the WiFi connectivity layer for an ESP32 device that must connect to a WPA2 access point, send sensor readings over a TCP socket, and recover from network failures. Cover: the ESP-IDF event model (which events fire and in what order), how you structure the reconnect logic, where you store credentials (NVS vs Kconfig), and what happens to buffered data if the TCP connection drops mid-transmission.	ESP-IDF models WiFi and network events through an event loop. The sequence on successful connect: (1) Application calls <code>esp_wifi_start()</code> → <code>WIFI_EVENT_STA_START</code> fires → application calls <code>esp_wifi_connect()</code>. (2) After 802.11 association: <code>WIFI_EVENT_STA_CONNECTED</code> fires — the link is up but no IP yet. (3) DHCP completes: <code>IP_EVENT_STA_GOT_IP</code> fires with the assigned address. Only after this event is it safe to open sockets. On disconnect: <code>WIFI_EVENT_STA_DISCONNECTED</code> fires with a reason code. Reconnect logic: register a handler for <code>WIFI_EVENT_STA_DISCONNECTED</code> that calls <code>esp_wifi_connect()</code> again, with a retry counter and exponential backoff (e.g., 1 s, 2 s, 4 s, up to 60 s) to avoid hammering a temporarily unavailable AP. Reset the counter on <code>IP_EVENT_STA_GOT_IP</code>. If retries are exhausted, enter deep sleep for a longer interval or raise a fault indicator. Credential storage: store WiFi SSID and password in NVS rather than Kconfig <code>#define</code> values. Kconfig values are baked into the binary — changing credentials requires a reflash. NVS allows runtime provisioning (via BLE or serial) and survives OTA updates since NVS lives in its own flash partition. TCP connection and mid-transmission drop: open a TCP socket after <code>IP_EVENT_STA_GOT_IP</code>. On <code>send()</code> failure (<code>errno ECONNRESET</code>, <code>EPIPE</code>, or <code>ENOTCONN</code>), close the socket, buffer unsent data in a FreeRTOS queue or ring buffer in RAM (or in NVS for durability across reboots), and attempt reconnect. After reconnecting WiFi and re-establishing the TCP connection, drain the buffer. For durability across power loss, periodically flush the ring buffer to NVS; on boot check whether NVS holds unsent records before opening the TCP connection.

Section 5 — Design Question

Question 21 — Indoor Asset Tracking

(answer not found)

A hospital wants to track the real-time location of **200 pieces of shared equipment** (infusion pumps, portable ECG monitors, wheelchairs) across a single building with **6 floors**, each floor roughly 60 m × 40 m. Requirements:

- Locate any item to the **correct room** (typical room ~5 m × 4 m) within **10 seconds** of a staff query.
- Each tag must run for at least **12 months** on a coin cell.
- The system must work indoors where GPS is unavailable.
- Infrastructure changes to the building must be minimal.

Section 3 — Short Answer

Subquestion 21.1 (short answer)

Which radio positioning technology would you use, and why? Compare at least two alternatives.

BLE (Bluetooth Low Energy) with RSSI-based localisation is the best choice. BLE infrastructure (access points / scanners) can be installed on each floor without cabling changes — they connect to existing Ethernet ports. BLE tags are tiny, run for 12+ months on a CR2032 at a 1-second advertising interval (average current ~5 μ A), and are standardised commodity hardware. Room-level accuracy (~3–5 m) is achievable with RSSI fingerprinting against a pre-built radio map.

Alternative considered — UWB: UWB (e.g., Decawave DW1000) provides 10–30 cm accuracy via time-of-flight ranging, which exceeds the room-level requirement. However, UWB anchors require precise timing synchronisation hardware, are more expensive, and consume more power than BLE, threatening the 12-month coin-cell budget. Overkill for room-level tracking.

Alternative rejected — WiFi RSSI: existing WiFi APs could be used, but commodity WiFi tags draw 50–200 mA during active scans — a CR2032 would last days, not months. Power-saving WiFi (802.11 TWT) helps but is not universally supported on small tags.

Subquestion 21.2 (short answer)

How do you assign and maintain the identity annotation for each piece of equipment? What happens when a pump is serviced and its tag is replaced?

Each piece of equipment is assigned a permanent logical ID (UUID) in the asset registry at procurement — linked to the equipment's serial number, type, and purchase record. A BLE tag is provisioned with this UUID and physically attached to the equipment (e.g., cable-tied). The tag broadcasts this UUID in its BLE advertising payload.

When a tag is replaced after servicing: the technician scans the new tag's hardware ID (printed as a QR code on the tag) using a mobile app, selects the equipment from the registry, and re-links the new tag's hardware ID to the existing logical UUID. All historical location records remain associated with the logical UUID; only the physical tag mapping is updated. The registry stores a history of tag-to-equipment associations so audit trails remain intact. The dashboard and location engine continue to use the logical UUID — they are unaware of the physical tag change.

Section 4 — Detailed Answer

Subquestion 21.3 (detailed answer)

Design the full location pipeline: from tag radio signal to room-level position on a dashboard. Cover the localisation algorithm (RSSI fingerprinting, trilateration, or other), where the computation runs, how position confidence degrades when infrastructure nodes fail, and how you validate that the system meets the 10-second and room-level accuracy requirements.

Infrastructure: Install one BLE scanner (a small Linux SBC or dedicated BLE sniffer) per 2–3 rooms on each floor, connected to the existing Ethernet backbone — roughly 20–30 scanners per floor, 120–180 total. Each scanner continuously records all advertising packets it hears: tag UUID, RSSI, timestamp.

Localisation algorithm — RSSI fingerprinting: During a one-time calibration walk, a technician carries each tag (or a reference tag) through every room and records the RSSI vector (signal strength at each nearby scanner) at known positions. This builds a radio map: a dictionary mapping room ID → expected RSSI vector. At runtime, the location engine receives the current RSSI vector for a tag (assembled from reports across all scanners that heard it in the last 5 seconds) and finds the nearest match in the radio map using a distance metric (e.g., Euclidean distance in RSSI space or a k-NN classifier). The result is the most likely room. Because BLE advertising at 1 Hz gives a fresh RSSI sample every second and the query buffers 5 seconds of data, position updates are available every 1–5 seconds — well within the 10-second SLA.

Position confidence degradation: If one scanner goes offline, the RSSI vector for tags near that scanner has missing dimensions. The matching algorithm falls back to the remaining scanners; accuracy degrades from room-level to 2–3 room uncertainty for equipment near the failed scanner. The system flags the affected scanner on the dashboard. If two or more adjacent scanners fail simultaneously, localisation for that zone is suspended and the last known position is reported with a "stale" indicator.

Computation placement: RSSI aggregation runs on each floor's fog server (one per floor); room-level classification runs on a central building server or small cloud instance. The fog server caches the radio map locally so localisation continues during internet outages; only dashboard updates and alert forwarding require cloud connectivity.

Validation: Test the 10-second SLA by scripted walks with a reference tag; record time from tag entering a room to correct room appearing on the dashboard. Test room-level accuracy by visiting each of the N rooms with a reference tag and measuring the confusion matrix — what fraction of positions are reported as the correct room. A target of ≥90% room-level accuracy across all rooms is a reasonable acceptance criterion. Re-run calibration after any building renovation or furniture rearrangement, as these change the radio map.

Practice Exam 8

Question

Model Answer

Section 1 — Multiple Choice

MC1. On a microcontroller, the Von Neumann memory map typically places code, heap, stack, and memory-mapped I/O at different address ranges. Which of the following correctly describes the typical growth direction of the stack and heap?

- A) Both the stack and heap grow upward toward higher addresses
- B) The heap grows upward and the stack grows downward, so they grow toward each other
- C) The stack grows upward and the heap grows downward

B

In the standard MCU memory layout, the heap occupies low-to-mid addresses and grows upward as `malloc` allocates more memory. The stack starts at the top of RAM and grows downward as functions are called and local variables are pushed. They grow toward each other — if they meet, you get a stack overflow or heap corruption. A, C, D are wrong.

• D) Both grow downward; code occupies the high-address region	
MC2. In the C compilation pipeline, what is the primary role of the linker? • A) Expanding <code>#include</code> directives and macro definitions into a single translation unit • B) Translating C source text into architecture-specific assembly instructions • C) Combining multiple compiled object files and resolving symbol references (e.g. function calls between files) into a single executable • D) Converting assembly mnemonics into binary machine code object files	C The linker takes the compiled object files (<code>.o</code>) produced by the assembler and resolves cross-file symbol references — e.g., a <code>main.c</code> object file calling <code>hl_wifi_init</code> defined in <code>hl_wifi.o</code>. It also links against libraries (<code>libc</code>, ESP-IDF components) and produces the final <code>.elf.bin</code> image. A describes the preprocessor; B describes the compiler; D describes the assembler.
MC3. In a UART RS232 frame, what is the purpose of the start bit? • A) It carries the parity information for error detection • B) It is a low (0) pulse that transitions the idle-high line, telling the receiver to start its bit-clock and begin sampling the data bits • C) It marks the boundary between two successive bytes when sending a multi-byte message • D) It signals that the transmitter is ready to accept new data from the host MC4. The inverse square law states that received signal power is proportional to $1/d^2$. If a LoRa sensor moves from 100 m to 400 m from the gateway, by what factor does the received power decrease? • A) $4\times$ • B) $8\times$ • C) $16\times$ • D) $2\times$	B UART is asynchronous — there is no shared clock. The line idles high (logic 1). A start bit is a low (0) pulse that alerts the receiver to start its internal bit-clock and begin sampling. The receiver samples at the agreed baud rate, reading each data bit in turn. A stop bit (high) returns the line to idle, ready for the next frame. A, C, D are wrong. C Distance increases by $4\times$ (100 m $\rightarrow$ 400 m). Power $\propto 1/d^2$, so the power ratio is $(100/400)^2 = (1/4)^2 = 1/16$. Received power drops by $16\times$. In dB: $10 \log_{10}(1/16) \approx -12$ dB. This illustrates why doubling range costs ~ 6 dB — you need to quadruple transmit power or accept a 6 dB lower link budget. A would be a $4\times$ distance with linear law; B has no basis; D is only $2\times$.
MC5. RPL (IPv6 Routing Protocol for Low-Power and Lossy Networks) builds a routing structure called a DODAG. What does DODAG stand for, and what is at its root? • A) Distributed Optimised Directed Acyclic Graph; rooted at the device with the lowest battery level • B) Destination-Oriented Directed Acyclic Graph; rooted at a border router that connects the mesh to the wider IP network • C) Dynamic On-demand Directed Acyclic Graph; rooted at whichever node initiates the first broadcast • D) Dual-path Optimised Directed Acyclic Graph; rooted at two redundant gateways for fault tolerance	B DODAG = Destination-Oriented Directed Acyclic Graph. It is a tree-like structure rooted at a border router (the DODAG root) that connects the constrained mesh to the wider IPv6 internet. Nodes select an upward parent using an Objective Function — the default (OF0) uses ETX (Expected Transmissions, a link quality metric). All traffic flows "upward" toward the root for uplink, and downward for downlink commands. A, C, D are wrong.

MC6. What do vector clocks track in a distributed IoT system? • A) The absolute UTC time on each node, synchronised via GPS • B) The causal ordering of events across nodes — whether one event happened-before another — without requiring a global clock • C) The cumulative energy expenditure of each node since last reset • D) The number of packets transmitted by each node, used for congestion control	B Vector clocks are a distributed systems technique for tracking causal ordering of events without a global wall clock. Each node maintains a vector of logical counters, one per node. By comparing two events' vectors you can determine whether A happened-before B, B happened-before A, or they are concurrent (neither dominates). A, C, D are wrong.
MC7. In performance evaluation, what is the distinction between a factor and a parameter? • A) A factor is a metric you measure; a parameter is a metric you calculate • B) A factor is an independent variable you deliberately vary to observe its effect on a metric; a parameter is a configuration value held constant across all test runs • C) A parameter is changed between experiments; a factor remains fixed • D) They are synonyms — both refer to variables that affect system behaviour	B A factor is a variable you deliberately change across experiments to observe its effect — e.g., varying the MQTT QoS level from 0 to 1 to 2. A parameter is a value that is fixed for a given experiment set but may differ between experiments as context — e.g., a fixed 10 ms publish interval used consistently within one test run. A confuses factors with metrics; C swaps the definitions; D is wrong.
MC8. What is ESP-NOW, and how does it differ from standard WiFi communication? • A) ESP-NOW is Espressif's LoRa library for ESP32; it differs from WiFi by using sub-GHz frequencies • B) ESP-NOW is a connectionless peer-to-peer protocol that allows ESP32 devices to exchange short data packets directly without associating to a WiFi access point or router • C) ESP-NOW requires an MQTT broker to route messages between ESP32 devices • D) ESP-NOW is the ESP-IDF name for Bluetooth Classic SPP communication	B ESP-NOW is an Espressif proprietary protocol that allows ESP32 (and ESP8266) devices to send 250-byte payloads directly to each other's MAC addresses using the 2.4 GHz WiFi radio without joining a network or using an AP. It has ~1 ms latency and works with devices in both station and sleep modes. Useful for direct device-to-device control in mesh scenarios. A, C, D are wrong.

Section 2 — Multiple Answer

MA1. Which stages are part of the standard C compilation pipeline, in order? • A) Preprocessing — expanding <code>#include</code>, <code>#define</code>, and conditional compilation directives • B) Compilation — translating preprocessed C source to assembly • C) Assembly — converting assembly mnemonics to binary object code (<code>.o</code> files)	MA1 A is correct: the preprocessor (cpp) handles <code>#include</code>, <code>#define</code>, <code>#ifdef</code> guards and produces a single expanded <code>.i</code> translation unit. B is correct: the compiler (cc1) translates C source to assembly (<code>.s</code> file). C is correct: the assembler (as) converts assembly mnemonics to binary machine code in an object file (<code>.o</code>). D is correct: the linker (ld) combines object files and static libraries, resolves all symbol references, applies the linker script (which maps sections to memory addresses), and produces the final <code>.elf</code> or <code>.bin</code>. E is wrong: C is compiled to native machine code, not bytecode; there is no VM at runtime.
---	---

• D) Linking — combining object files, resolving symbols, producing the final executable or <code>.bin</code> • E) Interpretation — running the object code through a bytecode VM at startup	
MA2. Which of the following are among the four IoT budget domains described in the course? • A) Radio budget — constraints on packet count, payload size, and duty cycle • B) Memory budget — RAM, flash size, write endurance • C) Processing budget — clock speed, FPU availability, DMA channels • D) Energy budget — battery capacity, average current draw, sleep/wake cycles • E) Security budget — encryption key length and rotation frequency	MA2 A, B, C, and D are the four domains as defined in the course. Radio (A) constrains packet rate, payload MTU, and duty cycle. Memory (B) constrains what software can run and how much data can be buffered locally. Processing (C) constrains how much computation can be performed at the edge. Energy (D) is the overarching budget that all other domains drain from. E (security budget) is not one of the four defined course domains, though security does consume processing and radio resources.
MA3. Which of the following are valid mitigations for the counting-to-infinity problem in distance vector routing? • A) Split horizon — do not advertise a route back to the neighbour you learned it from • B) Poison reverse — advertise failed routes with cost infinity • C) Hold-down timer — ignore route updates for a period after a failure is detected • D) Running Dijkstra's algorithm on the full topology graph • E) Increasing the maximum hop count to prevent the count ever reaching infinity	MA3 A is correct: split horizon prevents a node from advertising a route back through the neighbour it learned it from — the most common mitigation. B is correct: poison reverse actively advertises a failed route with cost ∞, telling neighbours to remove it immediately rather than waiting for a timeout. C is correct: a hold-down timer suppresses route updates after a failure, giving the network time to converge without accepting stale alternates. D is wrong: Dijkstra (link state) avoids counting-to-infinity entirely by having full topology knowledge, but it is not a mitigation applied to distance vector — it is a different algorithm. E is wrong: increasing the maximum hop count (e.g., from 15 to 255 in RIP) merely delays the problem, it does not eliminate it.
MA4. Which of the following are valid IoT network topologies, and what is a characteristic use case for each? • A) Star — all devices connect directly to a single central hub; simple, low-latency, single point of failure • B) Mesh — devices route for each other; resilient, extends range, higher protocol complexity • C) Tree — hierarchical star-of-stars; balances range extension with moderate complexity	MA4 A is correct: star topology (all devices connect to a central hub/gateway) is the most common IoT layout — simple to implement, low end-device complexity, but the hub is a single point of failure. B is correct: mesh topology (devices route for each other) is used in Zigbee, Thread, and BLE Mesh — extends range and adds resilience at the cost of routing overhead. C is correct: tree (hierarchical star) is used in LoRaWAN with multiple gateways and cluster heads — scales to large deployments. D is correct: bus topology is used in wired industrial systems (CAN bus in vehicles, Modbus RS-485 in industrial automation). E is wrong: ring topology is historically used in token ring and SONET fibre networks but is not common in modern wireless IoT because a single node failure breaks the ring.

- D) Bus — all devices share a single communication line; common in industrial wired fieldbus (e.g. CAN)
- E) Ring — each device connects to exactly two neighbours forming a loop; common in modern wireless IoT

Section 3 — Short Answer

SA1. Describe the Von Neumann memory map on a typical MCU. What are the four main address regions, and what does each hold?

A typical MCU address space has four main regions: (1) Memory-mapped I/O at the lowest addresses — special addresses wired directly to peripheral registers (GPIO, ADC, timers); writing to them controls hardware. (2) Code / text segment — read-only flash where compiled machine instructions are stored; on most Cortex-M MCUs this starts at 0x08000000. (3) Heap — starts after the static data sections in RAM and grows upward; used by `malloc`. (4) Stack — starts at the top of RAM and grows downward; holds function call frames, local variables, and return addresses. On most MCUs, RAM is 256 KB–4 MB and flash is 512 KB–16 MB.

SA2. What are the four stages of the C compilation pipeline? What is the input and output of each stage?

Four stages: (1) Preprocessing (input: `.c` source; output: `.i` expanded C) — the preprocessor (`cpp`) expands `#include` files, substitutes `#define` macros, and processes conditional compilation (`#ifdef`). (2) Compilation (input: `.i`; output: `.s` assembly) — the compiler translates C constructs to architecture-specific assembly instructions, applying optimisations. (3) Assembly (input: `.s`; output: `.o` object file) — the assembler converts assembly mnemonics to binary machine code; symbols remain unresolved. (4) Linking (input: multiple `.o` files and libraries; output: `.elf/.bin` executable) — the linker resolves cross-file function and variable references, applies the memory layout from the linker script, and produces the final flashable image.

SA3. Describe the structure of a UART RS232 frame. What are start bits, data bits, parity, and stop bits, and why does UART not need a separate clock wire?

UART is asynchronous — sender and receiver agree on baud rate in advance; no clock wire is needed. The line idles at logic high (1). A frame starts with a start bit: a mandatory low (0) pulse that triggers the receiver's internal bit-sampling clock. Then come 7 or 8 data bits (LSB first). An optional parity bit follows for error detection. Finally one or two stop bits (high, 1) return the line to idle, ensuring the receiver can detect the next start bit. The receiver samples each bit at the midpoint of its expected window using its local clock at the agreed baud rate — small clock drift accumulates over the frame but is reset by the next start bit.

SA4. What is the inverse square law for radio propagation, and what does it imply about the cost of extending range by increasing transmit power?

Received signal power from an omnidirectional antenna falls as $P_r \propto 1/d^2$. Every doubling of distance reduces power by 4× (6 dB). To compensate for doubling range with transmit power alone, you must quadruple transmit power (+6 dBm) — an expensive and often legally constrained option. In practice this means range extension is better achieved by: improving receiver sensitivity (lower noise floor), increasing antenna gain (directionality), raising the receiver above obstacles, or — for digital links — accepting a slower data rate with more robust modulation (e.g. higher SF in LoRa), which improves link budget without increasing TX power.

SA5. What is the OSI model, and which three layers are most relevant to IoT protocol design? Give one IoT example for each.

The OSI model partitions communication into 7 layers: Physical, Data Link, Network, Transport, Session, Presentation, Application. Three are most relevant to IoT: Physical layer (Layer 1) — defines the radio modulation, frequency, and bit encoding. IoT example: LoRa CSS modulation at 868 MHz. Data Link layer (Layer 2) — defines MAC addressing, channel access (CSMA/CA), and frame format. IoT example: IEEE 802.15.4 MAC used by Zigbee and Thread. Network layer (Layer 3) — routing and addressing. IoT example: IPv6 with 6LoWPAN header compression over 802.15.4, or RPL routing within a Thread mesh. Application layer (Layer 7) — end-to-end protocol. IoT example: MQTT, CoAP, HTTP.

SA6. What is RPL, and what is the DODAG structure it builds? What metric does the default objective function use to select parent nodes?	RPL (RFC 6550) is the IETF standard routing protocol for IPv6 over Low-Power and Lossy Networks (LLNs). It builds a Destination-Oriented Directed Acyclic Graph (DODAG): a directed tree rooted at a border router, with all edges pointing "upward" toward the root. Nodes select their parent by evaluating an Objective Function. The default OF0 uses ETX (Expected Transmissions) — the average number of transmissions needed to deliver a packet to a neighbour including retransmissions; lower ETX means a better link. Nodes pick the parent with the lowest path ETX to the root. This creates a routing structure that naturally avoids poor links and adapts as link quality changes.
SA7. What is the difference between a vector clock and a Lamport timestamp? When would you need a vector clock rather than a Lamport timestamp in an IoT system?	A Lamport timestamp is a scalar counter. On send, a node increments its counter and attaches it. On receive, the receiver takes the max of its counter and the received value, then increments. Lamport timestamps preserve happens-before: if A happened-before B, then $L(A) < L(B)$. However, $L(A) < L(B)$ does not imply A happened-before B — you cannot detect concurrent events. A vector clock is a vector of counters, one per node. It fully captures causality: you can determine whether $A \rightarrow B$, $B \rightarrow A$, or $A \parallel B$ (concurrent). In an IoT system, vector clocks are needed when you must detect and resolve conflicting writes — e.g., two edge nodes both update a device state record while disconnected from each other; on reconnect you need to know which update is the cause of the other, or whether they are independent conflicts requiring merge logic.
SA8. What is the difference between a synthetic workload and a replayed workload in a performance evaluation experiment? When is each appropriate?	A synthetic workload is artificially generated to match a specified statistical profile — e.g., a script that publishes 1000 MQTT messages/second with random 20-byte payloads. Use it when you need controlled, repeatable tests that isolate specific parameters (message rate, payload size) without needing real production data. A replayed workload uses a recorded trace from a real deployment — e.g., a 24-hour log of actual sensor publishes replayed at 10× speed. Use it when you need to capture realistic burst patterns, diurnal variation, and correlated failures that a synthetic generator would miss. Replayed workloads are harder to parameterise but produce more realistic results for validating a system against real operating conditions.
SA9. What are the four IoT budget domains? For each, give one concrete constraint that forces a design trade-off.	(1) Radio budget: maximum packets per day (e.g., LoRaWAN 140 uplinks/day at SF12), maximum payload size (51 bytes at SF12), and duty-cycle limits (1% EU). Constraint: a sensor needing 10 readings/min cannot use LoRaWAN at SF12. (2) Memory budget: limited RAM (e.g., 520 KB on ESP32) and flash (4 MB). Constraint: a moving-average buffer of 1000 float32 values costs 4 KB RAM — feasible, but a 100,000-sample history buffer does not fit. (3) Processing budget: limited clock speed (240 MHz on ESP32, but no double-precision FPU on Cortex-M0). Constraint: running FFT on 200 Hz vibration data at the edge consumes significant CPU cycles, potentially conflicting with radio and sleep scheduling. (4) Energy budget: fixed battery capacity (e.g., 3000 mAh × 2 AAs). Constraint: a device drawing 1 mA average lasts ~125 days — insufficient for a 1-year deployment without solar.

SA10. What is a C function pointer, and give a concrete IoT example where using a function pointer improves code structure compared to a chain of if/else or switch statements?

A function pointer is a variable that holds the address of a function, allowing the function to be called indirectly. Syntax: `void (*handler)(uint8_t* data, size_t len)`. IoT example: a serial parser that receives frames with a 1-byte type field. Instead of: `if (type == 0x01) handle_temp(data, len); else if (type == 0x02) handle_humidity(data, len); ...`, you define a dispatch table: `handler_t dispatch[256] = {[0x01]=handle_temp, [0x02]=handle_humidity, ...}`; and call `dispatch[type](data, len)`. This is $O(1)$ lookup instead of $O(n)$ chain, is cleaner to extend (add a new type without modifying existing code), and is the pattern used by TinyOS component interfaces and ESP-IDF event handlers.

Section 4 — Detailed Answer

DA1. Compare distance vector routing, link state routing, and RPL. Explain how each works, what trade-offs each makes in terms of convergence speed, memory, and control traffic, and why RPL is specifically designed for constrained IoT mesh networks rather than using OSPF or RIP.

Distance vector routing (e.g. RIP, DSDV): each node maintains a routing table `{destination → (next_hop, cost)}` and periodically broadcasts its table to direct neighbours. Neighbours merge received tables using the Bellman-Ford equation: if the neighbour's cost to D plus the link cost to that neighbour is less than the current best, update. Advantages: simple, low per-node memory (only the routing table, not full topology). Disadvantages: slow convergence — the counting-to-infinity problem means a link failure can take many rounds to propagate; mitigated by split horizon and poison reverse but not eliminated.

Link state routing (e.g. OSPF): each node floods its complete local link-state advertisement (LSA) to every other node. Every node builds the full topology graph and runs Dijkstra to compute shortest paths. Advantages: fast convergence (single Dijkstra run after topology change), no counting-to-infinity. Disadvantages: $O(N^2)$ memory for topology graph, high control traffic (flooding all LSAs to all nodes), CPU-intensive Dijkstra — completely impractical on a 256 KB RAM MCU with hundreds of nodes.

RPL is designed specifically for the constraints of IoT LLNs: MCU-class nodes, lossy links, asymmetric connectivity, low bandwidth. It borrows from distance vector (nodes do not need full topology) but builds a single DODAG tree rather than arbitrary shortest paths. Nodes only need to know their parent (uplink route) and a small set of neighbours. The Objective Function (ETX by default) selects parents based on link quality, adapting automatically as links degrade. Control traffic is reduced because most data flows upward in the tree — only Destination Advertisement Objects (DAOs) propagate downward routes. This makes RPL feasible on nodes with 8–64 KB RAM running a minimal 6LoWPAN stack, whereas OSPF's state machine alone would not fit.

DA2. Describe the performance evaluation methodology from the course. Cover: the role of goals, services, and metrics; the distinction between parameters and factors; the purpose of a test harness; how synthetic vs. replayed workloads differ; and what the clock alignment problem is and why it matters for distributed IoT benchmarks.

A well-designed evaluation starts with Goals: what question are you trying to answer? (e.g., "Does increasing QoS level from 0 to 1 reduce message loss under network congestion?"). Goals define the scope and prevent evaluating irrelevant things. Services are the functional capabilities of the system under test that the experiment exercises (e.g., MQTT publish throughput, end-to-end latency). Metrics are quantitative measurements of those services: latency in milliseconds, message loss rate as a percentage, CPU utilisation. Metrics must be defined precisely — "latency" could mean sender-to-broker or sender-to-subscriber.

DA3. Explain the four IoT budget domains and how they interact. For each domain, give one concrete constraint and explain how a design decision in that domain forces a trade-off in at least one other domain. Then describe the generic 5-step budget allocation approach.

Parameters are configuration values held constant within a test run — e.g., message payload size fixed at 20 bytes. Factors are independent variables deliberately varied: QoS level (0, 1, 2), publish rate (10, 100, 1000 msg/s). A full-factorial experiment tests every combination (2^k for k binary factors), which scales exponentially — for 10 factors that is 1024 test cases. Hill-climbing (vary one factor at a time) reduces combinations but may miss interactions. Workload is the input fed to the system: synthetic (generated to match a specification, repeatable, controllable) or replayed (real production traces, realistic but harder to reproduce exactly).

A test harness automates the mapping of factors to configuration, launches the workload generator, collects logs, and feeds them to analysis. Without it, manual tests are inconsistent. The clock alignment problem arises in distributed tests: if the MQTT publisher and subscriber run on different machines with unsynchronised clocks, a naïve latency calculation ($\text{subscribe_time} - \text{publish_time}$) accumulates clock skew error. Solution: synchronise all machines to NTP or GPS-PPS before the experiment and measure skew to bound the error; or use a single-machine loopback test for latency where clock skew is zero.

The four domains are Radio, Memory, Processing, and Energy. They interact through the fundamental constraint that every operation consumes energy, and the energy budget is fixed by the battery.

Radio budget: a LoRaWAN node is limited to 140 uplinks/day at SF12 and 51-byte maximum payload. This forces the Memory budget decision: if you cannot transmit every reading, you must buffer readings in flash or RAM. Buffering in RAM is fast but volatile; buffering in flash extends lifetime but consumes flash write endurance (typically 100,000 cycles). If you buffer 100 readings before each transmission, you spend less radio budget but more memory budget ($100 \times 12 \text{ bytes} = 1.2 \text{ KB RAM}$).

Memory budget: if RAM is insufficient to hold the averaging buffer or the firmware itself, you must reduce feature complexity. Removing features (e.g., dropping local anomaly detection) pushes computation to the cloud, increasing radio budget (more frequent transmissions to ensure the cloud has sufficient data to detect anomalies).

Processing budget: running a local FFT to compress vibration data into frequency features before transmission reduces the radio budget (features are smaller than raw samples) but increases the processing budget. On a Cortex-M0 without an FPU, software floating-point FFT may run for tens of milliseconds — consuming energy and potentially conflicting with a tight sleep cycle.

Energy budget: all three domains drain from it. The 5-step algorithm: (1) Identify all operating states and their current draw (active TX, active RX, sensor read, compute, deep sleep). (2) Assign each state a duty cycle fraction based on application requirements. (3) Calculate average current: $I_{\text{avg}} = \sum(I_{\text{state}} \times \text{duty_fraction})$. (4) Estimate lifetime: $t = \text{Capacity} / I_{\text{avg}}$. (5) If lifetime is insufficient, iterate — reduce radio frequency, increase sleep fraction, reduce payload size, lower SF if link budget allows, or add solar harvesting — until the budget closes.

Question 22 — Smart Bridge Structural Monitoring

(answer not found)

A civil engineering agency wants to deploy IoT sensors on a **suspension bridge** to monitor structural health. The system has:

- **12 vibration sensors** (accelerometers) — one at each cable anchor point, sampling at **200 Hz**.
- **8 strain gauges** — measuring stress on main support beams, sampling at **10 Hz**.
- **2 weather stations** — wind speed, temperature, humidity, sampling at **1 Hz**.

Requirements:

- Detect anomalous vibration patterns (potential fatigue) within **5 seconds**.
- Store all raw sensor data for **20 years** for structural engineering analysis.
- The bridge has **no mains power** at sensor locations; sensors are solar + battery.
- The nearest cellular or internet connection is **2 km away**.

Section 3 — Short Answer

Subquestion 22.1 (short answer)

Estimate the raw data volume generated per year across all sensors, and argue whether raw data or aggregated features should be stored long-term.

Vibration sensors: $12 \text{ sensors} \times 200 \text{ Hz} \times 2 \text{ bytes (int16 accelerometer value)} \times 3 \text{ axes} = 12 \times 200 \times 6 = 14,400 \text{ bytes/s} = \sim 1.24 \text{ GB/day} = \sim 453 \text{ GB/year}$.

Strain gauges: $8 \times 10 \text{ Hz} \times 4 \text{ bytes (float32)} = 320 \text{ bytes/s} = \sim 27.6 \text{ MB/day} = \sim 10 \text{ GB/year}$.

Weather: $2 \times 1 \text{ Hz} \times 3 \text{ channels} \times 4 \text{ bytes} = 24 \text{ bytes/s} = \sim 2 \text{ MB/day} = \sim 0.7 \text{ GB/year}$.

Total raw: $\sim 464 \text{ GB/year}$, $\sim 9.3 \text{ TB}$ over 20 years.

Storing raw vibration data is valuable for structural engineering analysis — anomaly patterns that are only visible in the frequency domain at specific times may not be recoverable from aggregated features. However, 9.3 TB at $\sim \text{€}20/\text{TB/year}$ cold storage is $\sim \text{€}186/\text{year}$ — affordable. The recommendation: store full raw vibration data at high priority for at least 5 years on warm storage; after 5 years, compress to frequency-domain features (FFT bins, peak frequencies, RMS per 1-second window) and archive to cold storage. Strain gauge and weather data are small enough to store raw for the full 20 years.

Subquestion 22.2 (short answer)

Which radio technology connects sensors to the gateway, and what challenges does the bridge environment (metal structure, long span, outdoor) introduce?

LoRa (point-to-point or LoRaWAN) is the right choice for the 2 km backhaul to the nearest internet connection. A single LoRa gateway at the bridge approaches the 2 km link trivially at SF7–SF9 (line-of-sight over open water/road). BLE or Zigbee cannot reach 2 km.

For sensor-to-gateway hops within the bridge (span up to $\sim 500 \text{ m}$), a 2.4 GHz Zigbee or Thread mesh is suitable for the strain gauges and weather stations (low data rate). For the vibration sensors (high data rate), a local WiFi link (802.11n at 2.4 GHz) or a dedicated sub-GHz proprietary radio between sensor clusters and a local aggregator is needed — LoRaWAN's 51-byte/140-message limit cannot carry 200 Hz raw data.

Bridge-specific challenges: (1) Metal structure causes significant multipath reflection and shadowing — the RF environment is highly non-uniform and link quality varies along the span. Site survey is required. (2) Outdoor exposure — antennas and connectors must be rated IP67+. (3) Vibration — connectors must be strain-relieved; MCU components must be rated for vibration (automotive-grade preferred). (4) Temperature range (–20 °C to +60 °C) affects battery capacity and radio frequency stability.

Section 4 — Detailed Answer

Subquestion 22.3 (detailed answer)

Design the full data pipeline from sensor to long-term storage and anomaly detection. Specify where each processing step runs, how the 5-second anomaly detection SLA is met, how you handle the 200 Hz vibration data within the radio budget, and what happens during a 3-day storm that prevents solar charging.

Sensor layer: Vibration sensors (accelerometers) sample at 200 Hz continuously. To stay within radio budget, the sensor MCU computes a local FFT over a 1-second window (200 samples) and extracts: peak frequency, dominant frequency bin amplitudes (top 8 bins), and RMS value — yielding ~40 bytes of features per second per sensor instead of 1200 bytes of raw samples. If a feature anomaly is detected locally (RMS exceeds a threshold, or a new dominant frequency appears), the sensor immediately transmits the full raw 1-second window (1200 bytes) via WiFi to the local aggregator for high-fidelity analysis.

Local aggregator (fog, per bridge span — one Raspberry Pi per ~6 sensors): Receives feature packets every second from all sensors via WiFi/Zigbee. Runs the 5-second anomaly detection: if 3 or more adjacent sensors show correlated anomalous frequency patterns (characteristic of resonance or fatigue), it raises an alert and simultaneously requests a 10-second raw data burst from all sensors. It buffers raw data locally on an SD card with write-ahead logging. It forwards alert events and feature summaries to the gateway over LoRa.

Gateway (one per bridge, with LoRa + LTE modem, solar + large battery): Relays alert events to the cloud within seconds via LTE. Buffers feature summaries during LTE outages. Raw data is not sent over LTE in real-time — it is uploaded in batch via SFTP during low-traffic periods (e.g., overnight) from the aggregator's SD card via Ethernet between aggregator and gateway.

5-second alert SLA: Local anomaly detection on the fog aggregator runs in under 1 second; LoRa transmission to the cloud adds ~1–2 s; total alert latency < 5 s even with LoRa.

3-day storm energy plan: The gateway and aggregators have oversized batteries (e.g., 100 Ah LiFePO₄) calculated to sustain 72 hours without solar. During the storm, non-critical transmissions (batch raw data upload) are suspended; only alert events and hourly feature summaries are transmitted. Sensor MCUs drop to minimum sampling rate (10 Hz) for strain gauges and weather during storm mode, keeping energy use well within the battery reserve.

Practice Exam 9

Question

Model Answer

Section 1 — Multiple Choice

MC1. If you average N independent samples of a noisy sensor signal, by what factor does the SNR improve? • A) N (linear improvement) • B) N^2 (quadratic improvement) • C) $\sqrt{N}$ (square-root improvement) • D) $\log(N)$ (logarithmic improvement)	C When you average N independent, identically distributed noise samples, the signal components sum linearly ($N \times$ signal amplitude) while uncorrelated noise components grow as $\sqrt{N}$ (random walk). The SNR ratio therefore improves by $N/\sqrt{N} = \sqrt{N}$. Practically: averaging 4 samples improves SNR by $2\times$ (3 dB); 16 samples gives $4\times$ (6 dB). This is the mathematical basis for oversampling. A would require correlated noise; B and D have no basis.
MC2. In an Elixir GenServer, which callback handles messages sent with plain <code>send/2</code> or <code>Process.send_after/3</code> — i.e., messages that are neither <code>call</code> nor <code>cast</code>? • A) <code>handle_call/3</code> • B) <code>handle_cast/2</code> • C) <code>handle_info/2</code> • D) <code>handle_message/2</code> MC3. In a voltage divider with supply voltage V_{in}, fixed resistor R_1 (top), and variable resistor R_2 (bottom), what is the output voltage V_{out} measured across R_2? • A) $V_{in} \times R_1 / (R_1 + R_2)$ • B) $V_{in} \times R_2 / (R_1 + R_2)$ • C) $V_{in} \times (R_1 - R_2) / (R_1 + R_2)$ • D) $V_{in} / (R_1 \times R_2)$	C <code>handle_info/2</code> is called for any message in the GenServer's mailbox that was not sent via <code>GenServer.call</code> or <code>GenServer.cast</code> — including bare <code>send(pid, msg)</code>, <code>Process.send_after(self(), :tick, 1000)</code>, and <code>monitor/link :DOWN:EXIT</code> signals. A and B only handle call/cast; D does not exist. B In a voltage divider, $V_{out} = V_{in} \times R_2 / (R_1 + R_2)$. V_{out} is the voltage across the lower resistor R_2. A gives the voltage dropped across R_1; C and D are wrong.
MC4. An NTC (Negative Temperature Coefficient) thermistor is used in a voltage divider as R_2. As temperature rises, what happens to V_{out}? • A) V_{out} increases, because R_2 increases with temperature • B) V_{out} decreases, because R_2 decreases with temperature • C) V_{out} stays constant — only a PTC thermistor changes voltage • D) V_{out} increases, because higher temperature means lower resistance and more current through R_1	B NTC = Negative Temperature Coefficient: resistance decreases as temperature rises. With R_2 (NTC) in the lower leg of the divider: as $T \uparrow$, $R_2 \downarrow$, so $V_{out} = V_{in} \times R_2 / (R_1 + R_2)$ decreases. A describes a PTC thermistor in the lower leg; C is wrong; D correctly identifies the current increase but gives the wrong voltage direction.
MC5. What makes backscatter communication ultra-low-power on the tag side?	B

• A) The tag uses a highly efficient Class-D RF amplifier at 1 mW • B) The tag has no active transmitter — it reflects and modulates an incident carrier wave from an external reader or ambient source, harvesting energy from the field itself • C) The tag uses BLE advertising with a 10-second interval • D) The tag stores energy in a supercapacitor that discharges during each transmission burst	Backscatter tags contain no oscillator, amplifier, or battery. They are powered by energy harvested from the reader's RF field (or ambient RF sources). The tag modulates the reflected wave by switching a load on/off, changing the backscattered signal amplitude or phase. Total tag power consumption is in the nanowatt to microwatt range. A, C, D all describe active transmitters.
MC6. In CSMA/CA with Clear Channel Assessment (CCA), what does an RSSI-threshold CCA check determine before a node transmits? • A) Whether the destination node has acknowledged the previous packet • B) Whether the measured received signal strength on the channel is below a threshold, indicating the channel is idle • C) Whether the node's own transmit power exceeds the legal limit • D) Whether the packet CRC is valid before retransmission	B CCA with RSSI threshold measures the current energy on the channel. If the measured RSSI is below a configured threshold (e.g., -85 dBm), the channel is deemed idle and the node may transmit. This prevents transmitting into an ongoing conversation. A, C, D are unrelated to CCA.
MC7. What is the primary advantage of a CDF (Cumulative Distribution Function) plot over a histogram for analysing IoT message latency data? • A) A CDF uses less memory to compute than a histogram • B) A CDF allows you to directly read exact percentile values (e.g., "95% of messages arrived within X ms") without choosing bin sizes • C) A CDF shows bimodal distributions more clearly than a histogram • D) A CDF plots latency against message size, revealing correlation	B A histogram requires you to choose bin sizes, which affects the visual result, and reading a specific percentile requires integrating across bins mentally. A CDF plots the fraction of samples $\leq x$ on the y-axis against x; you read the 95th percentile by finding where the curve crosses 0.95. This is directly actionable for SLA analysis. A, C, D are wrong.
MC8. In C, what is the accepted legitimate use of <code>goto</code> in embedded firmware, as discussed in the course? • A) Implementing loop constructs that <code>for</code> and <code>while</code> cannot express • B) Structured forward-only cleanup: jumping to a single <code>cleanup: label</code> at the end of a function to release resources after any error, avoiding deeply nested <code>if/else</code> return paths • C) Replacing recursive function calls to avoid stack overflow • D) Implementing state machines that transition between labelled states	B The course introduces <code>goto</code> as a structured cleanup pattern: all error paths jump forward to a single <code>cleanup: label</code> at the end of the function, which releases all allocated resources (<code>close file</code>, <code>free memory</code>, <code>release mutex</code>) before returning. This avoids duplicating cleanup code across every error branch. A, C, D are either wrong or bad practice.
Section 2 — Multiple Answer	
MA1. Which of the following are valid SPARQL features beyond basic triple-pattern matching?	MA1

- A) **FILTER** — restricts results to rows matching a boolean expression (e.g., numeric range, regex)

- B) **OPTIONAL** — includes a pattern if it matches, but does not exclude the row if it does not (left outer join)

- C) **UNION** — returns results matching either of two patterns

- D) **INSERT DATA** — adds new triples to the graph (part of SPARQL Update)

- E) **CONSTRUCT** — returns a new RDF graph built from matched patterns rather than a variable binding table

MA2. Which of the following correctly describe when to use each visualization type for IoT data?

- A) Scatter plot — best for identifying correlations between two variables or visualising a time series

- B) Histogram — best for showing the distribution of a single variable (e.g., packet sizes, latency values)

- C) CDF plot — best for reading exact percentile latencies (e.g., p95, p99 SLAs)

- D) Violin plot — best for comparing factor-dependent distributions side by side (e.g., latency at QoS 0 vs 1 vs 2)

- E) KDE plot — best for hard-bounded data where you want to avoid artificial tails

MA3. Which of the following are correct properties of C bitfields?

- A) A bitfield member declared as `unsigned int flag : 1` occupies exactly 1 bit within the containing struct

- B) Bitfields allow you to map a C struct directly onto a hardware register or protocol frame, reading individual flag bits by name

- C) The C standard guarantees the exact bit-ordering and padding of bitfields across all compilers and platforms

- D) Bitfields cannot be used with `typedef struct` — they require a plain struct declaration

- E) Combining bitfields with `__attribute__((packed))` removes inter-field padding and matches the on-wire byte layout

MA4. Which of the following are valid return values from a GenServer's `init/1` callback?

All five are valid parts of the SPARQL 1.1 specification. **FILTER** (A) restricts rows by expression. **OPTIONAL** (B) adds patterns as a left outer join — rows without the optional match are still returned with unbound variables. **UNION** (C) combines two graph patterns with OR semantics. **INSERT DATA** (D) is part of SPARQL Update (1.1), which adds triples to a graph. **CONSTRUCT** (E) returns a new RDF graph built by substituting matched variables into a template — useful for transforming one RDF schema into another.

MA2

A is correct: scatter plots show two-variable relationships or time-ordered data points. B is correct: histograms show the frequency distribution of a single variable. C is correct: CDF plots are the best choice for SLA analysis (p95, p99 latency targets). D is correct: violin plots combine density estimation and quartile boxes, ideal for comparing how a metric varies across categorical factors. E is wrong: KDE (Kernel Density Estimation) plots are explicitly unsuitable for hard-bounded data (e.g., a 0–100% range) because the kernel smears probability mass outside the valid range.

MA3

A is correct: `unsigned int flag : 1` occupies one bit; the compiler packs multiple bitfield members into a single unsigned int storage unit. B is correct: this is the primary motivation — naming individual bits of a hardware register or protocol frame header makes code readable and avoids manual bit-shift/mask operations. C is wrong: the C standard explicitly leaves bit ordering, padding between fields, and the exact underlying type representation as implementation-defined — bitfields are not portable across compilers or architectures without care. D is wrong: bitfields work perfectly with `typedef struct`. E is correct: `__attribute__((packed))` removes padding between storage units, which combined with careful bit ordering produces a layout matching the on-wire format (with the caveat that bit ordering within a byte is still compiler-dependent).

MA4

• A) `{:ok, initial_state}` — start the server with the given state

• B) `{:ok, initial_state, timeout}` — start the server; send a `:timeout` message to `handle_info` if no message arrives within the given milliseconds

• C) `{:stop, reason}` — abort startup and return an error to the caller of `start_link`

• D) `:ignore` — silently stop the process without reporting an error (used when startup conditions are not met but this is not exceptional)

• E) `{:error, reason}` — return an error tuple directly from `init/1`

A is correct: `{:ok, state}` is the normal success return. B is correct: `{:ok, state, timeout}` causes the server to emit a `:timeout` atom to `handle_info` if no message arrives within the specified number of milliseconds — useful for heartbeat or watchdog logic. C is correct: `{:stop, reason}` causes `start_link` to return `{:error, reason}` and the process exits. D is correct: `:ignore` causes `start_link` to return `:ignore` — used when the server decides it should not start (e.g., a device is not present) without it being an error. E is wrong: `{:error, reason}` is not a valid `init/1` return tuple in OTP — use `{:stop, reason}` instead.

Section 3 — Short Answer

SA1. Describe the voltage divider circuit used to read an NTC thermistor. What is the output voltage formula, and why does V_{out} decrease as temperature rises with an NTC thermistor as R_2 ?

A voltage divider places a fixed resistor R_1 between V_{in} and the output node, and the NTC thermistor R_2 between the output node and GND. The output voltage is $V_{out} = V_{in} \times R_2 / (R_1 + R_2)$. An NTC thermistor has Negative Temperature Coefficient: as temperature rises, R_2 decreases. Substituting into the formula: lower R_2 means the denominator $(R_1 + R_2)$ shrinks less than the numerator, so $V_{out} = V_{in} \times R_2 / (R_1 + R_2)$ falls. The MCU ADC reads V_{out} and computes $R_2 = R_1 \times V_{out} / (V_{in} - V_{out})$, then maps R_2 to temperature using the manufacturer's resistance-temperature table or the Steinhart-Hart equation.

SA2. What is the radio performance space trade-off triangle (range, data rate, power)? Give a concrete example of where LoRa, BLE, and NB-IoT each sit on it.

The three axes are range, data rate, and transmit power. Improving any one forces a trade-off in at least one other: extending range requires either more power or a lower data rate (more robust modulation). LoRa sits at high range (2–15 km), very low data rate (~250 bps–50 kbps), and low power — it sacrifices throughput for range. BLE sits at short range (~10–100 m), moderate data rate (~1 Mbps), and very low power — it sacrifices range for power efficiency and throughput. NB-IoT sits at moderate range (city-scale via cell towers), low data rate (~250 kbps), but requires a cellular modem with higher baseline power than LoRa or BLE. WiFi sits at moderate range (~50 m), high data rate (50+ Mbps), and high power — it trades power for throughput.

SA3. What are the three ACK architectures described in the course (immediate, delayed/block, piggybacked)? Give an IoT scenario where piggybacked ACKs are advantageous.

Immediate ACK: the receiver sends an ACK frame immediately after receiving each individual data frame. Simple and predictable, but the overhead is one ACK per data frame — at 50% channel utilisation. Delayed/Block ACK: the receiver accumulates several data frames and sends a single ACK covering the whole block. Reduces ACK overhead for bulk transfers (e.g., file download) but increases latency before the sender knows if delivery succeeded. Piggybacked ACK: the receiver holds the ACK and appends it to the next outgoing data frame in the reverse direction. Ideal for full-duplex bidirectional links — e.g., a sensor sending readings and a controller sending configuration commands simultaneously. Each direction's data frame carries the ACK for the other direction, halving the number of frames needed.

SA4. Why does averaging N samples improve SNR by $\sqrt{N}$ rather than N? What does this mean practically for oversampling on a noisy ADC?	When averaging N samples, the true signal (deterministic) adds constructively: summing N copies of the signal gives N times the amplitude. Noise is assumed to be independent and identically distributed (uncorrelated) between samples — the noise components add as a random walk, so the RMS noise grows as $\sqrt{N}$. The SNR of the average is therefore $N \times \text{signal} / (\sqrt{N} \times \text{noise}) = \sqrt{N} \times (\text{signal}/\text{noise})$. This gives a 3 dB improvement per 4× oversampling rate, or equivalently one extra effective bit of resolution per 4× oversampling. Practically: to gain 2 extra bits of ADC resolution you need $4^2 = 16$ samples per output value — feasible for slow sensors but impractical for fast ones.
SA5. What is <code>handle_info/2</code> in an Elixir GenServer, and give a concrete IoT backend use case where it is the right callback to use?	<code>handle_info/2</code> receives any message sent to the GenServer's mailbox that is not routed through <code>call</code> or <code>cast</code> — plain send, <code>Process.send_after</code>, or system messages like <code>{:DOWN, ref, :process, pid, reason}</code> from a monitor. Concrete IoT use: a device-state GenServer registers a <code>Process.send_after(self(), :heartbeat_check, 30_000)</code> in <code>init/1</code>. Every 30 seconds <code>handle_info({:heartbeat_check}, state)</code> fires, iterates over the registered devices, finds any that have not reported in over 60 seconds, marks them as "offline" in state, and publishes an alert. It then re-arms the timer with another <code>send_after</code> for the next cycle.
SA6. What is backscatter radio, and what distinguishes passive RFID from ambient backscatter?	Backscatter communication works by having a tag modulate the reflection of an incident carrier wave. An RFID reader transmits a continuous carrier; the tag switches a load impedance on/off, changing how much RF energy is reflected back — the reader detects this modulation. Passive RFID tags harvest all their operating power from the reader's field: they have no battery, no oscillator, and no active transmitter. Current consumption is in the microwatt range. Ambient backscatter extends this further: instead of a dedicated reader, the tag reflects existing TV or WiFi signals from ambient infrastructure, encoding data in the modulation pattern. This enables battery-free sensors that communicate using already-present RF energy, but at very low data rates and short range (< 1 m for TV backscatter, ~1 m for WiFi).
SA7. What is CCA (Clear Channel Assessment), and how does it interact with LPL (Low-Power Listening) to reduce collisions on a shared wireless channel?	Low-Power Listening (LPL) nodes wake briefly, sample the channel energy level (CCA), and sleep again if it is quiet. A CCA using an RSSI threshold checks whether the measured signal strength on the channel exceeds a threshold — if it does, another node is transmitting and the listening node stays awake to receive; if not, it returns to sleep. Before transmitting, a node also performs CCA: if the channel is busy, it applies a random backoff (CSMA/CA) and retries, preventing collisions. LPL and CCA together: the sleep-wake cycle (LPL) minimises idle listening energy; CCA (performed during the wake window) prevents both missed receptions and transmitted collisions on a shared channel.
SA8. What is the <code>goto</code> statement in C, and what is the structured cleanup pattern? Why is this pattern preferred over deeply nested <code>if/else</code> error handling in embedded firmware?	<code>goto</code> transfers control to a labelled statement anywhere in the same function. In production embedded C, it is used for the structured cleanup pattern: when a function must allocate multiple resources (open file, acquire mutex, allocate buffer) and any step can fail, <code>goto</code> jumps forward to a single cleanup label at the function's end that releases whatever was acquired. Without <code>goto</code>, each error branch duplicates the cleanup — error at step 3 must release step 1 and step 2 resources, error at step 2 only step 1, creating code duplication. The goto-to-cleanup pattern keeps cleanup in one place: <code>if (err) goto cleanup;</code> at each step, and <code>cleanup: release_resources(); return err;</code> at the end. Linux kernel code uses this pattern extensively.

SA9. What are C bitfields, and how do they help when parsing or constructing a radio frame header in an MCU? Give one portability caveat.

A bitfield declares struct members with an explicit bit width: `unsigned int type : 4` occupies 4 bits. The compiler packs consecutive bitfield members into an underlying integer storage unit (typically `unsigned int`). This maps a struct directly onto a protocol frame header: `channel : 6; type : 4; source : 8; destination : 8;` for a 26-bit header is readable and maintainable compared to manual bit shifts and masks. Portability caveat: the C standard does not define whether bitfields within a storage unit are packed from the MSB or LSB end — this is implementation-defined. Code using bitfields to match a specific on-wire frame format is not portable across compiler/architecture combinations without explicit verification against the hardware's byte-order conventions.

SA10. What is the difference between an outlier and a "black swan" in the context of IoT sensor data? Why is blindly removing outliers from raw data potentially dangerous?

An outlier is a measurement that lies far outside the typical distribution — usually attributed to sensor noise, EMI, or a transient glitch, and often removed by a median filter or threshold. A black swan is a low-probability, high-impact real-world event: a sensor reading that looks like an outlier but actually represents a genuine physical phenomenon — a sudden pressure spike indicating pipe failure, an anomalous temperature caused by a fire, a structural vibration at an unusual frequency indicating fatigue. Blindly removing outliers risks discarding the most important signals in the dataset. The correct approach is to preserve raw data and apply filters only to derived/processed streams; flag outliers rather than delete them; and investigate extreme values rather than assuming they are noise.

Section 4 — Detailed Answer

DA1. Explain how a voltage divider and an NTC thermistor are used together to measure temperature with an MCU ADC. Cover: the divider equation, how the NTC resistance relates to temperature (qualitative and the B-parameter model), how you convert the measured ADC value to a temperature, what sources of measurement error exist, and how to minimise them.

Circuit: V_{in} (3.3 V) $\rightarrow$ R1 (fixed, e.g. 10 k Ω) $\rightarrow$ V_{out} $\rightarrow$ R2 (NTC thermistor) $\rightarrow$ GND. The ADC pin reads V_{out} . From the divider: $V_{out} = V_{in} \times R2 / (R1 + R2)$. Rearranging to recover R2 from the measured voltage: $R2 = R1 \times V_{out} / (V_{in} - V_{out})$.

NTC resistance vs temperature: NTC thermistors follow an exponential relationship. The simplified B-parameter model: $R(T) = R_{ref} \times \exp(B \times (1/T - 1/T_{ref}))$, where T is absolute temperature (Kelvin), T_{ref} is a reference temperature (usually 298.15 K = 25 °C), R_{ref} is the resistance at T_{ref} , and B is the thermistor's material constant (typically 3000–4000 K). To convert the measured R2 to temperature: $T = B / (\ln(R2/R_{ref}) + B/T_{ref})$, then convert Kelvin to Celsius: $T_{C} = T - 273.15$.

Error sources: (1) ADC quantisation — at 12-bit resolution and 3.3 V reference, 1 LSB = 0.81 mV. Near the midpoint of the divider this translates to ~0.05 °C, but near the extremes (high or low temperature) where $R2 \gg R1$ or $R2 \ll R1$, the sensitivity drops and 1 LSB may represent several degrees. Choose $R1 \approx R2$ at the centre of the expected temperature range to maximise sensitivity. (2) Self-heating — current through the thermistor dissipates power ($I^2 \times R2$), warming the element and biasing the reading upward. Use a high-value R1 to limit current, or duty-cycle the excitation voltage. (3) ADC reference voltage — if $V_{ref} = V_{in}$ and they are the same supply, supply noise appears in both and partially cancels in the ratio; use a stable reference for the best accuracy. (4) Contact resistance and lead resistance — for precision measurements use a 4-wire (Kelvin) connection to exclude lead resistance.

DA2. Explain the radio performance space for IoT. What are the three axes, what fundamental trade-offs exist between them, and how do LoRa, BLE, NB-IoT, Zigbee, and WiFi each occupy different regions? Use this model to justify a technology choice for two contrasting IoT scenarios: (a) a remote weather buoy 20 km offshore that reports temperature once per hour, and (b) a real-time industrial robot arm that streams joint-angle telemetry at 100 Hz within a factory floor.

The radio performance triangle has three axes: range (how far a signal can travel with sufficient link margin), data rate (bits per second delivered to the application), and power consumption (energy per bit or average current draw). These three are fundamentally linked by physics: the Shannon-Hartley capacity limit $C = B \log_2(1 + S/N)$ means that for a fixed channel bandwidth and noise floor, you can either transmit more data per second (accepting less reliability at distance) or use a more robust slow modulation to reach farther.

LoRa sits at the high-range, low-data-rate, low-power vertex: CSS modulation at SF12/BW 125 kHz reaches 15 km line-of-sight at ~250 bps. TX energy per bit is high (1.5 s × 40 mA for a 20-byte packet), but the low duty cycle keeps average current to ~10–50 µA. Ideal for sparse, infrequent sensors across wide areas.

BLE sits at the low-range, moderate-data-rate, ultra-low-power vertex: 1–3 Mbps at 1–100 m, TX current ~5–15 mA for ~1 ms per advertising event. Average power in beacon mode: ~5 µA. Ideal for body-area networks, room-scale sensors, and cable replacements.

NB-IoT sits at the moderate-range, low-data-rate, moderate-power vertex: cellular coverage (km-scale), ~250 kbps downlink, but modem draw during registration ~200 mA and requires a SIM/subscription. Ideal when reliable QoS and server-initiated downlink are needed and power is less critical.

Zigbee and Thread sit between BLE and LoRa: 10–100 m per hop (mesh extends range), 250 kbps, ~30 mA TX. WiFi sits at short-range, very high-data-rate, high-power: 50–300 m, 10–300 Mbps, 100–300 mA TX.

Scenario (a) — remote weather buoy 20 km offshore: LoRa with SF12 and a good external antenna. Data rate (one reading/hour) is negligible. Range (20 km) rules out BLE, Zigbee, and WiFi. NB-IoT would work but requires a cellular subscription and higher power; solar-charged and reporting once an hour, LoRa's duty cycle budget is not limiting. LoRa wins on power and cost.

Scenario (b) — factory robot arm streaming at 100 Hz: WiFi (802.11n). At 100 Hz × 8 joint angles × 4 bytes = 3200 bytes/s = 25.6 kbps, LoRa (250 bps), BLE (1 Mbps but interference-prone in factories), and NB-IoT (latency) are all unsuitable. The robot is mains-powered, so WiFi's power draw is irrelevant. WiFi provides the needed throughput, and 802.11r fast roaming handles movement. Industrial WiFi (802.11n with MIMO) is standard in factory environments.

`start_link/1` is typically called from a supervisor's child spec. It calls `GenServer.start_link(Module, init_arg, opts)`, which spawns a new process linked to the caller. The new process immediately calls `Module.init(init_arg)`.

`init/1` sets up the initial server state. Valid returns: `{:ok, state}` — server starts; `{:ok, state, timeout}` — server starts and emits `:timeout` to `handle_info` after the given ms if no message arrives; `{:stop, reason}` — startup fails; `:ignore` — silently don't start. IoT example: a device registry server. `init(%{})` returns `{:ok, %{devices: %{}}, 30_000}` — initial empty state, 30-second inactivity timeout for a startup health check.

DA3. Describe the complete Elixir `GenServer` lifecycle. Cover `start_link/1`, `init/1`, `handle_call/3`, `handle_cast/2`, `handle_info/2`, and `terminate/2` — what each does, what it must return, and how state flows through them. Include a concrete IoT example (e.g., a device-state registry) that exercises all six callbacks.

	<code>handle_call(request, from, state)</code> handles synchronous requests from <code>GenServer.call/2</code>. Must return <code>{:reply, response, new_state}</code> or <code>{:noreply, new_state}</code> (deferred reply). Example: <code>handle_call({:get_device, id}, _from, state)</code> returns <code>{:reply, Map.get(state.devices, id), state}</code>. The caller blocks until the reply is sent. <code>handle_cast(message, state)</code> handles asynchronous messages from <code>GenServer.cast/2</code>. Must return <code>{:noreply, new_state}</code>. Example: <code>handle_cast({:register_device, id, info}, state)</code> returns <code>{:noreply, put_in(state, [:devices, id], info)}</code>. The caller does not wait. <code>handle_info(message, state)</code> handles all other messages. Must return <code>{:noreply, new_state}</code>. Example: the <code>:timeout</code> from <code>init</code> arrives here: <code>handle_info(:timeout, state)</code> logs "server ready", re-arms a periodic heartbeat timer with <code>Process.send_after(self(), :heartbeat, 60_000)</code>, and returns <code>{:noreply, state}</code>. Also handles <code>{:DOWN, ref, :process, pid, reason}</code> messages from monitored device processes — removing the device from the registry when its process dies. <code>terminate/2</code> is called when the <code>GenServer</code> is about to stop (supervisor shutdown, or <code>{:stop, reason, state}</code> returned from a callback). Example: <code>terminate(:normal, state)</code> writes a snapshot of <code>state.devices</code> to NVS or a database for crash recovery on restart. Not guaranteed to be called on abrupt kills — use it for graceful cleanup only. State flows as the return value of every callback: each handler receives the current state and returns a new state; the BEAM stores this as the process's loop data. There is no mutable variable — the state is functionally threaded through the entire lifecycle.
--	---

Section 5 — Design Question

Question 23 — Smart Electricity Metering (AMI) A utility company wants to deploy Advanced Metering Infrastructure (AMI) across a city of 10,000 residential electricity meters. Requirements: - Each meter must report a 15-minute energy reading to the utility backend. - The system must detect tamper events (magnetic interference, cover removal) within 60 seconds and alert the utility. - Meters must support remote disconnect — the utility can cut power to a meter on demand (e.g., on non-payment). - All readings must be stored for 7 years for billing and regulatory compliance. - GDPR applies: consumption data reveals when residents are home, awake, etc.	(answer not found)
--	--------------------

Section 3 — Short Answer

Subquestion 23.1 (short answer) Which radio technology connects meters to the utility backend, and why? The meters are in basements and metal enclosures. Discuss at least one alternative.	PLC (Power Line Communication) or NB-IoT via cellular are the standard choices for AMI.
--	--

	PLC (e.g., PRIME, G3-PLC, or DLMS/COSEM over PLC) sends data over the existing electricity grid wiring. Advantages: no separate radio infrastructure needed — every meter already connects to the grid; works inside metal meter enclosures and basements because the signal travels on the power cable, not through air. Disadvantages: speed is low (10–500 kbps), signal quality varies with grid noise, requires concentrators (data concentrator units, DCUs) at substations to aggregate data. NB-IoT over cellular: a SIM-equipped modem in each meter connects to the mobile network. Advantages: global coverage, bidirectional (critical for remote disconnect commands and tamper alerts), managed QoS. Disadvantages: ongoing SIM subscription cost per meter (significant at 10,000 units), requires cellular signal inside basements (signal penetration can be poor in metal enclosures). Rejected — LoRaWAN: excellent range but 140 messages/day limit at SF12 conflicts with 96 readings/day (every 15 minutes) plus tamper events and command downlinks. Class C (continuous listen) for remote disconnect would drain a battery instantly. LoRaWAN works for gas/water meters but is marginal for electricity AMI with bidirectional requirements. Choice: NB-IoT for new deployments. The utility already pays grid connection costs; NB-IoT adds a known per-meter subscription. Bidirectional capability for remote disconnect and tamper alerts justifies the cost, and NB-IoT's licensed spectrum gives interference-free operation unlike LoRa.
Subquestion 23.2 (short answer) Identify the two main GDPR obligations the utility must fulfil for this deployment, and describe one technical control that enforces each.	(1) Purpose limitation (Article 5(1)(b)): the utility may only use consumption data for the stated purpose (billing and grid management) and not for secondary purposes (e.g., selling behaviour profiles to advertisers). Technical control: role-based access control in the backend — only billing and grid management services can query raw 15-minute reads; all other internal systems see only monthly aggregates. A data governance policy enforces this at the API layer. (2) Data minimisation (Article 5(1)(c)): the utility should not retain more granular data than necessary for the purpose. 15-minute reads are sufficient for billing and grid balancing; second-level reads (if technically possible) should not be collected unless explicitly required. Technical control: configure the meter firmware to transmit 15-minute aggregates only; raw second-level data never leaves the meter. Implement an automatic retention policy: delete or aggregate raw 15-minute reads after 7 years (billing retention requirement), retaining only monthly summaries beyond that.

Section 4 — Detailed Answer

Subquestion 23.3 (detailed answer)

Design the full system: meter → concentrator → utility backend → billing and long-term storage. How is the remote disconnect command delivered reliably? How are tamper alerts handled? How do you ensure no billing records are lost if a concentrator fails?

Meter (edge): Each meter measures energy consumption, buffers 15-minute interval data in NVS, and transmits to an NB-IoT Data Concentrator Unit (DCU) or directly to the utility backend via NB-IoT. The meter also monitors a tamper switch (cover removal) and a magnetic field sensor (external magnet to bypass the mechanism). On tamper detection, the meter immediately sends an out-of-band alert frame (QoS 1, high priority) regardless of the 15-minute schedule. The remote disconnect relay is controlled by a downlink command; the meter acknowledges the command with an ACK and confirms the new state (connected/disconnected) in the next uplink.

Concentrator (fog/substation): A DCU at each substation aggregates readings from ~100–200 meters via PLC or NB-IoT. It stores all readings locally in a SQLite database with a write-ahead log, acting as a buffer for the cloud uplink. It forwards data to the utility backend over HTTPS/MQTT. If the backend is unreachable, the DCU buffers up to 30 days of readings (typically a few MB). On reconnect, it bulk-uploads with sequence numbers so the backend can detect and fill any gaps. The DCU also validates meter clock drift: if a meter's timestamps deviate by more than 5 minutes from the DCU's NTP-synchronised clock, it flags the readings as potentially unreliable.

Remote disconnect reliability: The utility backend sends a disconnect command to the DCU (MQTT, QoS 2). The DCU relays it to the meter over NB-IoT as a confirmed downlink (NB-IoT PSM wake window). If the meter does not acknowledge within 60 seconds, the DCU retries up to 3 times. If still unacknowledged (meter offline, in deep sleep), the command is queued at the DCU and delivered on the meter's next scheduled uplink wake. The backend receives a final "disconnect confirmed" event before updating the billing system.

Tamper alert flow: Meter detects tamper → immediate NB-IoT uplink with event type TAMPER_COVER_OPEN → DCU receives and forwards to a dedicated alert topic on the MQTT broker → alert service sends a push notification to the field operations team within 60 seconds. All tamper events are written to an immutable audit log (append-only) for regulatory and legal purposes.

Long-term storage: All meter readings are written to a TimescaleDB time-series database with a 7-year retention policy and a compression schedule (lossless delta-of-delta after 90 days). A daily backup job to object storage (S3 or equivalent) provides a second copy. After 7 years, data is aggregated to monthly totals (for dispute resolution) and the raw 15-minute data is deleted per GDPR data minimisation rules.

Practice Exam 10

Question

Model Answer

Section 1 — Multiple Choice

MC1. A PWM signal has a period of 10 ms. If the duty cycle is 30%, what is the ON-time per period?

B

• A) 0.3 ms

30% of 10 ms = 3 ms ON-time. Duty cycle = ON-time / period.

• B) 3 ms • C) 7 ms • D) 30 ms	
MC2. In Elixir, <code>Process.send_after(self(), :tick, 1000)</code> is called inside a <code>GenServer</code>. Which callback handles the <code>:tick</code> message when it arrives?	C
• A) <code>handle_call/3</code> • B) <code>handle_cast/2</code> • C) <code>handle_info/2</code> • D) <code>handle_timeout/2</code>	<code>Process.send_after</code> delivers a plain message; <code>handle_info/2</code> handles all non-call/cast messages in a <code>GenServer</code>.
MC3. In Turtle RDF syntax, what does the prefix declaration <code>@prefix ex: <http://example.org/> .</code> allow you to write instead of <code><http://example.org/sensor>?</code>	B
• A) <code>ex/sensor</code> • B) <code>ex:sensor</code> • C) <code>#sensor</code> • D) <code>@sensor</code>	After <code>@prefix ex: <http://example.org/> .</code>, the CURIE <code>ex:sensor</code> expands to <code><http://example.org/sensor></code>.
MC4. In C, given <code>int arr[] = {10, 20, 30}; int *p = arr;</code>, what is the value of <code>*(p + 2)?</code>	C
• A) 10 • B) 20 • C) 30 • D) The address of <code>arr[2]</code>	<code>p + 2</code> advances the pointer by $2 \times \text{sizeof}(\text{int})$ bytes, pointing to <code>arr[2]</code> which holds 30. Dereferencing gives 30.
MC5. What is the purpose of a sample-and-hold (S&H) circuit placed before an ADC?	B
• A) To amplify the signal to the ADC's full-scale voltage range • B) To freeze the input voltage at the moment of sampling so the ADC can digitise a stable value during its conversion period • C) To filter out frequencies above the Nyquist limit before sampling • D) To convert the digital output back to an analogue voltage for verification	An S&H circuit captures and holds the input voltage steady during the ADC's conversion time, preventing aperture error from a changing input.
MC6. What is the Matter protocol in IoT?	B
• A) A proprietary Zigbee extension developed by Amazon for Alexa devices • B) An open, IP-based application-layer standard (running over Thread or WiFi) enabling interoperability between smart-home devices from different manufacturers • C) A sub-GHz LPWAN standard competing with LoRaWAN for wide-area sensor networks • D) A DTLS-based security profile for CoAP that replaces TLS in constrained environments	Matter is an open IP-based application standard (CSA, backed by Apple/Google/Amazon) running over Thread (802.15.4 mesh) or WiFi, enabling cross-vendor smart-home interoperability.
MC7. When downsampling a signal by factor M, what must you do before discarding samples to avoid aliasing?	B

• A) Upsample first by the same factor M to preserve energy • B) Apply a low-pass anti-aliasing filter with cutoff at $f_{\text{Nyquist}}/(2M)$ before keeping every M-th sample • C) Apply a high-pass filter to remove the DC component • D) Increase the ADC resolution by $\log_2(M)$ bits to compensate for lost samples	Before discarding every M-th sample, apply a low-pass filter at $f_{\text{Nyquist}}/(2M)$ — the new Nyquist limit — or higher-frequency components alias into the kept band.
MC8. In C, what is the danger of the following code: <code>char buf[8]; sprintf(buf, "%s", user_input);</code> when <code>user_input</code> is controlled by an untrusted source? • A) <code>sprintf</code> is not available on embedded systems and will cause a linker error • B) If <code>user_input</code> is longer than 7 characters, <code>sprintf</code> will write beyond <code>buf</code>, causing a stack buffer overflow • C) The <code>%s</code> format specifier is deprecated in C99 and the compiler will warn • D) <code>sprintf</code> always null-terminates, so no overflow is possible regardless of input length	B <code>sprintf</code> writes until it hits a null in <code>user_input</code> with no length check; input > 7 bytes overwrites the stack beyond <code>buf</code>, enabling stack buffer overflow / code execution.
Section 2 — Multiple Answer	
MA1. Which of the following correctly describe properties or use cases of PWM on a microcontroller? • A) PWM can control the brightness of an LED by varying the duty cycle — the eye averages the pulses at sufficiently high frequency • B) PWM can drive a DC motor at variable speed by varying the duty cycle fed to an H-bridge or MOSFET driver • C) A higher PWM frequency always produces smoother output regardless of the connected load's time constant • D) PWM can be used to generate an analogue voltage when followed by a low-pass RC filter, acting as a simple DAC • E) The PWM duty cycle is the ratio of ON-time to the full period, expressed as a percentage	MA1 C is false: smoothness depends on the RC time constant relative to PWM frequency, not frequency alone — a very low-frequency PWM produces visible flicker regardless of duty cycle.
MA2. Which of the following are true of Elixir for comprehensions? • A) <code>for x <- list, do: expr</code> maps over <code>list</code> applying <code>expr</code> to each element • B) A filter clause <code>for x <- list, x > 0, do: x</code> skips elements where the condition is false • C) Comprehensions can iterate over multiple generators: <code>for x <- xs, y <- ys, do: {x, y}</code> produces the Cartesian product • D) <code>for</code> comprehensions always return a list; you cannot produce a map directly • E) The <code>into:</code> option lets you collect results into a different collectable, such as a map or binary	MA2 D is false: with <code>into: %{} </code> you can collect into a map. <code>for k <- keys, do: {k, val}, into: %{} </code> returns a map.
MA3. Which of the following are true of Turtle RDF syntax? • A) A triple is written as <code>subject predicate object .</code> with a period terminating each statement • B) The semicolon <code>;</code> allows multiple predicate-object pairs for the same subject without repeating the subject • C) The comma <code>,</code> allows multiple objects for the same subject and predicate without repeating either • D) Blank nodes (written <code>_:</code> <code>label</code> or <code>[]</code>) represent anonymous resources without a global URI	MA3 E is false: Turtle is a text-based, human-readable serialisation. RDF/EXI or binary Turtle (HDT) are compact formats.

- E) Turtle is a binary serialisation format designed for compact on-wire transmission

MA4. Which of the following are valid ways to implement a periodic task in an Elixir GenServer without an external library?

- A) Call `Process.send_after(self(), :tick, interval_ms)` inside `init/1` and reschedule inside `handle_info(:tick, state)` after each tick
- B) Use `:timer.send_interval(interval_ms, self(), :tick)` inside `init/1` to send `:tick` repeatedly without manual rescheduling
- C) Call `GenServer.cast(self(), :tick)` inside a receive block to create a loop
- D) Override `handle_timeout/2` with a fixed interval set in `init/1`
- E) Spawn a dedicated process that loops with `Process.sleep(interval_ms)` and sends a message to the GenServer PID

MA4

C is wrong (`GenServer.cast` inside `receive` breaks the GenServer loop). D is wrong (no `handle_timeout/2` callback exists in GenServer).

Section 3 — Short Answer

SA1. What is PWM (Pulse Width Modulation), and how is it used to control an LED's brightness on a microcontroller? What is meant by duty cycle, and why does frequency matter?

PWM rapidly switches the GPIO between HIGH and LOW at a fixed frequency. The duty cycle — the fraction of the period the pin is HIGH — determines average power delivered. At $\geq \sim 100$ Hz the human eye averages the pulses and perceives intermediate brightness. At too low a frequency (< 50 Hz), visible flicker occurs. On an MCU, a hardware timer compare register sets the ON/OFF transition times without CPU intervention.

SA2. What is a DAC (Digital-to-Analog Converter), and how does a PWM-plus-RC-filter approach approximate a DAC on an MCU that lacks a dedicated DAC peripheral?

A PWM signal followed by a low-pass RC filter (cutoff $\ll$ PWM frequency) averages the pulses into a DC voltage: $V_{out} \approx V_{cc} \times \text{duty_cycle}$. The ripple amplitude $\approx V_{cc} / (RC \times f_{PWM})$. Resolution equals the PWM counter bit-depth: an 8-bit counter gives 256 steps = $\sim 0.4\%$ resolution. This is simpler than a dedicated DAC (resistor ladder or delta-sigma) and adequate for slow actuators (servo, audio tone, LED dimming).

SA3. What is sample-and-hold, and why is it necessary before an ADC conversion? What artefact does it prevent?

An S&H circuit uses a capacitor and a switch. When the switch closes (sample phase), the cap charges to the input voltage. The switch then opens (hold phase), freezing the voltage on the cap while the ADC performs its multi-step conversion. Without S&H, if the input continues changing during conversion, the digitised value is undefined — this is called aperture error. S&H is especially critical for fast signals or slow successive-approximation ADCs.

SA4. What is downsampling, and what is the critical step required before reducing the sample rate to avoid aliasing? Give a concrete example.

Downsampling by M means keeping every M -th sample, reducing the sample rate from f_{in} to f_{in}/M . Before doing so, a low-pass anti-aliasing filter must cut off all signal content above $f_{in}/(2M)$ (the new Nyquist frequency). Without this, high-frequency components fold back (alias) into the baseband and corrupt the signal. Example: a vibration signal sampled at 1 kHz downsampled by 4 to 250 Hz must be low-pass filtered at 125 Hz first.

SA5. What is pointer arithmetic in C? Given `int *p` pointing to `arr[0]`, what does `p + 3` point to, and how many bytes does that advance the pointer on a 32-bit system where `sizeof(int) == 4`?

Pointer arithmetic advances by multiples of the pointed-to type's size, not by bytes. Given `int *p = &arr[0]`, `p + 3` points to `arr[3]` — $3 \times 4 = 12$ bytes past `p` on a 32-bit system. `*(p + 3)` dereferences that address. This works for arrays because C guarantees contiguous storage. Pointer arithmetic past the end of an array is undefined behaviour and common in firmware bugs.

SA6. What is the <code>Process.send_after/3</code> pattern in Elixir? Describe how a <code>GenServer</code> uses it to implement a periodic heartbeat, and what happens if <code>handle_info</code> is slow relative to the interval.	Call <code>Process.send_after(self(), :tick, interval_ms)</code> at the end of <code>init/1</code> and at the end of each <code>handle_info(:tick, state)</code> handler. Each invocation schedules the next message, producing a periodic beat. If <code>handle_info</code> takes longer than <code>interval_ms</code> to complete, the next tick fires late and the timer drifts forward; real-time drift accumulates. For more precise intervals, record the scheduled time and adjust each reschedule: <code>remaining = interval - elapsed</code>. Alternatively, <code>:timer.send_interval/2</code> sends at a fixed wall-clock cadence.
SA7. What are CURIE prefixes and blank nodes in RDF/Turtle? Give an example of each and explain when blank nodes are useful.	A CURIE (Compact URI Expression) is a short alias: <code>@prefix schema: <https://schema.org/> .</code> lets you write <code>schema:name</code> instead of the full URI. Blank nodes (<code>_:b1</code> or anonymous <code>[]</code>) represent resources without a global identifier — useful for intermediate nodes in a graph, e.g. a measurement observation that has a value and unit but no standalone URI. Example: <code>_:obs1 a sosa:Observation ; sosa:hasResult "22.5"^^xsd:float .</code>
SA8. What is the Matter protocol and what two transport layers does it run over? Why was it created?	Matter (formerly CHIP, from the Connectivity Standards Alliance) is an open application-layer protocol released in 2022. It runs over Thread (IEEE 802.15.4 mesh, via the Thread Border Router) or WiFi/Ethernet. Matter devices expose a standard data model with clusters (OnOff, LevelControl, Thermostat, etc.) queryable over IPv6. It was created to replace the fragmented proprietary smart-home ecosystems (Zigbee, Z-Wave, proprietary WiFi) with a single interoperable standard backed by Apple, Google, Amazon, and Samsung.
SA9. What is <code>snprintf</code> in C, and why should it be preferred over <code>sprintf</code> in embedded firmware? Give the function signature and explain the key parameter.	<code>int snprintf(char *buf, size_t n, const char *fmt, ...)</code> writes at most <code>n - 1</code> characters plus a null terminator into <code>buf</code>, preventing buffer overflow. <code>sprintf</code> has no length limit and will overwrite memory if the formatted output exceeds <code>buf</code>. In embedded firmware where buffers are stack-allocated, <code>snprintf</code> is mandatory with safety-critical or untrusted inputs. Check the return value: if it $\geq n$, output was truncated.
SA10. In system reliability, what is component availability, and how does the availability of a series system with N components each at availability A relate to N? What does this imply for complex IoT systems?	Availability $A = \text{MTBF} / (\text{MTBF} + \text{MTTR})$ — the fraction of time a component is operational. A series system where all N components must work has availability $A_{\text{series}} = A \times A \times \dots \times A$. For identical components each at $A = 0.999$, a chain of 10 gives $A_{\text{series}} = 0.999^{10} \approx 0.990$. As N grows, system availability degrades multiplicatively. For complex IoT systems with many subsystems, this drives the need for redundancy, failover, and designing each component to be as available as possible.

Section 4 — Detailed Answer

DA1. Explain how a microcontroller generates analogue output using PWM. Cover: the hardware mechanism of PWM (timer, compare register, duty cycle), how an RC low-pass filter converts it to a DC voltage, the relationship between RC time constant and PWM frequency for ripple, the resolution limitation (how bit-depth and clock frequency relate), and two concrete IoT actuator use cases.	A hardware PWM peripheral consists of a free-running counter driven by the MCU clock, a period register (sets frequency), and a compare register (sets duty cycle). When the counter matches the compare value, the GPIO toggles. The CPU sets registers once and the hardware runs autonomously, leaving the CPU free. An RC low-pass filter (resistor R + capacitor C) averages the PWM output. The output DC voltage equals $V_{cc} \times D$ where D is duty cycle. Ripple voltage is approximately $V_{cc} \times D(1-D) / (f_{\text{PWM}} \times C \times R)$; to keep ripple $< 1\%$, choose $RC \gg 1/f_{\text{PWM}}$. Resolution: with a P-bit timer counter, you get 2^P duty cycle steps. A 10-bit counter at 80 MHz allows $f_{\text{PWM}} = 80 \text{ MHz} / 1024 \approx 78 \text{ kHz}$, giving 1024 levels $\approx 3 \text{ mV}$ steps with 3.3 V supply. Concrete IoT actuators:
---	---

DA2. Explain upsampling and downsampling in signal processing for IoT. What is the purpose of each, what filtering is required and why, what are the relationships between sample rate, Nyquist limit, and aliasing, and how does this connect to oversampling for ADC noise reduction? Give a concrete example of each operation in an IoT pipeline.

1. LED dimming for a smart streetlight: duty cycle maps to luminance; RC filter not needed as the LED itself is a switched load.
2. Servo motor angle control: a 50 Hz PWM with 1–2 ms pulse width positions the servo arm; no RC filter — the PWM waveform itself is the control signal.

A dedicated DAC gives lower ripple and finer steps but requires a dedicated peripheral or external IC. PWM-DAC is the fallback when only timers are available.

Nyquist–Shannon theorem: a signal must be sampled at $\geq 2 \times$ its highest frequency component to be reconstructed without aliasing.

Oversampling for ADC noise reduction: sample at $f_{\text{new}} = N \times f_{\text{Nyquist}}$, then low-pass filter and downsample. Each $4\times$ oversampling adds 1 effective bit of ADC resolution (SNR improves $\sqrt{N}$ or 3 dB per doubling). A 12-bit ADC oversampled $16\times$ and decimated yields ~ 14 effective bits.

Downsampling pipeline: (1) low-pass filter at $f_{\text{new}}/2 = f_{\text{old}}/(2M)$ to remove content that would alias; (2) decimate — keep every M -th sample. Example: a 1 kHz accelerometer signal downsampled to 100 Hz ($M=10$) needs a 50 Hz low-pass filter applied first.

Upsampling pipeline: insert $M-1$ zeros between samples (zero-insertion), then apply a low-pass interpolation filter at the original Nyquist frequency to fill in the values. This increases sample rate without adding information — it smooths the signal for compatibility with a higher-rate processing stage. Example: merging a 10 Hz temperature stream with a 100 Hz vibration stream for joint analysis requires upsampling temperature to 100 Hz.

IoT relevance: edge nodes running at 200 Hz for local FFT analysis may downsample to 1 Hz summary statistics before LPWAN transmission — critical for staying within the radio budget while preserving anomaly-detection accuracy locally.

DA3. Describe the Elixir for comprehension in depth. Cover: the generator syntax, filter clauses, multiple generators (and their semantics), the `into:` option, and how comprehensions compare to `Enum.map/filter/reduce`. Then describe the `Process.send_after` periodic pattern in a `GenServer`, including how the timer is set up, how it is rescheduled, what drift behaviour results, and how to use `:timer.send_interval` as an alternative. Give a concrete IoT use case for each.

for comprehensions:

```
for x <- list, x > 0, do: x * 2
```

Generator `x <- list` binds each element. Filter `x > 0` drops elements failing the predicate (not an error, just excluded). Multiple generators produce Cartesian products: `for x <- [1,2], y <- [:a, :b], do: {x,y} → [{1, :a}, {1, :b}, {2, :a}, {2, :b}]`. The `into:` option specifies the collectable: `for {k,v} <- pairs, into: %{}, do: {k, v+1}` returns a map. Without `into:`, result is always a list.

Compared to Enum: comprehensions are more readable for multi-generator cases; Enum.map/filter is more composable in pipelines; Stream is lazy while for is eager. For simple transforms, for is idiomatic; for complex pipelines, Enum or Stream is clearer.

Process.send_after periodic pattern:

```
def init(state) do
  Process.send_after(self(), :tick, @interval)
  {:ok, state}
end

def handle_info(:tick, state) do
  new_state = do_work(state)
  Process.send_after(self(), :tick, @interval)
  {:noreply, new_state}
end
```

Each tick reschedules the next one. Drift: if do_work takes t ms, the actual period is $@interval + t$ — drift accumulates forward. To reduce drift, calculate elapsed time and adjust:

```
Process.send_after(self(), :tick, max(0, @interval -
elapsed)).
```

:timer.send_interval(@interval, self(), :tick) sends at fixed wall-clock intervals without manual rescheduling, but cannot be cancelled per-message (use :timer.cancel/1 with the returned reference).

IoT use cases:

- Process.send_after: a device-state cache GenServer that evicts stale entries every 5 seconds — it needs to adjust for variable eviction time.
- :timer.send_interval: a sensor aggregator that must publish MQTT summary stats every 60 seconds at a steady rate regardless of processing time.

Section 5 — Design Question

Question 24 — Smart Building HVAC Automation

(answer not found)

A facilities management company wants to automate HVAC (Heating, Ventilation, and Air Conditioning) in an office building with **50 floors** and **20 zones per floor** (1000 zones total). Requirements:

- Each zone has a **temperature + humidity sensor** (sampling every 30 seconds) and a **motorised vent actuator** (receives open/close/percentage commands).
- A **zone controller** runs a local PID loop: sensor reading → setpoint comparison → vent command. Target latency sensor-to-actuator: **< 2 seconds**.
- The building management system (BMS) must collect all sensor data, log it for **10 years**, and allow facilities managers to view live dashboards and adjust setpoints remotely.
- The building has structured Ethernet cabling on every floor, but individual zones only have power — **no Ethernet drop at each sensor/actuator**.
- Energy efficiency is a key business goal: the system must minimise unnecessary heating/cooling.

Section 3 — Short Answer

Subquestion 24.1 (short answer)

Recommended: Zigbee (IEEE 802.15.4 mesh) or Thread.

Which wireless technology connects zone sensors and actuators to the per-floor controller, and why? Discuss at least one alternative and its trade-offs.

Both operate at 2.4 GHz with low power and mesh routing — important because $50 \times 20 = 1000$ zones spread across multiple floors with walls, ducts, and HVAC metalwork attenuating signals. Mesh operation lets devices relay packets around obstacles. Zigbee is mature with a proven IoT ecosystem; Thread adds IPv6 natively (no coordinator translation layer). Both have enough throughput for 30-second temperature/humidity readings plus vent commands.

Alternative: BLE mesh (Bluetooth Mesh). Flood-based publish-subscribe mesh requiring fewer infrastructure nodes and supported by smartphones for commissioning. Trade-off: higher power than Zigbee/Thread, flood-forwarding is less efficient for large networks, and latency is less predictable — marginal for < 2 -second control if the mesh is large.

Not recommended: WiFi. Power consumption too high for battery-backed sensors; access point density requirement is impractical for 1000 nodes.

Subquestion 24.2 (short answer)

Estimate the sensor data volume per year across all 1000 zones, and describe what data should be stored long-term vs. what can be discarded.

Each zone produces 2 values (temperature + humidity) at 30-second intervals.

Samples per zone per year: $2 \times (365 \times 24 \times 60 \times 60 / 30) = 2 \times 1,051,200 \approx 2.1 \text{ M samples/year}$.

At 4 bytes per float + 8-byte timestamp = 12 bytes/sample $\times 2$ values = 24 bytes per reading $\times 2.1 \text{ M readings/zone/year} \approx 50 \text{ MB/zone/year} \times 1000 \text{ zones} = \sim 50 \text{ GB/year raw}$.

Long-term storage strategy: Keep full-resolution data for 90 days (regulatory, fault investigation). Downsample to 5-minute averages (6 $\times$ compression) for 1 year. Downsample to hourly averages for 10 years. This reduces long-term storage to $\sim 500 \text{ MB/year}$ — easily within a small database. Additionally, store PID setpoint changes and alarm events at full resolution forever; they are sparse and low-volume.

Section 4 — Detailed Answer

Subquestion 24.3 (detailed answer)

Design the full system from sensor to BMS dashboard. Specify: the zone-level control loop, the per-floor concentrator role, the BMS backend architecture (ingestion, storage, PID setpoint API), how the < 2 -second control latency is met, how remote setpoint updates propagate to zone controllers, and how the system degrades gracefully if the BMS goes offline.

Zone level (sensor + controller): Each zone has a Zigbee/Thread node with a combined temperature+humidity sensor (SHT3x) and a motorised vent driver. A small zone controller (e.g., ESP32 or nRF52840) runs a local PID loop entirely on-device: reads sensor $\rightarrow$ compares to local setpoint $\rightarrow$ adjusts vent position. This makes the < 2 -second control latency achievable locally regardless of network or BMS state. Sensor $\rightarrow$ PWM vent command round-trip is $< 100 \text{ ms}$.

Per-floor concentrator: Each floor has a Zigbee/Thread coordinator connected to the building Ethernet. It acts as a border router, translating IEEE 802.15.4 to IP. It receives all 20-zone sensor readings and forwards them over MQTT (QoS 1) to the BMS. It also relays setpoint updates from the BMS down to zone nodes. The concentrator buffers readings locally (SQLite, ~24 h) to handle BMS outages without data loss.

BMS backend (central server on building Ethernet):

- **Ingestion:** MQTT broker (e.g., EMQX or Mosquitto) receives all 1000-zone streams. An ingestion service subscribes and writes to the time-series database.

- **Storage:** InfluxDB or TimescaleDB stores sensor readings with zone ID, floor, timestamp. Grafana provides live dashboards. Downsampling jobs run nightly.

- **Setpoint API:** A REST/WebSocket service lets facilities managers adjust setpoints per zone, floor, or building-wide. Setpoint changes are published to MQTT topics subscribed by floor concentrators, which forward to the relevant zone node. Zone nodes persist the setpoint in non-volatile storage (NVS) so it survives power loss.

Remote disconnect / setpoint propagation reliability: Setpoint updates use MQTT QoS 1 (at-least-once) with the concentrator retaining the last value (`retained: true`). When a zone node reconnects after power loss, it will receive the current setpoint immediately.

BMS offline degradation: Zone controllers hold their last setpoint locally and continue the PID loop independently. Floor concentrators continue storing sensor readings to local SQLite. When BMS reconnects, concentrators replay buffered readings (with timestamps) to fill the gap — no data lost up to the 24-hour local buffer.

Tamper / alarm path: If a vent actuator fails or a sensor reads out-of-range, the zone node publishes to an alarm MQTT topic. The BMS processes alarms separately from telemetry, with a push notification to the facilities manager app. Priority QoS 2 for alarm messages ensures exactly-once delivery.

Energy efficiency goal: BMS-level analytics identify zones with consistently over- or under-shooting setpoints and auto-tune PID parameters. Occupancy schedules can be pushed as setpoint profiles (setback temperatures at night/weekends), reducing HVAC runtime. The aggregated sensor data enables building-wide thermal modelling.

Practice Exam 11

Question

Model Answer

Section 1 — Multiple Choice

MC1. In LoRaWAN, which device class receives a downlink only in the two short receive windows immediately after each uplink transmission?

- A) Class B — uses scheduled beacon-based receive slots
- B) Class C — keeps the receive window open continuously
- C) Class A — opens RX1 and RX2 windows after each uplink, then sleeps
- D) Class D — a bidirectional class for industrial use

C

Class A opens RX1 (1 s after uplink) and RX2 (2 s after uplink), then sleeps until the next uplink. The most power-efficient class.

MC2. In Elixir, what happens if <code>Task.await(task, 5000)</code> is called and the task does not finish within 5000 ms? • A) It returns <code>{:error, :timeout}</code> and the task continues running • B) It raises an exit signal with reason <code>:timeout</code> in the calling process, and also sends an exit to the task • C) It blocks indefinitely until the task completes • D) It returns <code>:timeout</code> as an atom and the task is killed silently	B <code>Task.await</code> raises an exit with <code>:timeout</code> in the calling process if the deadline passes. The task process also receives an exit signal.
MC3. In C, what is the purpose of the <code>volatile</code> keyword when applied to a variable (e.g., <code>volatile uint32_t *reg</code>)? • A) It places the variable in non-volatile flash memory so it survives reset • B) It tells the compiler not to cache or optimise away reads and writes to that variable, because it can change outside normal program flow (e.g., hardware register, ISR-written flag) • C) It marks the variable as read-only; the compiler rejects any write attempt • D) It forces the variable to be stored in a register rather than RAM for speed	B <code>volatile</code> prevents the compiler from optimising away repeated reads/writes, ensuring every access hits the actual memory location. Essential for hardware registers and ISR-shared flags.
MC4. In I2C, what is clock stretching? • A) Increasing the SCL frequency mid-transaction to speed up a long burst transfer • B) A slave device holding SCL low after a byte to pause the master while it processes data • C) The master sending extra clock pulses to recover from a bus-stuck condition • D) A hardware feature that extends the physical I2C bus range by repeating clock edges	B Clock stretching: a slave holds SCL low after receiving a byte to stall the master while the slave finishes processing before the next byte.
MC5. What is the CoAP Observe option (RFC 7641)? • A) A CoAP option that sets the maximum retransmission count for Confirmable messages • B) A mechanism allowing a client to register interest in a resource; the server then pushes notifications whenever the resource changes, without the client repeatedly polling • C) A debugging option that causes the CoAP proxy to log all message payloads • D) A CoAP option equivalent to MQTT QoS 2 that ensures exactly-once delivery	B CoAP Observe (RFC 7641) lets a client register as an observer; the server sends notifications (2.05 Content responses) whenever the resource state changes, avoiding repeated polling.
MC6. What is the key property of a FIR (Finite Impulse Response) digital filter that an IIR filter does not have? • A) FIR filters can achieve arbitrarily sharp roll-off with fewer coefficients than IIR • B) FIR filters are always stable and can achieve linear phase (constant group delay), meaning all frequency components are delayed by the same amount • C) FIR filters use feedback from previous output samples, making them more computationally efficient • D) FIR filters require floating-point arithmetic; IIR filters work with integers	B FIR filters have no feedback loop → always stable. Linear phase is achievable with symmetric coefficients, meaning all frequencies experience the same group delay. IIR filters can be unstable and have non-linear phase.
MC7. In MQTT, what does setting <code>cleanSession = false</code> (or <code>CleanStart = 0</code> in MQTTv5) when connecting to a broker do?	B

• A) Prevents the broker from persisting any messages for this client	<code>cleanSession = false</code> tells the broker to persist the session: subscriptions survive disconnect, and QoS 1/2 messages queued during offline periods are delivered on reconnect.
• B) Asks the broker to resume a previous session: restore subscriptions and deliver any QoS 1/2 messages that arrived while the client was offline • C) Forces the broker to delete all retained messages for this client's topics • D) Causes the broker to reject reconnections if the client previously disconnected uncleanly	
MC8. In a Successive Approximation Register (SAR) ADC, how many clock cycles does a conversion take for an N-bit result?	C
• A) 1 cycle (flash conversion) • B) 2^N cycles (one per possible value) • C) N cycles (one per bit, binary search) • D) N^2 cycles (one per bit pair)	SAR performs a binary search: it tests each bit from MSB to LSB, requiring exactly N comparisons (cycles) for an N-bit result.

Section 2 — Multiple Answer

MA1. Which of the following are true of LoRaWAN join procedures?	MA1
• A) OTAA (Over-the-Air Activation) performs a dynamic join: the device sends a JoinRequest with DevEUI and AppEUI; the server responds with a JoinAccept containing session keys • B) ABP (Activation By Personalisation) hard-codes DevAddr and session keys into the device at manufacture, skipping the join exchange • C) OTAA is more secure than ABP because session keys are derived fresh each join and are never transmitted over the air • D) ABP devices cannot use confirmed uplinks because they have no session keys • E) OTAA requires network coverage at first power-on to activate; ABP can transmit immediately	D is false: ABP devices have hard-coded session keys and can send confirmed uplinks normally. The distinction is how keys are provisioned, not what message types are available.
MA2. Which of the following are valid FreeRTOS task notification mechanisms (as opposed to queues or semaphores)?	MA2
• A) <code>xTaskNotify</code> — sends a 32-bit value directly to a task's notification register • B) <code>xTaskNotifyGive</code> — increments the target task's notification value, acting like a lightweight binary/counting semaphore • C) <code>ulTaskNotifyTake</code> — blocks until the notification value is non-zero, then decrements or clears it • D) <code>xTaskNotifyWait</code> — waits for a direct-to-task notification with bit-mask control over which bits to clear on entry/exit • E) Task notifications can be used to unblock a task from an ISR without waking the scheduler	All five are correct. Task notifications are a lightweight per-task signalling mechanism with a 32-bit value register. They can be used from ISRs using the <code>FromISR</code> variants and avoid the overhead of queue/semaphore data structures.
MA3. Which of the following correctly describe C enum?	MA3
• A) An enum assigns symbolic names to integer constants, improving readability over bare <code>#define</code> integers • B) By default, enum values start at 0 and increment by 1, but specific values can be assigned explicitly • C) The underlying integer type of an enum is guaranteed to be <code>int</code> on all C compilers	C is false: the C standard says the underlying type is implementation-defined (it must be able to hold all values but can be smaller than <code>int</code>). D is false: in C, enum names are in the enclosing scope, so two enums with the same member name conflict.

- D) enum values are scoped to the enum definition; two different enums can use the same name without conflict in standard C
- E) `typedef enum { ... } MyType`; allows you to use `MyType` as a type name without the `enum` keyword

MA4. Which of the following are true of CoAP (Constrained Application Protocol)?

- A) CoAP runs over UDP by default, making it suitable for constrained networks where TCP overhead is prohibitive
- B) CoAP Confirmable (CON) messages require an ACK from the receiver; unacknowledged CON messages are retransmitted with exponential back-off
- C) CoAP uses a four-character method set: GET, POST, PUT, DELETE — analogous to HTTP
- D) CoAP message IDs and tokens serve different purposes: the message ID deduplicates at the transport layer, the token correlates a request with its response at the application layer
- E) CoAP requires TLS for encryption; DTLS is not supported

MA4

E is false: CoAP is specifically designed to use DTLS (Datagram TLS) for security, since CoAP runs over UDP. TLS requires TCP and is not used with CoAP.

Section 3 — Short Answer

SA1. What are LoRaWAN device classes A, B, and C? For each, describe when it opens its receive window and give an IoT use case where that class is the right choice.

Class A (All): Opens RX1 (1 s post-uplink) and RX2 (2 s post-uplink), then sleeps. Downlink only possible after device-initiated uplink. Most power-efficient. Use case: battery-powered soil moisture sensor reporting hourly.

Class B (Beacon): Class A behaviour plus scheduled receive slots synchronised to a network beacon every 128 s. Enables predictable downlink latency. Use case: smart valve that must be remotely opened within a few minutes.

Class C (Continuous): Receiver always open (except during uplink). Lowest latency downlink but highest power draw — needs mains power. Use case: smart streetlight controller requiring immediate dim/on/off commands.

SA2. What is `Task.Supervisor` in Elixir, and how does it differ from spawning tasks with `Task.async`? Give a use case where `Task.Supervisor` is the right choice.

`Task.async/await` links the task to the caller: if the task crashes, the caller crashes too. `Task.Supervisor` runs tasks under a supervisor, isolating crashes — a failing task does not take down the caller. Use `Task.Supervisor.async_nolink/3` when you want `await`-style result collection but crash isolation. Use case: fan-out HTTP requests to 100 IoT devices — a single unreachable device should not crash the aggregator process.

SA3. What is the `volatile` keyword in C, and give two concrete embedded firmware scenarios where omitting it causes a bug?

`volatile` instructs the compiler never to cache the variable in a register or eliminate "redundant" reads — every access goes to memory. **Bug 1:** Without `volatile`, a spin-loop `while (!flag)` where `flag` is set by an ISR may be optimised to an infinite `while (true)` because the compiler sees no write in the loop body. **Bug 2:** A memory-mapped peripheral register read for status (`while (UART->SR & TX_EMPTY == 0)`) gets optimised to a single read if `SR` is not `volatile`, so the loop never sees the hardware updating the register.

SA4. What is the difference between I2C and SPI in terms of bus lines, addressing, and typical use cases? When would you choose SPI over I2C on an MCU?

I2C: 2 wires (SDA, SCL), multi-master capable, 7-bit address space (112 usable), ~400 kHz (fast) or 3.4 MHz (high-speed). Overhead: start/stop conditions, ACK bits. Good for sensors on the same board where wiring simplicity matters.

SPI: 4 wires (MOSI, MISO, SCK, CS per device), no addressing — CS pin selects device. Speeds up to tens of MHz. Full-duplex. No ACK overhead.

	Choose SPI when: you need high data throughput (e.g., ADC at 1 Msps, display driver, SD card), the speed of I2C is insufficient, or full-duplex operation is required.
SA5. What is the CoAP Observe pattern, and how does it differ from MQTT publish/subscribe? What does the server send when the resource changes?	CoAP Observe: client sends a GET with Observe option = 0 to register; server responds with the current value and keeps sending notifications (Observe option = sequence number) whenever the resource changes. The client sends Observe = 1 to deregister. Point-to-point between client and server; no broker. MQTT publish/subscribe: requires a broker that routes messages from publishers to all subscribers matching a topic. Scales to many subscribers easily; CoAP Observe is more direct (no broker) but scales less gracefully with many observers. When the resource changes, the server sends a 2.05 Content response with the new payload and an updated Observe sequence number.
SA6. What is a FIR filter, and how does a moving-average filter (the simplest FIR) work on IoT sensor data? What is the trade-off between window length and responsiveness?	A moving average computes the mean of the last N samples: $y[n] = (x[n] + x[n-1] + \dots + x[n-N+1]) / N$. This is an N-tap FIR filter with all coefficients = $1/N$. It attenuates high-frequency noise by averaging it out while passing slow trends. Trade-off: larger N gives smoother output (more noise rejection, steeper roll-off) but introduces more latency (the output lags the input by $N/2$ samples) and reduces responsiveness to real step changes in the sensor reading.
SA7. What is MQTT persistent session (<code>cleanSession = false</code>)? What does the broker store, and what are the implications for a device that is offline for a long time?	With <code>cleanSession = false</code>, the broker stores: (1) the client's topic subscriptions, (2) in-flight QoS 1/2 messages that have not been acknowledged, and (3) queued QoS 1/2 messages that arrived while the client was offline. On reconnect, delivery resumes. Risk: if the device is offline for a long time, the broker's queue can grow unboundedly. Brokers typically enforce a max queue size or message TTL (MQTTv5 <code>Message-Expiry-Interval</code>). For ephemeral sensors that don't need catch-up, <code>cleanSession = true</code> is simpler.
SA8. Describe the SAR (Successive Approximation Register) ADC conversion process. How does it determine each bit, and why does a 12-bit SAR need exactly 12 clock cycles?	The SAR ADC uses a comparator, a DAC, and an N-bit shift register. Conversion proceeds bit by bit from MSB to LSB. Each cycle: the DAC outputs a trial voltage (current accumulated value + test bit), the comparator checks input > DAC output, the bit is kept (1) or cleared (0), and the next bit is tested. After N cycles the register holds the N-bit digital value. A 12-bit SAR needs exactly 12 cycles → fast and power-efficient; conversion time is deterministic and proportional to resolution.
SA9. What is a FreeRTOS direct-to-task notification, and why is it faster and lighter-weight than using a semaphore or queue for single-producer/single-consumer signalling?	Each FreeRTOS task has a built-in 32-bit notification register. <code>xTaskNotify</code> / <code>xTaskNotifyGive</code> write directly to that register from any context (task or ISR); <code>ulTaskNotifyTake</code> / <code>xTaskNotifyWait</code> block until notified. Unlike semaphores and queues, there is no separate kernel object to allocate or lock — just one atomic register write and a task state change. This removes the overhead of queue data structures, RAM for queue storage, and the mutex protecting the queue. The limitation: only one pending notification per task (no accumulation beyond a 32-bit value).

SA10. What is an IIR (Infinite Impulse Response) filter? Write the general first-order IIR (exponential moving average) formula and explain the role of the smoothing factor α . What is the stability risk with IIR filters?

The first-order IIR (exponential moving average) is: $y[n] = \alpha \times x[n] + (1 - \alpha) \times y[n - 1]$, where $\alpha \in (0, 1]$ is the smoothing factor. α close to 1 = tracks input quickly, little smoothing. α close to 0 = heavy smoothing, slow response. **Stability risk:** IIR filters use feedback from past outputs. If the filter coefficients are chosen so that the poles of the transfer function are at or outside the unit circle, the filter is unstable — output grows without bound. For the first-order EMA, stability requires $0 < \alpha \leq 1$, which is always satisfiable. Higher-order IIR filters (e.g., Butterworth, Chebyshev) require careful coefficient design to keep all poles inside the unit circle.

Section 4 — Detailed Answer

DA1. Explain the LoRaWAN protocol stack in depth. Cover: the three device classes (A/B/C) and their downlink windows, the OTAA vs ABP activation procedures and their security trade-offs, the role of the Network Server and Application Server, how ADR (Adaptive Data Rate) works and why it matters for battery life and network capacity, and the EU868 duty cycle restriction and how it interacts with the choice of spreading factor.

Physical layer: LoRa modulation — chirp spread-spectrum with configurable spreading factor (SF7–SF12) and bandwidth (125/250/500 kHz). Higher SF → longer range, lower data rate, longer time-on-air.

Device classes:

- **Class A:** uplink-triggered RX windows (RX1 at +1 s, RX2 at +2 s, fixed frequency/SF). Lowest power. Only downlinks possible after device-initiated uplink.
- **Class B:** Class A + periodic beacon-synchronised receive slots every 128 s, enabling predictable downlink with bounded latency (~minutes).
- **Class C:** receiver always open. Immediate downlink at any time. Requires mains power.

Join procedures:

- **OTAA:** Device broadcasts JoinRequest (DevEUI, AppEUI, AppKey). Network Server validates and responds with JoinAccept containing encrypted session keys (NwkSKey, AppSKey) derived per-session. Session keys never transmitted in the clear. Freshly derived keys per join limit replay attack windows.
- **ABP:** DevAddr and session keys programmed at manufacture. No join traffic; works immediately even with no server. Risk: keys never rotate; if extracted from device, all traffic is compromised.

Network architecture: End device → Gateway (transparent RF bridge) → Network Server (deduplication, ADR, MACLayer) → Application Server (payload decryption, business logic). Gateways forward all received packets; the Network Server selects the best copy.

ADR (Adaptive Data Rate): The Network Server monitors uplink SNR. If SNR is consistently high (strong signal), it commands the device to use a lower SF (higher data rate, shorter time-on-air) and reduce TX power. This saves battery, increases channel capacity, and reduces duty-cycle consumption. ADR should be disabled for mobile devices.

EU868 duty cycle: Each sub-band has a 1% duty cycle limit (some 0.1%). With SF12/125 kHz, time-on-air for a 20-byte payload is ~2.5 s → maximum one transmission per ~250 s on that sub-band. SF7/125 kHz: ~50 ms → ~5 s inter-packet gap. ADR is critical for dense deployments: pushing devices to SF7 allows ~50× more messages per hour per device within the duty cycle.

DA2. Explain digital filtering for IoT sensor data. Cover: why raw sensor readings are noisy, the difference between FIR and IIR filters (structure, stability, phase), the moving-average FIR and exponential moving average IIR as concrete examples with their formulas, the trade-off between filter aggressiveness and response latency, and where in the IoT pipeline (on-device vs edge vs cloud) each filter type is most appropriate.

DA3. Describe the Elixir Task module and supervision model. Cover: Task.async/await for parallel work, Task.async_stream for bounded concurrency over collections, the difference between linked tasks (crash propagates) and supervised tasks (crash is isolated), Task.Supervisor.start_child and Task.Supervisor.async_nolink, and how this fits into an IoT backend that must fan out HTTP/MQTT requests to 100 devices concurrently without crashing the caller on individual device timeouts.

Why filter: Sensors produce noise (thermal, quantisation, EMI pickup). Raw ADC readings fluctuate $\pm \text{LSBs}$ even on a stable input. Filtering extracts the true signal trend.

FIR (Finite Impulse Response): $y[n] = \sum_{k=0}^{N-1} b[k] \times x[n-k]$ for $k=0..N-1$. Uses only past inputs; no feedback. Always stable. Linear phase achievable (symmetric coefficients) $\rightarrow$ all frequencies delayed equally $\rightarrow$ no phase distortion. Computationally requires N multiplications per sample. Moving average is the simplest: all $b[k] = 1/N$.

IIR (Infinite Impulse Response): $y[n] = \sum_{k=0}^{N-1} b[k] \times x[n-k] - \sum_{k=1}^M a[k] \times y[n-k]$. Uses feedback from past outputs. Can achieve steep roll-off with far fewer coefficients than equivalent FIR. More efficient for MCUs. Non-linear phase. Stability not guaranteed — poles must be inside the unit circle. Exponential moving average is the simplest IIR: $y[n] = \alpha x[n] + (1-\alpha) y[n-1]$.

Aggressiveness vs latency trade-off: A large N (FIR) or small α (IIR) smooths heavily but introduces lag — the output responds slowly to genuine step changes. For alarm detection (e.g., sudden temperature spike), too much filtering delays alert. Dual-stage approach: use heavy filter for trend reporting, raw or lightly filtered for threshold alerting.

Placement in IoT pipeline:

- **On-device (MCU):** Simple IIR (EMA) is ideal — one multiply + one add per sample, no buffer. Reduces transmitted noise before radio transmission.
- **Edge/gateway:** More complex FIR (moving average, windowed sinc) feasible with more compute. Apply before downsampling.
- **Cloud:** Full DSP toolchains, arbitrary filter design. Post-processing historical data for pattern detection.

Task.async/await: Spawns a linked process. Task.async(fn -> work() end) returns a Task struct; Task.await(task, timeout) blocks for the result. The task is linked to the caller — if the task crashes, the caller crashes. If await times out, the task is killed and the caller gets an exit. Use for: parallel map — spawn N tasks, collect all results with Task.await_many.

Task.async_stream: Task.async_stream(collection, fn, max_concurrency: N) processes items with at most N concurrent tasks. Results stream lazily as each finishes. Suitable for bounded fan-out (e.g., query 500 devices but limit to 20 concurrent connections). Crashes propagate by default unless on_timeout: :kill_task and ordered: false are set.

Task.Supervisor: A supervisor that dynamically spawns tasks. Task.Supervisor.start_child(sup, fn) — fire-and-forget; crash is isolated from caller.

Task.Supervisor.async_nolink(sup, fn) — spawns a task under the supervisor without linking to the caller; you can still await the result, but a crash does not kill the awaiting process (it returns {:exit, reason}).

Task.Supervisor.async_stream_nolink combines stream processing with crash isolation.

IoT fan-out scenario: An MQTT backend receives a broadcast command to poll 100 field devices via HTTP. Using plain `Task.async`, one unresponsive device (timeout) kills the aggregator. Using `Task.Supervisor.async_stream_nolink` with `timeout: 3_000`, `on_timeout: :kill_task`: each device is queried concurrently (max 20), timeouts are logged as errors, and results for responsive devices are returned normally. The aggregator process remains alive and responds to the next command regardless of individual failures.

Section 5 — Design Question

Question 25 — Urban Air Quality Monitoring Network

(answer not found)

A city environmental agency wants to deploy a **dense air quality monitoring network** with **500 sensor nodes** spread across the city. Each node measures: PM2.5, PM10, NO₂, O₃, CO, temperature, and humidity.

Requirements:

- Nodes are **solar + battery** powered with no wired connectivity.
- Each node must upload a **1-minute average reading** (all 7 values) to the city backend.
- The backend must provide a **public API** and **live map** updated every 5 minutes.
- When **PM2.5 exceeds 75 µg/m³**, the system must trigger an **air quality alert** within **3 minutes** to notify the city's health dashboard.
- Data must be retained for **5 years** for epidemiological research.
- Nodes are deployed in diverse environments: open plazas, street canyons, parks, underpasses.

Section 3 — Short Answer

Subquestion 25.1 (short answer)

Which radio technology connects the nodes to the backend, and why? Address the mixed urban environments (open areas vs street canyons vs underpasses) and the power constraint. Discuss one alternative.

Recommended: LoRaWAN.

Rationale: 500 nodes spread across a city with solar + battery power. LoRa's range (2–5 km line-of-sight, 1–2 km in dense urban) allows a small number of gateways (8–12) to cover a city. Power consumption is very low — Class A devices sleep between 1-minute transmissions. Sub-GHz penetration (868 MHz EU) helps in street canyons and underpasses compared to 2.4 GHz technologies.

Challenge: EU868 duty cycle (1%). A 1-minute reading with 7 values (e.g., ~30 bytes payload) at SF9/125 kHz has ToA ≈ 200 ms → duty cycle ≈ 0.33% per node, well within 1%. SF may need tuning for underpasses.

Alternative: NB-IoT. Deeper building/urban penetration (up to 20 dB gain over GSM), guaranteed 3GPP QoS, no duty cycle restriction. Trade-off: higher module cost, cellular subscription cost per node, higher modem power consumption (less suitable for solar + small battery). Better for underground sensors but economically harder at 500-node scale.

Subquestion 25.2 (short answer)

Estimate total data volume per year across all 500 nodes, and describe your storage and retention strategy.

Each node sends 7 float values (4 bytes each) + timestamp (8 bytes) + metadata (node ID, 4 bytes) = 40 bytes per reading. At 1 reading/minute: $40 \times 60 \times 24 \times 365 = \sim 21 \text{ MB/node/year}$ × 500 nodes = **~10 GB/year raw**.

Storage strategy:

- Full resolution (1-minute readings) retained for **90 days** → ~2.5 GB active storage; indexed time-series DB (InfluxDB/TimescaleDB).
- Downsample to 5-minute averages after 90 days, keep for **2 years** → ~500 MB.
- Downsample to hourly averages for **5 years** → ~50 MB.
- Raw data exported to cold object storage (S3/MinIO) for 5 years for epidemiological research access.
- Alert events (threshold exceedances) stored at full resolution permanently — sparse, negligible size.

Section 4 — Detailed Answer

Subquestion 25.3 (detailed answer)

Design the full system from sensor node to public API and alert pipeline. Specify: what processing runs on the node vs in the cloud, how the 3-minute alert SLA is met, how the public map is served at scale, how you handle nodes that go offline due to cloudy weather (battery depletion), and what happens to data integrity during a backend outage.

Node-side processing: Each node samples sensors continuously, averages over 60 seconds, and transmits a single LoRaWAN uplink (Class A) once per minute. On-device IIR filter (EMA, $\alpha \approx 0.3$) smooths noisy ADC readings before averaging. PM2.5 threshold check runs on-device: if a single 1-second reading exceeds $100 \mu\text{g}/\text{m}^3$ (a safety margin above the $75 \mu\text{g}/\text{m}^3$ alert threshold), the node immediately sends an out-of-cycle alert uplink. This is the fast-path for the 3-minute SLA.

Gateway → Network Server → Application Server:

8–12 LoRaWAN gateways cover the city. The Network Server (e.g., TTN or self-hosted ChirpStack) deduplicates packets from multiple gateways and forwards decoded payloads to the Application Server via MQTT or HTTP webhook.

Application Server / Backend:

- **Ingestion service:** subscribes to MQTT broker, writes to TimescaleDB with node ID and timestamp.
- **Alert pipeline:** a separate stream processor (e.g., Elixir GenStage or a simple Kafka consumer) evaluates each incoming reading. If $\text{PM}_{2.5} > 75 \mu\text{g}/\text{m}^3$, it immediately writes to the alerts table and triggers a webhook/push notification to the health dashboard. Latency: LoRa uplink $\approx 1 \text{ s}$, network server processing $\approx 1 \text{ s}$, alert pipeline $\approx < 1 \text{ s}$ → total $< 3 \text{ s}$ for the on-device fast-path trigger. For the 1-minute average path, worst case is 1 minute + processing $\approx 62 \text{ s}$, well within the 3-minute SLA.

Public API and live map:

- REST API (e.g., Phoenix/Elixir or FastAPI) queries TimescaleDB for the latest reading per node (materialised view, refreshed every 60 s).
- Public map frontend (Leaflet.js) polls the API every 5 minutes. Alternatively, a WebSocket endpoint pushes updates. Map tiles served from CDN; sensor data API behind a rate-limited public endpoint.
- For scale (city-wide public access during high-pollution events), API responses are cached in Redis with a 60-second TTL.

Battery-depleted offline nodes:

Nodes that run out of battery (3+ cloudy days) simply stop transmitting. The backend marks nodes as "offline" after 5 missed expected readings (5 minutes). The public map shows last-known reading with a staleness indicator. No data integrity issue — the backend never received data that was not transmitted. When the node recovers and reconnects, it resumes Class A uplinks; there is no replay buffer (LoRaWAN Class A has no store-and-forward). For critical nodes (e.g., near schools), deployment with larger battery bank or redundant adjacent node is recommended.

Backend outage:

Nodes continue transmitting independently. LoRaWAN gateways buffer packets locally (if gateway has local storage) or the Network Server queues them. If the Application Server is unavailable, MQTT messages accumulate in the broker's queue (persistent session). On recovery, the ingestion service replays the queue. For a complete outage > broker queue depth, packets are lost — mitigated by deploying the backend with HA (primary + replica) and a durable message queue (Kafka with replication) between the Network Server and Application Server.

Practice Exam 12**Question****Model Answer****Section 1 — Multiple Choice**

MC1. In SPI, what do CPOL (clock polarity) and CPHA (clock phase) together define?

- A) The baud rate and word length of each SPI transaction
- B) The idle state of the clock line and the edge (rising or falling) on which data is sampled, determining which of the four SPI modes is used
- C) Whether the SPI bus operates in full-duplex or half-duplex mode
- D) The number of chip-select lines and their active polarity

B

CPOL sets the idle clock level (0=low, 1=high); CPHA sets whether data is sampled on the first or second edge. Together they define one of four modes (0–3).

MC2. What is the primary purpose of the Elixir Registry module?

- A) A key-value store backed by ETS for persisting process state across restarts
- B) A process name registry that allows processes to be looked up by a key (e.g., a device ID), supports both unique and duplicate registrations, and enables pub/sub dispatch
- C) A module that registers MFA callbacks for supervision tree configuration
- D) A replacement for GenServer that adds automatic persistence

B

Registry provides per-process name registration keyed by arbitrary terms, enabling lookup by device ID, and supports dispatch/3 for pub/sub fan-out to all registered processes under a key.

MC3. In C, what is a union, and what distinguishes it from a struct?

- A) A union allocates separate memory for each member, like a struct, but members are type-checked at compile time
- B) A union allocates only enough memory for its largest member, and all members share the same memory — reading a different member than was written reinterprets the bytes
- C) A union is identical to a struct but is padded to a power-of-two size
- D) A union can only hold primitive types; struct supports any member type

B

A union reserves space equal to its largest member and overlays all members at the same address. Writing one member and reading another reinterprets the bytes — useful for type-punning and frame parsing.

MC4. Why is UDP typically preferred over TCP for constrained IoT devices and LPWAN protocols?

- A) UDP provides reliable delivery with less CPU overhead than TCP

B

UDP eliminates the TCP three-way handshake, retransmission state, and per-connection buffers. For sensors that rarely need retransmission (application-layer ACKs are sufficient), UDP is far lighter.

• B) UDP is connection-less and has no handshake, no retransmission logic, and no per-connection state — reducing overhead in RAM, code size, and transmission count for devices that tolerate occasional loss • C) UDP is mandatory by the IPv6 standard for all IoT devices • D) UDP supports multicast; TCP does not, which is required for IoT group commands	
MC5. What is MQTT-SN (MQTT for Sensor Networks), and how does it differ from standard MQTT? • A) MQTT-SN is MQTT with TLS always enabled; standard MQTT is unencrypted • B) MQTT-SN is a variant designed for non-TCP networks (e.g., Zigbee, UDP); it compresses topic names to short numeric IDs and removes the TCP dependency • C) MQTT-SN adds QoS level 3 (exactly-once with hardware acknowledgement) for sensor data • D) MQTT-SN requires a dedicated hardware MQTT chip; standard MQTT runs in software	B MQTT-SN replaces string topic names with 2-byte topic IDs pre-registered with a gateway, and operates over UDP or Zigbee — removing the TCP requirement.
MC6. In FreeRTOS, what does passing <code>portMAX_DELAY</code> as the timeout to a blocking API call (e.g., <code>xQueueReceive</code>) mean? • A) The call times out immediately and returns <code>pdFALSE</code> if the resource is not available • B) The call blocks indefinitely — it will not return until the resource becomes available, regardless of how long it takes • C) The call uses the maximum configured tick count (e.g., 1000 ticks) as the timeout • D) The call converts the delay to milliseconds using <code>pdMS_TO_TICKS</code>	B <code>portMAX_DELAY</code> causes the call to block without a timeout until the condition is met (space in queue, semaphore available, etc.). It maps to the maximum tick count value.
MC7. What is DMA (Direct Memory Access) on a microcontroller, and what problem does it solve? • A) A cache controller that speeds up flash reads by prefetching instructions • B) A hardware peripheral that transfers data between memory and peripherals (or memory to memory) without CPU involvement, freeing the CPU to execute other code during the transfer • C) A protection mechanism that prevents user code from accessing kernel memory regions • D) A debugger interface that reads MCU memory over JTAG without halting the core	B DMA lets a peripheral or memory region transfer a block of data directly to another memory region or peripheral while the CPU runs other tasks. Common for UART/SPI/I2C DMA transfers and ADC burst sampling.
MC8. In the SSN/SOSA ontology (W3C), what does <code>sosa:Observation</code> represent? • A) A physical sensor device attached to a platform • B) The act of observing a property of a feature of interest, producing a result at a specific time • C) The calibration metadata for a sensor, including accuracy and measurement range • D) A SPARQL query template for retrieving sensor readings from a triplestore	B <code>sosa:Observation</code> is the core class representing a single act of sensing: it links a sensor (<code>sosa:madeBySensor</code>), the observed property, the feature of interest, the result, and the result time.
Section 2 — Multiple Answer	
MA1. Which of the following are true of RDF named graphs?	MA1

• A) A named graph is an RDF graph identified by a URI, allowing multiple graphs to coexist in a dataset and be queried independently • B) SPARQL's GRAPH keyword lets you restrict a query pattern to a specific named graph • C) Named graphs are used to attach provenance metadata (source, timestamp) to a set of triples without mixing it with the data triples • D) The default graph in a SPARQL dataset always has a URI and can be named explicitly • E) Named graphs are an RDF 1.1 feature; RDF 1.0 had no concept of multiple graphs in a dataset	D is false: the default graph typically has no URI (it is anonymous); it is not a "named" graph in the RDF 1.1 dataset model.
MA2. Which of the following are valid ETS table types in Elixir/Erlang, and what distinguishes them? • A) set — one entry per key; inserting a duplicate key overwrites the existing entry • B) ordered_set — like set but keys are stored in sorted term order, enabling range scans • C) bag — multiple entries with the same key are allowed; all values for a key are distinct • D) duplicate_bag — multiple entries with the same key AND same value are allowed • E) list — a linked-list backed table optimised for sequential access	MA2 E is false: there is no list ETS table type. The four types are set, ordered_set, bag, and duplicate_bag.
MA3. Which of the following are correct descriptions of Zigbee network node roles? • A) Coordinator — exactly one per network; forms the network, assigns addresses, holds the trust centre for security • B) Router — a mains-powered node that can relay packets for other nodes and join new child nodes to the network • C) End device — a typically battery-powered node that cannot route; sleeps between transmissions and polls its parent router for buffered messages • D) Gateway — a Zigbee-specific node that connects the Zigbee mesh to an Ethernet backbone; required in every Zigbee network • E) Sleepy end device — a sub-type of end device that uses long sleep intervals; parents buffer downlink messages on its behalf	MA3 D is false: "Gateway" is not a Zigbee-defined role — it is an external component that may be built on a coordinator or router node, but the Zigbee specification does not define a gateway role. Every network must have exactly one coordinator, but a gateway (to IP) is an architectural choice, not a required node type.
MA4. Which of the following are valid IoT security mechanisms described in the course? • A) DTLS (Datagram TLS) — TLS adapted for UDP, used to secure CoAP; provides the same confidentiality and authentication as TLS but tolerates packet loss and reordering • B) Pre-shared keys (PSK) — an alternative to certificate-based DTLS where both parties share a secret key provisioned at manufacture, reducing the handshake overhead • C) OSCORE (Object Security for CoRE) — end-to-end encryption of CoAP payloads at the application layer, surviving proxies that would otherwise see decrypted traffic • D) WPA2-Enterprise requires each IoT device to have a user certificate and cannot use PSK • E) TLS 1.3 reduces the handshake to one round-trip (1-RTT), making it more feasible for constrained IoT devices than TLS 1.2	MA4 D is false: WPA2-Personal (PSK) is widely used in IoT WiFi deployments. WPA2-Enterprise uses 802.1X/EAP (often certificates), but it is not the only option and is not a blanket requirement.

Section 3 — Short Answer

SA1. What are the four SPI modes (Mode 0–3)? Define CPOL and CPHA and explain which edge data is sampled on in Mode 0 and Mode 3.	<pre> Mode CPOL CPHA Clock idle Sample edge </pre> <pre> ----- ----- ----- ----- ----- </pre> <pre> 0 0 0 Low Rising </pre> <pre> 1 0 1 Low Falling </pre> <pre> 2 1 0 High Falling </pre> <pre> 3 1 1 High Rising </pre> Mode 0: clock idles low; data is sampled on the first (rising) edge. Mode 3: clock idles high; data is sampled on the second (rising) edge. Modes 0 and 3 both sample on a rising edge but differ in idle state. Most common SPI sensors default to Mode 0.
SA2. What is the Elixir Registry module, and how would you use it to look up a GenServer process managing a specific IoT device by its device ID?	Registry is started as part of the supervision tree: {Registry, keys: :unique, name: MyApp.Registry}. A GenServer registers itself on start: Registry.register(MyApp.Registry, device_id, nil). To look up and send a message: [{pid, _}] = Registry.lookup(MyApp.Registry, device_id); GenServer.call(pid, :get_state). This eliminates the need for a global process name per device — important when 10,000 device processes are running concurrently. keys: :duplicate enables pub/sub (multiple processes per key); dispatch/3 calls a function for each registered process.
SA3. What is a C union, and give a concrete embedded firmware use case where a union between a struct and a byte array is useful for protocol frame parsing?	A union overlays all members at the same memory address: <pre>typedef union { struct { uint8_t type; uint16_t length; uint8_t payload[60]; } fields; uint8_t raw[63]; } Frame;</pre> When a radio DMA interrupt delivers raw bytes into frame.raw, the firmware immediately reads frame.fields.type without any manual byte-shifting or memcpy. The bytes are reinterpreted in place. Caveat: endianness and struct padding must match the on-wire format — use __attribute__((packed)) to eliminate padding.
SA4. What is NTP (Network Time Protocol), and why is accurate time synchronisation important for IoT systems? What is the typical accuracy of NTP over the internet vs a local NTP server?	NTP (RFC 5905) synchronises clocks by measuring round-trip delay to a time server and adjusting the local clock. Over the public internet: typical accuracy 10–100 ms (varies with network jitter). Local NTP server on the same LAN: 1–10 ms. GPS-disciplined NTP: < 1 μs. IoT importance: timestamps from multiple sensors must be comparable to correlate events (e.g., vibration onset across 12 accelerometers on a bridge). Without synchronisation, a 500 ms timestamp error makes cross-sensor event ordering meaningless. Distributed systems also need synchronized clocks for vector clock comparison and log ordering.

SA5. What is CRC (Cyclic Redundancy Check), and how is it used to detect errors in IoT radio frames? What does a CRC mismatch indicate and what action should the receiver take?	A CRC appends a short checksum (typically 16 or 32 bits) to a frame. The sender divides the data polynomial by a generator polynomial; the remainder is the CRC. The receiver performs the same division on the received frame — if the remainder is zero (or a fixed constant), no error was detected; otherwise, one or more bits were corrupted. A CRC mismatch means the frame should be discarded. The receiver does not attempt correction — it either requests a retransmission (ARQ) or relies on forward error correction (FEC) coded separately. IEEE 802.15.4 (Zigbee, Thread) uses a 16-bit CRC; LoRa has an optional payload CRC.
SA6. What is DMA, and give a concrete IoT firmware example where DMA significantly reduces CPU load compared to interrupt-driven or polling-based data transfer?	Without DMA, receiving 1024 ADC samples at 100 ksps via SPI requires the CPU to handle an interrupt every 10 μs, execute ISR code, copy the byte — consuming significant CPU time and causing jitter. With DMA: configure a DMA channel to transfer 1024 bytes from the SPI data register directly into a buffer; the CPU is interrupted only once when the transfer completes. Concrete example: an audio or vibration MCU sampling a MEMS accelerometer at 1 kHz over SPI — DMA fills a 200-sample buffer while the CPU runs the FFT on the previous buffer (double-buffering), achieving full throughput without CPU polling overhead.
SA7. What is a SPARQL CONSTRUCT query, and how does it differ from SELECT? Give an example use case in an IoT context.	CONSTRUCT returns a new RDF graph (a set of triples) built by substituting matched variables into a template, rather than a table of bindings (SELECT). Example: <pre> CONSTRUCT { ?obs sosa:hasSimpleResult ?val ; schema:dateCreated ?t } WHERE { ?obs a sosa:Observation ; sosa:hasResult/sosa:value ?val ; sosa:resultTime ?t }</pre> IoT use case: transforming raw sensor observations from SOSA into a simplified schema for a public API, or converting one ontology representation to another for interoperability — without materialising the transformation in the database.
SA8. What is the Kalman filter, and why is it used in IoT sensor fusion (e.g., combining accelerometer and gyroscope data)? Name the two steps of the filter.	The Kalman filter is an optimal linear estimator that fuses noisy measurements with a process model to estimate hidden state. It assumes Gaussian noise. Two steps per cycle: Predict: use the process model (e.g., physics of motion) to project the state forward: $\mathbf{x}_{k+1 k} = \mathbf{F} \times \mathbf{x}_{k k}$, $\mathbf{P}_{k+1 k} = \mathbf{F} \times \mathbf{P}_{k k} \times \mathbf{F}^T + \mathbf{Q}$. Update: incorporate the new measurement to correct the prediction: compute Kalman gain $\mathbf{K}$, then $\mathbf{x}_{k k} = \mathbf{x}_{k+1 k} + \mathbf{K}(\mathbf{z} - \mathbf{H} \mathbf{x}_{k+1 k})$. IoT use: fuse accelerometer (noisy, drifts) with gyroscope (good short-term, long-term drift) for orientation estimation (e.g., drone attitude control). The filter weights each source by its noise covariance — trusting the accelerometer for gravity-aligned correction and the gyroscope for fast angular rate.
SA9. What is MQTT-SN, and in what type of IoT network would you deploy it instead of standard MQTT? Describe the topic alias mechanism.	MQTT-SN is a publish/subscribe protocol for constrained, non-TCP networks (IEEE 802.15.4, UDP, serial). Key differences from MQTT: (1) Topic aliases: string topic names are replaced with 2-byte <code>TopicId</code> integers. Devices first register a topic string with the MQTT-SN gateway (REGISTER $\rightarrow$ REGACK), then use the short ID in all PUBLISH messages — reducing overhead from 20+ bytes to 2 bytes per topic reference. (2) No TCP: runs directly over UDP, Zigbee frames, or serial. (3) Sleep support: end devices can enter sleep mode and notify the gateway; the gateway buffers messages and delivers them when the device wakes and sends AWAKE. Deploy when devices have < 1 kB RAM and use IEEE 802.15.4 radio.

SA10. What are the Zigbee coordinator, router, and end device roles? How does a sleepy end device receive downlink messages from its router parent?

Coordinator: one per network. Starts the network on a chosen channel and PAN ID, manages the trust centre (distributes the network key), and assigns short 16-bit addresses. Always mains-powered.

Router: mains-powered. Participates in mesh routing (AODV-based), accepts child associations from end devices, and buffers downlink messages for sleeping children.

End device: typically battery-powered. Cannot route. Joins the network through a parent router. **Downlink for sleepy end device:** the device sends a Data Request MAC command to its parent on waking. The parent responds with any buffered frames (frame-pending bit set in ACK). If no pending data, the parent sends an empty response. The end device then returns to sleep.

Section 4 — Detailed Answer

DA1. Explain the distinction between edge computing, fog computing, and cloud computing in the context of IoT. For each layer, describe what processing is appropriate, the latency and bandwidth trade-offs, and give a concrete example. Then explain how data flows through all three layers in a real-time industrial IoT pipeline with a 10 ms control latency requirement.

Cloud computing: centralised data centres with virtually unlimited compute, storage, and analytics. Latency: 10–200 ms round-trip. Best for: long-term storage, complex ML model training, historical analytics, billing, dashboards. Not suitable for real-time control.

Edge computing: compute embedded in or co-located with the device itself — MCU on the sensor, a gateway at the machine, or a small server at the factory floor. Latency: < 1 ms (local). Best for: closed-loop control, anomaly detection that must act within milliseconds, preprocessing to reduce uplink bandwidth. Constrained by local RAM/CPU.

Fog computing: intermediate compute layer between edge and cloud — a local server (industrial PC, rack server) at the site/floor. Latency: 1–10 ms. Best for: site-level aggregation, complex processing that the edge MCU cannot run but does not need cloud round-trip, local historian, on-site ML inference.

Data flow in a 10 ms industrial control pipeline:

1. Sensor (edge MCU) samples at 10 kHz, runs a local PID loop — actuator response in < 1 ms.
2. Edge node streams compressed feature vectors (RMS, peak) at 100 Hz to the fog server.
3. Fog server runs anomaly detection (neural network inference) with 5–10 ms latency; raises alarms locally.
4. Fog sends hourly aggregates + model retraining data to the cloud.
5. Cloud retrains the anomaly model weekly; pushes updated model weights to the fog server via OTA.

The 10 ms control loop never leaves the edge — cloud or fog involvement would violate the latency budget.

DA2. Describe the IoT security stack. Cover: why security matters (attack surface), the role of DTLS vs TLS, PSK vs certificate-based authentication, OSCORE for end-to-end security through CoAP proxies, secure boot and firmware signing for MCUs, and the challenge of key management at scale for 10,000 deployed devices. Conclude with the principle of defence in depth.

Attack surface: IoT devices are physically accessible (firmware extraction, bus tapping), resource-constrained (can't run heavy crypto), and deployed in large homogeneous fleets (one firmware vulnerability affects thousands).

Transport security — DTLS vs TLS: CoAP runs over UDP → use DTLS (RFC 6347). DTLS adds a handshake for mutual authentication and a session key, then encrypts/MACs each datagram. Tolerates packet loss and reordering unlike TLS (TCP-dependent). Overhead: ~13 bytes header per record.

PSK vs certificates: Certificate-based DTLS requires a PKI, certificate storage (~1–2 kB), and elliptic-curve operations (~200 ms on Cortex-M0 without hardware acceleration). PSK: a pre-shared 16–32 byte key provisioned at manufacture. Handshake is faster (< 50 ms), requires no PKI. Trade-off: PSK is harder to revoke — if compromised, all devices sharing that key must be reprovisioned.

OSCORE: Encrypts/authenticates the CoAP payload end-to-end using AEAD ciphers. A CoAP proxy can forward the packet without decrypting it — providing application-layer security even through untrusted intermediaries. DTLS only secures the hop between device and proxy.

Secure boot + firmware signing: The MCU bootloader verifies a signature (ECDSA-P256) on the firmware image before executing it. The signing key is held offline by the manufacturer; the public key is burned into the device's OTP memory. Prevents loading modified or attacker-supplied firmware.

Key management at scale (10,000 devices): Each device should have a unique PSK or certificate — a fleet-wide shared secret is a single point of compromise. A device provisioning service (e.g., AWS IoT, Azure DPS) enrolls each device at manufacture with a unique identity, issues credentials, and manages revocation. Certificate rotation via OTA is possible but requires a robust OTA infrastructure.

Defence in depth: No single control suffices. Combine: physical tamper detection, DTLS/OSCORE for transport/application, secure boot for firmware integrity, minimal attack surface (disable unused peripherals/ports), network segmentation (IoT VLAN), intrusion detection on the gateway.

DA3. Describe Elixir GenStage and how it models a producer–consumer pipeline. Cover: the producer, producer-consumer, and consumer stage types; demand-driven flow control (back-pressure); how Broadway builds on GenStage for production pipelines; and a concrete IoT backend use case where a GenStage pipeline processes incoming sensor telemetry — ingesting from MQTT, enriching with metadata, and writing to a database — with back-pressure preventing database overload.

GenStage models data processing as a graph of stages with explicit demand-driven flow control. Three stage types:

- **Producer:** generates or receives events; only emits when downstream asks (`handle_demand/2`).
- **Producer-consumer:** receives events from upstream, processes them, and re-emits to downstream.
- **Consumer:** receives events and performs side effects (write to DB, publish MQTT); never emits.

Back-pressure: consumers ask for N events (demand); the producer emits at most N . If the database is slow, the consumer reduces demand → the pipeline slows → the MQTT broker's queue absorbs the backlog rather than RAM in the pipeline. This prevents unbounded buffering and out-of-memory crashes.

Broadway builds on GenStage, adding: batch processing (accumulate N messages before writing), acknowledgement tracking (ack each message to the source only after successful processing), configurable concurrency per stage, telemetry, and built-in failure handling. Data sources: RabbitMQ, SQS, MQTT (via custom producer), Kafka.

Concrete IoT pipeline:

MQTT Broker → [Broadway Producer] → [Enrichment Stage] → [DB Writer Consumer]

1. Broadway Producer subscribes to the MQTT broker, receives sensor payloads as messages.
2. Enrichment Producer-Consumer joins the device_id with metadata from ETS (location, calibration coefficients) — in-memory, fast.
3. DB Writer Consumer batches 100 readings and does a bulk INSERT into TimescaleDB. Acknowledges the batch to the MQTT broker only on successful INSERT.
4. If the DB is slow, back-pressure reduces demand from the MQTT producer, which lets the broker queue up — no messages are lost and the pipeline self-throttles. If the DB is down, messages wait in the broker (persistent session); the pipeline resumes on reconnection.

Section 5 — Design Question

Question 26 — Smart Waste Management

(answer not found)

A city waste management authority wants to deploy **IoT fill-level sensors** in **2,000 public waste bins** across the city to optimise collection routes. Requirements:

- Each bin has an ultrasonic fill-level sensor, reporting **fill percentage** (0–100%).
- Bins report their level **every 15 minutes** under normal conditions, but must report **immediately** when fill level crosses **80%**.
- The backend must compute **optimised collection routes** for the fleet of 30 garbage trucks, updated every hour.
- Data must be stored for **3 years** for waste management planning.
- Bins are street-furniture — powered by a **small solar panel + battery**. No wired connectivity.
- The city already has a LoRaWAN network deployed with 20 gateways.

Section 3 — Short Answer

Subquestion 26.1 (short answer)

Given the existing LoRaWAN infrastructure, evaluate whether it meets the requirements. What is the key challenge with 2,000 nodes all potentially triggering alert uplinks simultaneously during a large public event, and how do you mitigate it?

LoRaWAN suitability: 20 gateways in a city comfortably cover 2,000 bins. At SF9/125 kHz, a 10-byte payload (fill level + bin ID + timestamp) has ToA ≈ 185 ms. At one uplink per 15 minutes: duty cycle ≈ 0.02% per bin — well within the 1% EU868 limit. Normal operation poses no congestion issue.

Event storm problem: If a large public event (concert, market) fills many bins simultaneously, hundreds of bins may trigger the 80%-threshold alert within minutes. With 500 alert uplinks in a 2-minute window across 8 sub-bands, collision rates rise sharply — LoRaWAN's ALOHA MAC has no CSMA. Probability of collision approaches 1 as node density grows.

Mitigations:

	1. Randomised alert delay: When a bin crosses 80%, it waits a random back-off (0–120 s) before sending the alert uplink. Spreads the storm. 2. ADR to lower spreading factors: Bins in good coverage use SF7/SF8, reducing ToA to < 60 ms and increasing channel capacity ~8×. 3. Multiple sub-bands: LoRaWAN uses 8 EU868 channels; distributing load across all channels helps. 4. Threshold hysteresis: Alert only when level crosses 80% upward; suppress re-alerts until level drops below 70% — prevents repeated alerts from a bin that oscillates around the threshold.
Subquestion 26.2 (short answer) Describe how the backend determines optimal truck routes from the fill-level data. What data does the route optimisation algorithm need, and how is the result delivered to truck drivers?	The backend models the city as a graph of bin locations. Each hour it queries the database for the latest fill level per bin. Bins above a priority threshold (> 60% full, or > 80% alert) are flagged for collection; bins below 40% are skipped. The route optimiser solves a Vehicle Routing Problem (VRP): assign bins to trucks and order collection stops to minimise total travel distance/time, subject to truck capacity (weight) and shift duration. Inputs: bin GPS coordinates, fill levels, truck depot location, truck count (30). A practical approach uses a heuristic solver (Clarke-Wright savings algorithm or Google OR-Tools) since exact VRP is NP-hard. Result delivery: the backend publishes the route as a JSON payload (ordered list of bin IDs + GPS coordinates) to a driver mobile app (via REST or MQTT). The app displays a turn-by-turn route on a map. If a driver reports a bin already emptied (out-of-band), the backend updates the bin's state and re-optimises if needed.
Section 4 — Detailed Answer	
Subquestion 26.3 (detailed answer) Design the full system: bin sensor → LoRaWAN → backend → route optimisation → driver app. Cover the normal reporting cadence, the 80% threshold alert path, how the system handles bins that have been offline for > 2 hours (battery depleted or vandalism), the data storage and retention strategy, and how the system scales if the city doubles its bin count to 4,000.	Bin sensor node: Ultrasonic sensor (e.g., HC-SR04 or JSN-SR04T) measures distance to waste surface; microcontroller converts to fill percentage (full = distance < threshold, empty = near max). Node wakes every 15 minutes (RTC alarm), samples sensor, sends LoRaWAN Class A uplink (SF9, confirmed uplink with ACK to detect repeated failures), then sleeps. On fill > 80%: immediately wakes and sends an alert uplink (out-of-schedule). Stores last-sent level in NVS to avoid duplicate alerts on wake. LoRaWAN → Application Server: Gateways forward all uplinks to the Network Server (ChirpStack). Application Server decodes payloads, writes to TimescaleDB with bin ID, fill %, and receive timestamp. Alert uplinks are flagged and published to a dedicated MQTT topic <code>bins/alerts/{bin_id}</code> for fast downstream processing. Alert pipeline: A lightweight consumer (Elixir GenServer) subscribes to <code>bins/alerts/#</code>. On receipt, it immediately marks the bin as high-priority in a Redis sorted set (keyed by fill level) and triggers a route re-optimisation job. Alert-to-driver latency: LoRa uplink (~1 s) + NS processing (~1 s) + alert consumer (~1 s) + route update (~5 s) = ~8 s total — well within any reasonable operational SLA.

Offline bin handling: The backend tracks the expected-uplink time per bin (last received + 15 min + tolerance). If a bin misses 8 consecutive uplinks (2 hours), it is flagged offline. Possible causes: battery depleted (cloudy week), sensor failure, vandalism. The route optimiser treats offline bins as unknown-fill and includes them in a "check and empty" category with lower priority. A maintenance alert is sent to the operations dashboard. On reconnection, the bin resumes normal reporting; no gap-fill — the fill level is simply unknown during the outage.

Storage and retention: Each uplink is ~20 bytes of payload. 2,000 bins × 96 uplinks/day = 192,000 records/day ≈ 3.8 MB/day ≈ 1.4 GB/year. Store full-resolution data for 6 months in TimescaleDB. Downsample to daily min/max/avg per bin for 3 years (< 50 MB). Raw data archived to object storage (S3) for the full 3 years for planning analysis.

Scaling to 4,000 bins: LoRaWAN capacity scales well — 4,000 bins at SF9 still well within EU duty cycle limits. The Network Server and Application Server are horizontally scalable (add ingestion worker replicas). TimescaleDB handles the ~3 GB/year load trivially with proper hypertable partitioning. The VRP solver runtime grows (4,000 bins vs 2,000), but hourly re-optimisation with a fast heuristic (< 10 s) remains feasible. Add gateways if coverage gaps emerge with denser deployment in new city areas.

Practice Exam 13

Question

Model Answer

Section 1 — Multiple Choice

MC1. What does the following code print?

```
int arr[] = {10, 20, 30, 40};  
int *p = arr + 1;  
printf("%d %d", *p, *(p + 2));
```

- A) 10 30
- B) 20 40
- C) 20 30
- D) Undefined behaviour — pointer arithmetic past `arr[1]` is illegal

B

`p` points to `arr[1]` (value 20). `*(p+2)` points to `arr[3]` (value 40). Both within array bounds — defined behaviour.

MC2. What is `sizeof(s)` for the following struct on a typical 32-bit embedded compiler with default alignment?

```
struct s {  
    uint8_t a;  
    uint32_t b;  
    uint8_t c;  
};
```

C

`a` (1 byte) + 3 bytes padding + `b` (4 bytes) + `c` (1 byte) + 3 bytes padding = 12 bytes. The struct is padded so `b` is 4-byte aligned and the struct size is a multiple of the largest alignment (4).

• A) 6 bytes • B) 8 bytes • C) 12 bytes • D) 4 bytes	
MC3. What does this preprocessor macro expand to when called as <code>MAX(x++, y)</code>? <pre>#define MAX(a, b) ((a) > (b) ? (a) : (b))</pre> • A) It evaluates <code>x++</code> exactly once and returns the larger value • B) It may evaluate <code>x++</code> twice, incrementing <code>x</code> an extra time if <code>x > y</code> • C) It is a compile-time error because macros cannot use the ternary operator • D) It always increments <code>x</code> once regardless of which branch is taken MC4. In FreeRTOS, a task calls <code>xQueueSend(queue, &data, 0)</code> with a timeout of 0. The queue is currently full. What happens? • A) The task blocks until space is available • B) The function returns <code>errQUEUE_FULL</code> immediately without blocking • C) The oldest item is overwritten with the new data • D) A FreeRTOS assertion fires and the system halts	B The macro expands to <code>((x++) > (y) ? (x++) : (y))</code>. If <code>x > y</code>, <code>x++</code> is evaluated twice — once in the condition and once for the result — incrementing <code>x</code> by 2. This is a classic macro pitfall. B Timeout of 0 means non-blocking. If the queue is full, <code>xQueueSend</code> returns <code>errQUEUE_FULL</code> immediately.
Section 2 — Multiple Answer	
MA1. Which of the following C patterns are correct for safely working with hardware registers on an MCU? • A) Declaring the register pointer as <code>volatile uint32_t *reg = (volatile uint32_t *)0x40020000;</code> • B) Using <code>reg = (1 << 5);</code> to set bit 5 without disturbing other bits • C) Using <code>reg = 0x20;</code> to set only bit 5, which clears all other bits — acceptable for write-only status-clear registers • D) Reading <code>reg</code> in a tight loop without <code>volatile</code> to check a ready bit — the compiler may optimise this to one read • E) Using a <code>#define</code> for the address: <code>#define GPIOA_ODR ((volatile uint32_t *)0x40020014)</code> and then <code>*GPIOA_ODR = (1<<5);</code> MA2. Which of the following are correct FreeRTOS patterns? • A) A task must never return; it should contain an infinite loop or call <code>vTaskDelete(NULL)</code> at the end • B) <code>xSemaphoreGiveFromISR</code> must be called instead of <code>xSemaphoreGive</code> when releasing a semaphore from an interrupt handler • C) <code>taskENTER_CRITICAL()</code> / <code>taskEXIT_CRITICAL()</code> disable all interrupts for the duration of the critical section	MA1 D is incorrect: reading a <code>volatile</code>-less register in a loop is a bug — the compiler may hoist the read outside the loop, so the loop never sees the hardware updating the register. Always declare hardware registers as <code>volatile</code>. MA2 C is partially wrong: <code>taskENTER_CRITICAL()</code> disables interrupts up to <code>configMAX_SYSCALL_INTERRUPT_PRIORITY</code>, not necessarily all interrupts — high-priority non-maskable interrupts still run. D is wrong: in FreeRTOS, a higher numerical value means higher priority for tasks, which is correct — but this is a common confusion point. Wait — actually this IS correct in FreeRTOS (priority 5 > priority 1). So D is correct. Let me re-evaluate: A, B, D, E are correct. C is the nuanced one — it disables interrupts at or below the syscall priority threshold, not necessarily all interrupts. Correct answers: A, B, D, E

- D) A higher numerical priority value means a higher-priority task in FreeRTOS
- E) `vTaskDelay(pdMS_TO_TICKS(100))` puts the calling task to sleep for approximately 100 milliseconds

Section 3 — Short Answer

SA1. What does the following code print? Explain each line.

```
int x = 5;
int *p = &x;
*p = 10;
int **pp = &p;
**pp = 20;
printf("%d\n", x);
```

20

Line by line:

- `int x = 5` — `x` is 5.
- `int *p = &x` — `p` holds the address of `x`.
- `*p = 10` — dereferences `p` and writes 10 into `x`. Now `x = 10`.
- `int **pp = &p` — `pp` holds the address of the pointer `p`.
- `**pp = 20` — dereferences `pp` to get `p`, then dereferences `p` to write 20 into `x`. Now `x = 20`.
- `printf` prints 20.

SA2. The following code has a bug. Identify it and write the corrected version.

```
char *make_greeting(const char *name) {
    char buf[64];
    snprintf(buf, sizeof(buf), "Hello, %s!", name);
    return buf;
}
```

Bug: `buf` is a local (stack-allocated) array. Returning a pointer to it is undefined behaviour — the stack frame is invalidated on function return, and the caller's pointer is dangling.

Corrected version using heap allocation:

```
char *make_greeting(const char *name) {
    char *buf = malloc(64);
    if (!buf) return NULL;
    snprintf(buf, 64, "Hello, %s!", name);
    return buf; // caller must free()
}
```

Or using a caller-provided buffer (preferred in embedded code to avoid heap):

```
void make_greeting(const char *name, char *buf, size_t len)
{
    snprintf(buf, len, "Hello, %s!", name);
}
```

SA3. Write a C struct using bitfields that maps to this 8-bit protocol flag byte:

```
Bit 7:  direction (1=TX, 0=RX)
Bits 6-4: channel (0-7)
Bit 3:  ack_required
Bits 2-0: priority (0-7)
```

```
typedef struct __attribute__((packed)) {
    uint8_t priority    : 3; /* bits 2:0 */
    uint8_t ack_required : 1; /* bit 3   */
    uint8_t channel     : 3; /* bits 6:4 */
    uint8_t direction   : 1; /* bit 7   */
} FrameFlags;
```

Add `__attribute__((packed))` and explain why it is needed here.

`__attribute__((packed))` removes any inter-field padding that the compiler might insert for alignment. Without it, the compiler is free to add padding bytes between bitfield groups, making the struct's memory layout differ from the on-wire byte. With `packed`, the entire struct fits in exactly 1 byte matching the protocol definition.

Note: bitfield ordering within a byte is compiler/platform-dependent (LSB-first on most ARM GCC targets). Verify with a static assert: `_Static_assert(sizeof(FrameFlags) == 1, "size mismatch");`

SA4. Write a FreeRTOS ISR handler for a UART receive-complete interrupt. The ISR should post the received byte (`uint8_t rx_byte`) to a queue named `uart_rx_queue`. Include the `xHigherPriorityTaskWoken` pattern and the `yield` call.

```
/* Declared globally or accessible in scope */
extern QueueHandle_t uart_rx_queue;

void UART_RX_IRQHandler(void) {
    uint8_t rx_byte = UART->DR; /* read received byte from hardware */

    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    xQueueSendFromISR(uart_rx_queue, &rx_byte, &xHigherPriorityTaskWoken);

    /* If posting unblocked a higher-priority task, yield to it */
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
```

Key points: use `xQueueSendFromISR` (not `xQueueSend`) inside an ISR. Pass the address of `xHigherPriorityTaskWoken`; if posting woke a task with higher priority than the current running task, `portYIELD_FROM_ISR` triggers a context switch immediately on ISR exit rather than waiting for the next tick.

SA5. What does the following function do? What is the value of `result` when called as `flags_set(0b10110011, 0b00100001)`?

```
uint8_t flags_set(uint8_t reg, uint8_t mask) {
    return (reg & mask) == mask;
}
```

The function checks whether **all bits set in mask are also set in reg**. It returns 1 (true) if every bit in `mask` is present in `reg`, 0 otherwise.

`flags_set(0b10110011, 0b00100001):`

- `0b10110011 & 0b00100001 = 0b00100001`
- `0b00100001 == 0b00100001 → true → returns 1.`

Both bits in the mask (bit 5 and bit 0) are set in `reg`, so the result is 1.

SA6. Write a C function `swap(int *a, int *b)` that swaps two integers using pointers (not XOR). Then write a second version using a macro. Explain one risk of the macro version.

```
/* Pointer version */
void swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
}
```

SA7. Complete this FreeRTOS task so it reads temperature readings from `temp_queue` and turns on an LED (by calling `led_on()`) whenever the temperature exceeds 50. The task should block indefinitely waiting for queue items.

```
void temp_monitor_task(void *pvParameters) {
    float temperature;

    for (;;) {
        // YOUR CODE HERE
    }
}
```

SA8. Write the `goto`-based structured cleanup pattern in C for a function that: (1) allocates a buffer with `malloc`, (2) opens a file with `fopen`, and (3) acquires a mutex with `xSemaphoreTake`. Ensure all resources are released on any error path.

```
*b = tmp;
}

/* Macro version */
#define SWAP(a, b) do { \
    __typeof__(a) _tmp = (a); \
    (a) = (b); \
    (b) = _tmp; \
} while (0)
```

Risk of a simpler macro like `#define SWAP(a,b) { int t=a; a=b; b=t; }`: (1) The type is hardcoded to `int` — fails for other types. (2) Without the `do { } while(0)` wrapper, using it in an `if` without braces can cause logic errors. (3) If `a` or `b` are expressions with side effects, they may be evaluated multiple times.

```
void temp_monitor_task(void *pvParameters) {
    float temperature;
    for (;;) {
        if (xQueueReceive(temp_queue, &temperature,
            portMAX_DELAY) == pdTRUE) {
            if (temperature > 50.0f) {
                led_on();
            } else {
                led_off(); /* optional: turn off when
                    below threshold */
            }
        }
    }
}
```

`portMAX_DELAY` blocks indefinitely until an item arrives — no busy-waiting. The task yields the CPU to other tasks while waiting.

```
int do_work(const char *filename) {
    int ret = 0;
    uint8_t *buf = NULL;
    FILE *fp = NULL;

    buf = malloc(256);
    if (!buf) { ret = -1; goto cleanup; }

    fp = fopen(filename, "r");
    if (!fp) { ret = -2; goto cleanup; }
```

```

    if (xSemaphoreTake(my_mutex, pdMS_TO_TICKS(100)) !=
pdTRUE) {
        ret = -3; goto cleanup;
    }

    /* ... do actual work ... */

    xSemaphoreGive(my_mutex);

cleanup:
    if (fp) fclose(fp);
    if (buf) free(buf);
    return ret;
}

```

The goto cleanup pattern ensures every acquired resource is released in reverse order of acquisition, without deeply nested if/else. Each error path jumps directly to cleanup, which conditionally frees only what was successfully acquired (null-checks guard against double-free).

Section 4 — Detailed Answer

DA1. Write a complete FreeRTOS producer/consumer system. The producer task reads an ADC value every 200 ms and sends it to a queue. The consumer task waits on the queue and computes a running average of the last 8 samples, printing the average via printf. Requirements:

- Use xQueueCreate with depth 8.
- Use pdMS_TO_TICKS for the delay.
- The consumer should not busy-wait.
- Show main() creating both tasks and starting the scheduler.

```

#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include <stdio.h>
#include <stdint.h>

#define QUEUE_DEPTH 8
static QueueHandle_t adc_queue;

/* Simulated ADC read */
static uint16_t adc_read(void) {
    /* On real hardware: start conversion, wait, read result
register */
    return (uint16_t)(rand() % 4096);
}

/* Producer: samples ADC every 200 ms */
void producer_task(void *pvParameters) {
    (void)pvParameters;
    for (;;) {
        uint16_t sample = adc_read();
        xQueueSend(adc_queue, &sample, portMAX_DELAY);
    }
}

```

```

        vTaskDelay(pdMS_TO_TICKS(200));
    }
}

/* Consumer: computes running average of last 8 samples */
void consumer_task(void *pvParameters) {
    (void)pvParameters;
    uint16_t samples[8] = {0};
    uint8_t idx = 0;
    uint8_t count = 0;
    uint16_t sample;

    for (;;) {
        /* Block indefinitely until producer sends a sample */
        if (xQueueReceive(adc_queue, &sample,
portMAX_DELAY) == pdTRUE) {
            samples[idx] = sample;
            idx = (idx + 1) % 8;
            if (count < 8) count++;

            uint32_t sum = 0;
            for (uint8_t i = 0; i < count; i++) sum +=
samples[i];
            printf("Running avg (%u samples): %lu\n",
count, sum / count);
        }
    }
}

int main(void) {
    adc_queue = xQueueCreate(Queue_DEPTH,
sizeof(uint16_t));
    configASSERT(adc_queue != NULL);

    xTaskCreate(producer_task, "Producer", 256, NULL, 2,
NULL);
    xTaskCreate(consumer_task, "Consumer", 512, NULL, 1,
NULL);

    vTaskStartScheduler();
    for (;;) /* never reached */
}

```

Key decisions: queue depth of 8 means the producer can run up to $8 \times 200 \text{ ms} = 1.6 \text{ s}$ ahead without blocking if the consumer is slow. Producer has higher priority ($2 > 1$) so it preempts the consumer, ensuring samples are taken on time. Consumer uses `portMAX_DELAY` — no CPU wasted polling.

DA2. Write a C state machine for a simple door lock controller using function pointers. States: LOCKED, UNLOCKING, OPEN, ALARMED. Events: correct_pin, wrong_pin, door_opened, door_closed, timeout.

Requirements:

- Define a State typedef and an Event typedef as enums.
- Define a transition table as a 2D array of function pointers void (*action)(void) and next-state values.
- Write process_event(Event e) that looks up the current state + event and calls the action + transitions.
- Implement at least two action functions (unlock_door, trigger_alarm).

```
#include <stdio.h>

#include <stdint.h>

typedef enum { LOCKED, UNLOCKING, OPEN, ALARMED, STATE_COUNT } State;

typedef enum { CORRECT_PIN, WRONG_PIN, DOOR_OPENED, DOOR_CLOSED, TIMEOUT, EVENT_COUNT } Event;

/* Action function type */
typedef void (*ActionFn)(void);

/* Transition table entry */
typedef struct {
    ActionFn action;
    State next_state;
} Transition;

/* Action implementations */
static void do_nothing(void) { }
static void unlock_door(void) { printf("Door unlocking...\n"); }
static void lock_door(void) { printf("Door locked.\n"); }
static void trigger_alarm(void) { printf("ALARM! Wrong PIN!\n"); }
static void open_confirmed(void) { printf("Door open.\n"); }

/* Transition table [current_state][event] */
static const Transition table[STATE_COUNT][EVENT_COUNT] = {
/*
CORRECT_PIN      WRONG_PIN
DOOR_OPENED
DOOR_CLOSED      TIMEOUT      */
/* LOCKED */ { {unlock_door, UNLOCKING},
               {trigger_alarm, ALARMED}, {do_nothing, LOCKED},
               {do_nothing, LOCKED}, {do_nothing, LOCKED} },
/* UNLOCKING */ { {do_nothing, UNLOCKING},
                  {do_nothing, UNLOCKING}, {open_confirmed, OPEN},
                  {do_nothing, UNLOCKING}, {lock_door, LOCKED} },
/* OPEN */ { {do_nothing, OPEN},
              {do_nothing, OPEN}, {do_nothing, OPEN},
              {lock_door, LOCKED}, {do_nothing, OPEN} },
/* ALARMED */ { {do_nothing, ALARMED},
                 {do_nothing, ALARMED}, {do_nothing, ALARMED},
                 {do_nothing, ALARMED}, {lock_door, LOCKED} },
};

static State current_state = LOCKED;
```

```

void process_event(Event e) {
    const Transition *t = &table[current_state][e];
    if (t->action) t->action();
    current_state = t->next_state;
}

int main(void) {
    process_event(WRONG_PIN);    /* LOCKED → alarm */
    process_event(TIMEOUT);      /* ALARMED → locked */
    process_event(CORRECT_PIN); /* LOCKED → unlocking */
    process_event(DOOR_OPENED); /* UNLOCKING → open */
    process_event(DOOR_CLOSED); /* OPEN → locked */
    return 0;
}

```

The 2D table encodes the entire machine declaratively. `process_event` is one array lookup + one indirect function call — $O(1)$, no switch statement. Adding a state or event requires adding a row/column to the table, not modifying `process_event`. `do_nothing` fills "no-op" transitions cleanly without NULL checks.

Practice Exam 14

Question

Model Answer

Section 1 — Multiple Choice

MC1. What does the following Elixir expression return?

```

x = 5
{a, b} = {x + 1, x * 2}
a + b

```

- A) {6, 10}
- B) 16
- C) 11
- D) A MatchError because x is already bound

MC2. What does this pipeline return?

```

[1, 2, 3, 4, 5]
|> Enum.filter(fn x -> rem(x, 2) == 0 end)
|> Enum.map(fn x -> x * x end)
|> Enum.sum()

```

- A) 20
- B) 55
- C) [4, 16]

B

$a = 5+1 = 6$, $b = 5*2 = 10$, $a + b = 16$. Pattern matching destructures the tuple; both bindings succeed.

A

Filter keeps [2, 4]. Map squares to [4, 16]. Sum = 20.

• D) 4

MC3. What happens when this with expression is evaluated with input = "abc"?

```
with {:ok, n} <- Integer.parse(input),
  true   <- n > 0 do
  {:result, n}
else
  :error   -> {:error, :not_a_number}
  false    -> {:error, :not_positive}
  {n, _rest} -> {:result, n}
end
```

- A) {:result, 0}
- B) {:error, :not_a_number}
- C) {:error, :not_positive}
- D) A MatchError because Integer.parse does not return :error

MC4. In a GenServer, handle_call/3 must return a tuple. Which of the following is the correct return to reply with a value and keep the existing state?

- A) {:reply, value, state}
- B) {:ok, value, state}
- C) {:noreply, value}
- D) {:reply, value}

B

Integer.parse("abc") returns :error (not {n, rest}). The else clause matches :error -> {:error, :not_a_number}.

A

{:reply, value, state} is the 3-tuple reply: send value to caller, keep state as new state.

Section 2 — Multiple Answer

MA1. Which of the following Elixir pattern matches succeed (do not raise MatchError)?

- A) {:ok, x} = {:ok, 42} — binds x to 42
- B) [h | t] = [1, 2, 3] — binds h to 1 and t to [2, 3]
- C) %{a: x} = %{a: 1, b: 2} — binds x to 1 (map match allows extra keys)
- D) {a, a} = {1, 2} — binds a to 1 then 2
- E) "hello " <> rest = "hello world" — binds rest to "world"

MA2. Which of the following are valid GenServer callback return values?

- A) {:noreply, new_state} from handle_cast/2 — updates state, no reply sent
- B) {:reply, value, new_state} from handle_call/3 — sends reply and updates state
- C) {:stop, :normal, state} from handle_info/2 — stops the server cleanly
- D) {:ok, initial_state} from init/1 — starts the server
- E) {:noreply, state, {:continue, :init_data}} from init/1 — schedules a handle_continue callback

MA1

D fails: {a, a} = {1, 2} — a is bound to 1 on the left element, then the right element 2 must equal 1 (already-bound a). It does not, so MatchError.

MA2

E is not standard for init/1 — {:continue, ...} is returned as {:ok, state, {:continue, :init_data}} from init/1, which is valid. So E is also correct.

Correct answers: **A, B, C, D, E**

Section 3 — Short Answer

SA1. What does the following Elixir function return when called as `mystery([3, 1, 4, 1, 5, 9, 2])`? Trace through the logic.

```
def mystery(list) do
  list
  |> Enum.filter(fn x -> x > 3 end)
  |> Enum.map(fn x -> x * 2 end)
  |> Enum.reduce(0, fn x, acc -> acc + x end)
end
```

Input: [3, 1, 4, 1, 5, 9, 2]

Filter `x > 3`: [4, 5, 9]

Map `x * 2`: [8, 10, 18]

Sum: 36

Returns **36**.

SA2. The following `GenServer` `handle_call` has a bug. Identify it and fix it.

```
def handle_call(:get_count, _from, state) do
  {:reply, state.count}
end
```

Bug: `{:reply, state.count}` is a 2-tuple. `handle_call/3` must return a 3-tuple `{:reply, reply_value, new_state}`. The missing third element (the state) causes a `FreeRTOS` — wait, causes a crash in `GenServer` (it receives a badly-shaped return and raises).

```
def handle_call(:get_count, _from, state) do
  {:reply, state.count, state}
end
```

SA3. Write an Elixir function `count_by_type/1` that takes a list of `{:sensor, id}` or `{:actuator, id}` tuples and returns a map `%{sensor: n, actuator: m}` with the counts of each type. Use `Enum.reduce/3`.

```
def count_by_type(devices) do
  Enum.reduce(devices, %{sensor: 0, actuator: 0}, fn
    {:sensor, _id}, acc -> Map.update!(acc, :sensor,
      &(&1 + 1))
    {:actuator, _id}, acc -> Map.update!(acc, :actuator,
      &(&1 + 1))
  end)
end
```

`Enum.reduce/3` threads an accumulator map through the list. Each clause pattern-matches on the tuple tag and increments the relevant counter using `Map.update!/3` with a capture shorthand.

SA4. Complete this `GenServer` `init/1` so that it starts a periodic 5-second tick using `Process.send_after`, and write the `handle_info/2` clause that handles `:tick`, increments a counter in state, and reschedules the next tick.

```
defmodule Ticker do
  use GenServer

  def init(_) do
    # YOUR CODE HERE
  end
end
```

```
defmodule Ticker do
  use GenServer

  @interval 5_000

  def init(_) do
    Process.send_after(self(), :tick, @interval)
    {:ok, %{count: 0}}
  end
end
```

```
def handle_info(:tick, state) do
  # YOUR CODE HERE
end
```

SA5. What does this Elixir code print, and why?

```
pid = spawn(fn ->
  receive do
    {:hello, from} -> send(from, :world)
  end
end)

send(pid, {:hello, self()})

receive do
  msg -> IO.inspect(msg)
end
```

SA6. Write an Elixir function using for comprehension that takes a list of sensor readings `[{:temp, 22.5}, {:humidity, 60}, {:temp, 24.1}, {:humidity, 55}]` and returns a map where each key is a sensor type and the value is a list of all readings for that type: `%{temp: [22.5, 24.1], humidity: [60, 55]}`.

```
def handle_info(:tick, state) do
  new_state = Map.update!(state, :count, &(&1 + 1))
  IO.puts("Tick #{new_state.count}")
  Process.send_after(self(), :tick, @interval)
  {noreply, new_state}
end
```

`init/1` schedules the first tick immediately. Each `handle_info(:tick, ...)` increments the counter, does work, then reschedules the next tick. The actual interval is `@interval + processing_time` (forward drift). Use `:timer.send_interval/2` if you need a fixed wall-clock cadence.

Prints: `:world`

Trace:

1. `spawn` creates a new process that waits in receive for `{:hello, from}`.
2. `send(pid, {:hello, self()})` sends the message; `from` is bound to the caller's PID.
3. The spawned process runs `send(from, :world)` — sends `:world` back to the caller.
4. The caller's receive matches any msg, binds to `:world`, and `IO.inspect` prints `:world`.

```
def group_by_type(readings) do
  Enum.group_by(readings, fn {type, _val} -> type end, fn
    {_type, val} -> val end)
end
```

Or with a for comprehension building a map explicitly:

```
def group_by_type(readings) do
  types = readings |> Enum.map(fn {t, _} -> t end) |>
    Enum.uniq()

  for type <- types, into: %{} do
    vals = for {t, v} <- readings, t == type, do: v

    {type, vals}
  end
end
```

	<pre>end</pre> Enum.group_by/3 is idiomatic and more concise. The for version makes the grouping logic explicit.
SA7. Fix the supervision tree below so that crashing the :sensor worker does not restart the :logger worker. <pre>children = [{SensorWorker, name: :sensor}, {LoggerWorker, name: :logger}] Supervisor.start_link(children, strategy: :one_for_all)</pre> SA8. Write an Elixir module that uses ETS to store and look up device state. Provide: start/0 (creates the table), put/2 (stores {device_id, state}), and get/1 (returns {:ok, state} or :not_found).	Change :one_for_all to :one_for_one: <pre>children = [{SensorWorker, name: :sensor}, {LoggerWorker, name: :logger}] Supervisor.start_link(children, strategy: :one_for_one)</pre> :one_for_all restarts all children when any one crashes — inappropriate here since the logger is independent of the sensor. :one_for_one restarts only the crashed child, leaving others untouched. Use :one_for_all only when children have shared state dependencies that would be inconsistent if only one is restarted. <pre>defmodule DeviceStore do @table :device_store def start do :ets.new(@table, [:named_table, :public, :set]) end def put(device_id, state) do :ets.insert(@table, {device_id, state}) end def get(device_id) do case :ets.lookup(@table, device_id) do [{^device_id, state}] -> {:ok, state} [] -> :not_found end end end</pre> :named_table makes the table accessible by name from any process. :set means one entry per key. The ^device_id pin in the match ensures we match on the exact key we looked up, not any key.

Section 4 — Detailed Answer

DA1. Write a complete GenServer module called DeviceRegistry for an IoT backend. It must support:

- start_link/0 — starts the server.

```
defmodule DeviceRegistry do
```

- `register(device_id, info)` — stores device info (a map) keyed by device ID.
- `lookup(device_id)` — returns `{:ok, info}` or `{:error, :not_found}`.
- `list_all/0` — returns a list of all `{device_id, info}` pairs.
- `unregister(device_id)` — removes a device.

Use a Map as the internal state. Show `handle_call` implementations for each operation.

```
use GenServer

# Public API

def start_link(_opts \ \ []), do:
  GenServer.start_link(__MODULE__, %{}, name: __MODULE__)

def register(device_id, info), do:
  GenServer.call(__MODULE__, {:register, device_id, info})

def lookup(device_id), do:
  GenServer.call(__MODULE__, {:lookup, device_id})

def list_all, do:
  GenServer.call(__MODULE__, :list_all)

def unregister(device_id), do:
  GenServer.call(__MODULE__, {:unregister, device_id})

# Callbacks

def init(initial_state), do: {:ok, initial_state}

def handle_call({:register, id, info}, _from, state) do
  {:reply, :ok, Map.put(state, id, info)}
end

def handle_call({:lookup, id}, _from, state) do
  result = case Map.fetch(state, id) do
    {:ok, info} -> {:ok, info}
    :error      -> {:error, :not_found}
  end
  {:reply, result, state}
end

def handle_call(:list_all, _from, state) do
  {:reply, Map.to_list(state), state}
end

def handle_call({:unregister, id}, _from, state) do
  {:reply, :ok, Map.delete(state, id)}
end

end
```

All operations are synchronous calls — callers get immediate confirmation. State is a plain Map keyed by device ID. `Map.fetch/2` returns `{:ok, v}` or `:error`, enabling the `{:error, :not_found}` idiom without exceptions. `name: __MODULE__` registers the process globally so clients don't need to pass the PID.

DA2. Write a complete Elixir module `SensorPipeline` that:

1. Accepts a list of raw sensor reading maps: `[%{id: "s1", type: :temp, value: 22.5, unit: :celsius}, ...]`
2. Filters out readings where value is nil or outside the range `[-40, 125]` for `:temp` or `[0, 100]` for `:humidity`.
3. Converts all `:temp` readings from Celsius to Fahrenheit: $F = C * 9/5 + 32$.

```
defmodule SensorPipeline do

  @valid_ranges %{temp: {-40, 125}, humidity: {0, 100}}
```

4. Groups the valid, converted readings by type.

5. Returns %{temp: [...], humidity: [...]}.

Write using a pipeline of Enum functions. Then write a second version using a for comprehension for step 2 (filtering). Explain the trade-offs between the two approaches.

```
defp valid?(%{type: type, value: value}) when not
  is_nil(value) do
  case Map.get(@valid_ranges, type) do
    {lo, hi} -> value >= lo and value <= hi
    nil      -> false
  end
end
```

```
defp valid?(_, do: false)
```

```
defp convert(%{type: :temp, value: c} = r), do: %{r |
  value: c * 9 / 5 + 32, unit: :fahrenheit}
```

```
defp convert(r), do: r
```

```
# Version 1: Enum pipeline
```

```
def process(readings) do
```

```
  readings
```

```
  |> Enum.filter(&valid?/1)
```

```
  |> Enum.map(&convert/1)
```

```
  |> Enum.group_by(& &1.type)
```

```
end
```

```
# Version 2: for comprehension for filtering step
```

```
def process_for(readings) do
```

```
  valid = for r <- readings, valid?(r), do: convert(r)
```

```
  Enum.group_by(valid, & &1.type)
```

```
end
```

```
end
```

Trade-offs:

Enum pipeline: each step processes the full list before the next begins. Very readable — each transformation is a named, composable function. Easy to add, remove, or reorder steps. Standard idiomatic Elixir.

for comprehension: combines filtering and mapping into one pass over the list — slightly more efficient (one traversal instead of two for filter + map). Less composable: the guard and body are co-located, making it harder to extract or test valid? independently. For this simple case the difference is negligible; the Enum pipeline is preferred for clarity. If the list is very large and laziness matters, use Stream.filter |> Stream.map instead of Enum.